

DATA SCIENCE END-TO-END HANDBOOK

Cẩm nang Khoa học Dữ liệu chuẩn Production
từ cơ bản đến nâng cao — kèm code mẫu

Thu thập dữ liệu · EDA · Phân tích thống kê · Tiền xử lý · Chuẩn hóa · Chia dữ liệu
Huấn luyện · Đánh giá & Confusion Matrix · Giải thích mô hình · Trực quan hóa · MLOps

Phiên bản 1.0 — Tháng 10/2026

MỤC LỤC

DATA SCIENCE END-TO-END HANDBOOK

Lời nói đầu

- 0.1. Kế hoạch biên soạn handbook
- 0.2. Vòng đời dự án Data Science (CRISP-DM mở rộng cho Production)
- 0.3. Cấu trúc repository chuẩn Production
- 0.4. Bộ dữ liệu xuyên suốt handbook
- 0.5. Thiết lập môi trường

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN & THIẾT KẾ GIẢI PHÁP

- 1.1. Từ câu hỏi kinh doanh đến bài toán ML
- 1.2. Định nghĩa nhãn và cửa sổ thời gian (tránh leakage ngay từ thiết kế)
- 1.3. Từ chi phí kinh doanh đến metric
- 1.4. Tài liệu thiết kế (Design Doc) một trang

CHƯƠNG 2. THU THẬP DỮ LIỆU (DATA COLLECTION & INGESTION)

- 2.1. Bản đồ nguồn dữ liệu
- 2.2. Đọc file — cơ bản đến tối ưu
- 2.3. Đọc từ cơ sở dữ liệu (SQL)
- 2.4. Thu thập qua API — retry, rate limit, phân trang
- 2.5. Web scraping có trách nhiệm
- 2.6. Streaming (Kafka) — tiêu thụ sự kiện
- 2.7. Data Contract & kiểm tra chất lượng dữ liệu đầu vào
- 2.8. Versioning dữ liệu với DVC
- 2.9. Pitfalls khi thu thập dữ liệu

CHƯƠNG 3. PHÂN TÍCH KHÁM PHÁ DỮ LIỆU (EDA)

- 3.1. Mục tiêu của EDA
- 3.2. Tổng quan nhanh (First look)
- 3.3. Phân tích giá trị thiếu (Missing Values)
- 3.4. Phân tích đơn biến (Univariate)
- 3.5. Phân tích hai biến với target (Bivariate)
- 3.6. Tương quan & đa cộng tuyến
- 3.7. Phát hiện ngoại lai (Outliers)
- 3.8. Phân tích theo thời gian & phát hiện drift sớm
- 3.9. Phát hiện Leakage trong EDA
- 3.10. Mẫu báo cáo EDA gửi stakeholder

CHƯƠNG 4. PHÂN TÍCH DỮ LIỆU & THỐNG KÊ SUY LUẬN

- 4.1. Bốn cấp độ phân tích
- 4.2. Thống kê mô tả theo nhóm & phân tích cohort
- 4.3. Kiểm định giả thuyết — khung tư duy
- 4.4. Bootstrap — khoảng tin cậy cho mọi thống kê
- 4.5. A/B Testing chuẩn production
- 4.6. Hồi quy thống kê để giải thích (statsmodels)
- 4.7. Phân tích chuỗi thời gian cơ bản
- 4.8. Nghịch lý Simpson — bài học kinh điển

CHƯƠNG 5. TIỀN XỬ LÝ DỮ LIỆU & FEATURE ENGINEERING

- 5.1. Nguyên tắc số 1: Fit trên train, Transform trên mọi tập

- 5.2. Làm sạch dữ liệu (Data Cleaning)
- 5.3. Xử lý giá trị thiếu (Imputation)
- 5.4. Xử lý ngoại lai
- 5.5. Mã hóa biến phân loại (Categorical Encoding)
- 5.6. Feature Engineering — nơi tạo ra khác biệt lớn nhất
- 5.7. Pipeline tiền xử lý hoàn chỉnh (production-ready)
- 5.8. Lựa chọn đặc trưng (Feature Selection)
- 5.9. Xử lý mất cân bằng lớp (Imbalanced Data)

CHƯƠNG 6. CHUẨN HÓA & BIẾN ĐỔI DỮ LIỆU (SCALING & TRANSFORMATION)

- 6.1. Khi nào cần chuẩn hóa?
- 6.2. Các phương pháp scaling
- 6.3. Biến đổi phân phối (làm "chuẩn hóa" hình dạng)
- 6.4. Giảm chiều (Dimensionality Reduction)
- 6.5. Ghép nối: scaling khác nhau cho từng nhóm cột
- 6.6. Pitfalls

CHƯƠNG 7. CHIA DỮ LIỆU & CHIẾN LƯỢC VALIDATION

- 7.1. Ba tập dữ liệu và vai trò
- 7.2. Bảng chọn chiến lược chia
- 7.3. Hold-out split cơ bản (stratified)
- 7.4. Out-of-time split — chuẩn cho hầu hết bài toán kinh doanh
- 7.5. Cross-Validation
- 7.6. Nested Cross-Validation — ước lượng không bias khi có tuning
- 7.7. Out-of-Fold (OOF) predictions — nền tảng cho stacking & threshold tuning
- 7.8. Danh mục Data Leakage — kẻ thù số 1

CHƯƠNG 8. HUẤN LUYỆN MÔ HÌNH (MODEL TRAINING)

- 8.1. Lộ trình huấn luyện chuẩn
- 8.2. Chọn thuật toán
- 8.3. Baseline
- 8.4. So sánh nhiều mô hình bằng cùng một CV
- 8.5. Gradient Boosting chuyên sâu
- 8.6. Tối ưu siêu tham số (Hyperparameter Optimization)
- 8.7. Ensemble
- 8.8. Chẩn đoán bằng Learning Curve & Validation Curve
- 8.9. Deep Learning cho dữ liệu bảng (PyTorch)
- 8.10. Script huấn luyện production (kết nối MLflow)

CHƯƠNG 9. ĐÁNH GIÁ MÔ HÌNH: METRICS, CONFUSION MATRIX, THRESHOLD & CALIBRATION

- 9.1. Nguyên tắc chọn metric
- 9.2. Confusion Matrix — nền tảng của mọi metric phân loại
- 9.3. Metric không phụ thuộc ngưỡng
- 9.4. Chọn ngưỡng quyết định (Threshold Tuning)
- 9.5. Metric xếp hạng / kinh doanh: Lift & Gain
- 9.6. Calibration — xác suất có đáng tin không?
- 9.7. Metric hồi quy
- 9.8. Metric phân cụm
- 9.9. Metric truy hồi & xếp hạng (Search, RAG, Recommender)
- 9.10. So sánh mô hình có ý nghĩa thống kê
- 9.11. Phân tích lỗi & đánh giá theo phân khúc (Slice analysis)

9.12. Báo cáo đánh giá cuối cùng trên tập test

CHƯƠNG 10. GIẢI THÍCH MÔ HÌNH (XAI) & FAIRNESS

- 10.1. Vì sao cần giải thích?
- 10.2. Feature importance — các loại và cạm bẫy
- 10.3. SHAP (SHapley Additive exPlanations)
- 10.4. Partial Dependence (PDP), ICE và ALE
- 10.5. Mô hình "glass-box" — giải thích được từ bản chất
- 10.6. Fairness — công bằng giữa các nhóm
- 10.7. Model Card — tài liệu hóa mô hình

CHƯƠNG 11. TRỰC QUAN HÓA DỮ LIỆU & KẾT QUẢ

- 11.1. Nguyên tắc thiết kế biểu đồ
- 11.2. Chọn biểu đồ theo mục đích
- 11.3. Thiết lập style chung cho toàn dự án
- 11.4. Biểu đồ EDA (matplotlib + seaborn)
- 11.5. Dashboard biểu đồ đánh giá mô hình (một hình, 6 panel)
- 11.6. Biểu đồ tương tác với Plotly
- 11.7. Dashboard ứng dụng với Streamlit
- 11.8. Kể chuyện bằng dữ liệu (Data Storytelling)

CHƯƠNG 12. MLOps — ĐƯA MÔ HÌNH VÀO PRODUCTION VÀ GIỮ NÓ ĐÚNG

- 12.1. Mức độ trưởng thành MLOps (theo Google Cloud)
- 12.2. Kiến trúc tham chiếu
- 12.3. Experiment Tracking & Model Registry với MLflow
- 12.4. Testing hệ thống ML
- 12.5. Serving
- 12.6. Đóng gói với Docker
- 12.7. CI/CD với GitHub Actions
- 12.8. Chiến lược triển khai an toàn
- 12.9. Giám sát (Monitoring)
- 12.10. Retraining & Orchestration
- 12.11. Feature Store — chống training-serving skew
- 12.12. Tối ưu hiệu năng serving
- 12.13. Bảo mật, quyền riêng tư & quản trị

CHƯƠNG 13. CHECKLIST PRODUCTION & ANTI-PATTERNS

- 13.1. Checklist tổng hợp trước khi Go-Live
- 13.2. 20 Anti-patterns tôi gặp nhiều nhất
- 13.3. Câu hỏi review mô hình (dành cho Tech Lead / Reviewer)

PHỤ LỤC

- Phụ lục A. Cheatsheet thư viện theo giai đoạn
- Phụ lục B. Bảng chọn metric nhanh
- Phụ lục C. Bảng chọn thuật toán
- Phụ lục D. Thuật ngữ Anh – Việt
- Phụ lục E. Tài liệu tham khảo

DATA SCIENCE END-TO-END HANDBOOK

Cẩm nang Khoa học Dữ liệu chuẩn Production — từ cơ bản đến nâng cao

Phiên bản: 1.0 — 10/2026 **Ngôn ngữ code:** Python ≥ 3.10 **Đối tượng:** Data Analyst, Data Scientist, ML Engineer (Junior $\rightarrow$ Senior), Tech Lead, sinh viên/nghiên cứu sinh ngành Khoa học dữ liệu.

Lời nói đầu

Sau hơn 15 năm làm dữ liệu — từ các mô hình chấm điểm tín dụng chạy batch hằng đêm, hệ thống gợi ý xử lý hàng chục nghìn request mỗi giây, đến các dự án NLP/LLM gần đây — tôi nhận ra một điều: **mô hình hiếm khi là phần khó nhất**. Phần khó nằm ở chỗ đặt đúng bài toán, có dữ liệu sạch và đáng tin, đánh giá trung thực, và giữ cho mô hình *vẫn đúng* sau 6 tháng chạy production.

Cuốn handbook này được viết theo đúng thứ tự một dự án thực tế diễn ra. Mỗi chương gồm:

- 1. **Mục tiêu & khái niệm cốt lõi** — hiểu *tại sao* trước khi biết *làm thế nào*.
- 2. **Kỹ thuật từ cơ bản $\rightarrow$ nâng cao** — kèm tiêu chí lựa chọn.
- 3. **Code mẫu** — viết theo phong cách production (có type hint, hàm tái sử dụng, không rò rỉ dữ liệu).
- 4. **Bẫy thường gặp (Pitfalls)** — những lỗi tôi đã thấy lặp lại ở nhiều team.
- 5. **Checklist** — để review trước khi chuyển sang bước kế tiếp.

0.1. Kế hoạch biên soạn handbook

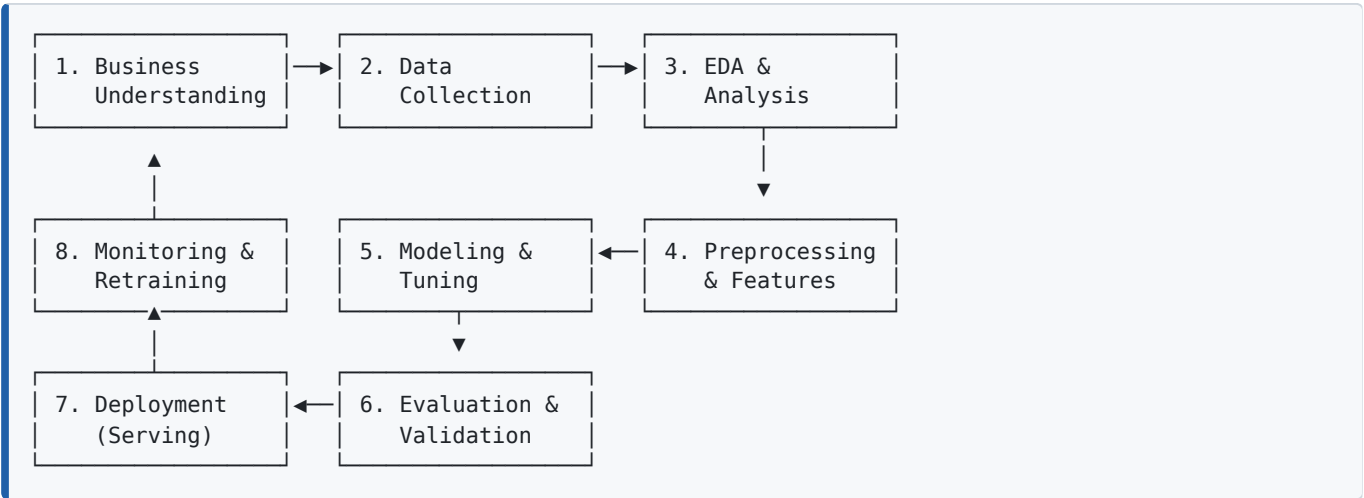
Phần	Chương	Nội dung chính	Thư viện chủ đạo
I. Nền tảng	0	Kế hoạch, vòng đời dự án, cấu trúc repo, môi trường	uv , poetry , git , pre-commit
	1	Định nghĩa bài toán & thiết kế giải pháp	—
II. Dữ liệu	2	Thu thập dữ liệu (file, SQL, API, scraping, streaming), data contract, versioning	pandas , polars , SQLAlchemy , requests , httpx , pandera , DVC
	3	Phân tích khám phá dữ liệu (EDA)	pandas , ydata-profiling , missingno , seaborn
	4	Phân tích dữ liệu & thống kê suy luận (kiểm định, A/B test, bootstrap)	scipy , statsmodels , pingouin
	5	Tiền xử lý & Feature Engineering	scikit-learn , category_encoders , imbalanced-learn

Phần	Chương	Nội dung chính	Thư viện chủ đạo
	6	Chuẩn hóa & biến đổi phân phối	scikit-learn
	7	Chia dữ liệu & chiến lược Validation	scikit-learn
III. Mô hình	8	Huấn luyện mô hình, tối ưu siêu tham số, ensemble, deep learning	scikit-learn , XGBoost , LightGBM , CatBoost , Optuna , PyTorch
	9	Đánh giá mô hình: metrics, confusion matrix, threshold, calibration	scikit-learn , scipy
	10	Giải thích mô hình (XAI) & Fairness	SHAP , scikit-learn.inspection , fairlearn
	11	Trực quan hóa dữ liệu & kết quả	matplotlib , seaborn , plotly , streamlit
IV. Production	12	MLOps: tracking, registry, serving, Docker, CI/CD, monitoring, drift, retraining	MLflow , FastAPI , Docker , GitHub Actions , Evidently , Prefect/Airflow
	13	Checklist Production & Anti-patterns	—
Phụ lục	A-E	Cheatsheet thư viện, bảng chọn metric, bảng chọn thuật toán, thuật ngữ, tài liệu tham khảo	—

Lộ trình đọc đề xuất

- **Người mới (0-1 năm):** Chương 0 → 1 → 2 → 3 → 5 → 6 → 7 → 8 (phần cơ bản) → 9 → 11.
- **Trung cấp (1-3 năm):** Toàn bộ, tập trung chương 4, 7, 8 (nâng cao), 9 (threshold, calibration), 10.
- **Senior / ML Engineer:** Chương 1, 7, 9, 12, 13 — đặc biệt phần leakage, validation, monitoring, retraining.

0.2. Vòng đời dự án Data Science (CRISP-DM mở rộng cho Production)



Nguyên tắc vàng:

1. **Vòng lặp, không phải thác nước.** EDA thường làm thay đổi định nghĩa bài toán; đánh giá thường buộc quay lại thu thập dữ liệu.
2. **Baseline trước, mô hình phức tạp sau.** Một baseline đơn giản trả lời câu hỏi "mô hình có đáng làm không?".
3. **Mọi thứ phải tái lập được (reproducible):** code (git), dữ liệu (DVC/snapshot), môi trường (lock file/Docker), tham số (config), random seed.
4. **Pipeline hóa từ sớm.** Notebook dùng để khám phá; code production nằm trong module có test.

Tỷ lệ thời gian thực tế trong một dự án

Giai đoạn	% thời gian (kinh nghiệm thực tế)
Hiểu bài toán, làm việc với stakeholder	10–15%
Thu thập, làm sạch, tiền xử lý dữ liệu	40–50%
EDA & feature engineering	15–20%
Huấn luyện & tuning	10–15%
Đánh giá, triển khai, giám sát	15–20%

0.3. Cấu trúc repository chuẩn Production

Dựa trên *Cookiecutter Data Science v2*, được điều chỉnh qua nhiều dự án:

```

churn-prediction/
├── .github/workflows/           # CI/CD: lint, test, train, deploy
│   └── ci.yml
├── configs/                     # Cấu hình tách khối code (YAML)
│   ├── data.yml
│   ├── train.yml
│   └── serve.yml
├── data/                        # KHÔNG commit vào git – quản lý bằng DVC
│   ├── raw/                    # Dữ liệu gốc, bất biến (immutable)
│   ├── interim/                # Dữ liệu trung gian
│   ├── processed/              # Dữ liệu sẵn sàng huấn luyện
│   └── external/               # Dữ liệu bên thứ 3
├── models/                     # Artifact mô hình (hoặc dùng MLflow registry)
├── notebooks/                  # Đặt tên: 01-tt-eda-churn.ipynb (số-tác giả-mô tả)
├── reports/figures/            # Hình ảnh, báo cáo
├── src/churn/                  # Package Python chính
│   ├── __init__.py
│   ├── config.py               # Đọc config (pydantic-settings)
│   ├── data/
│   │   ├── ingest.py           # Thu thập dữ liệu
│   │   ├── validate.py         # Data contract (pandera)
│   │   └── split.py            # Chia dữ liệu
│   ├── features/
│   │   └── build.py            # Feature engineering (sklearn transformers)
│   ├── models/
│   │   ├── train.py
│   │   ├── evaluate.py
│   │   └── predict.py
│   ├── monitoring/
│   │   └── drift.py
│   ├── serving/
│   │   └── api.py              # FastAPI
├── tests/
│   ├── unit/
│   ├── data/                  # Test chất lượng dữ liệu
│   └── model/                 # Test hành vi mô hình
├── Dockerfile
├── dvc.yml                     # Định nghĩa pipeline DVC
├── Makefile                    # make data / make train / make test
├── pyproject.toml              # Dependencies + cấu hình ruff, pytest, mypy
├── .pre-commit-config.yaml
├── .env.example                # Mẫu biến môi trường (KHÔNG chứa secret thật)
└── README.md

```

pyproject.toml mẫu

```

[project]
name = "churn"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "pandas>=2.2",
    "numpy>=1.26",
    "scikit-learn>=1.5",
    "lightgbm>=4.3",
    "optuna>=3.6",
    "pandera>=0.20",
    "mlflow>=2.14",
    "fastapi>=0.111",
    "uvicorn[standard]>=0.30",
    "pydantic>=2.7",

```

```

    "pydantic-settings>=2.3",
    "pyyaml>=6.0",
]

[project.optional-dependencies]
dev = ["pytest>=8", "pytest-cov", "ruff>=0.5", "mypy>=1.10", "pre-commit", "jupyterlab"]
viz = ["matplotlib>=3.8", "seaborn>=0.13", "plotly>=5.22", "shap>=0.45"]

[tool.ruff]
line-length = 100
target-version = "py310"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "N", "SIM", "PD", "NPY"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q --cov=src/churn --cov-report=term-missing"

```

Makefile mẫu

```

.PHONY: install lint test data train serve

install:
    pip install -e ".[dev,viz]" && pre-commit install

lint:
    ruff check src tests && ruff format --check src tests && mypy src

test:
    pytest

data:
    python -m churn.data.ingest --config configs/data.yaml

train:
    python -m churn.models.train --config configs/train.yaml

serve:
    uvicorn churn.serving.api:app --host 0.0.0.0 --port 8000

```

Quản lý cấu hình với pydantic-settings

```

# src/churn/config.py
from pathlib import Path

import yaml
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class TrainConfig(BaseModel):
    target: str = "churn"
    test_size: float = Field(0.2, gt=0, lt=1)
    random_state: int = 42
    n_trials: int = 50
    model_params: dict = {}

```

```

class Secrets(BaseSettings):
    """Secret đọc từ biến môi trường / file .env – KHÔNG bao giờ hard-code."""

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    db_url: str = "sqlite:///local.db"
    mlflow_tracking_uri: str = "file:./mlruns"

def load_config(path: str | Path) -> TrainConfig:
    with open(path, encoding="utf-8") as f:
        return TrainConfig(**yaml.safe_load(f))

```

0.4. Bộ dữ liệu xuyên suốt handbook

Để code mẫu chạy được ngay mà không phụ thuộc dữ liệu bên ngoài, toàn bộ handbook dùng một bộ dữ liệu **mô phỏng bài toán dự đoán khách hàng rời bỏ (Customer Churn)** của một công ty viễn thông. Bộ dữ liệu được thiết kế có chủ đích các "vấn đề thực tế": giá trị thiếu, ngoại lai, mất cân bằng lớp, biến phân loại có nhiều mức, biến thời gian.

```

# src/churn/data/synthetic.py
import numpy as np
import pandas as pd

def make_churn_data(n: int = 20_000, seed: int = 42) -> pd.DataFrame:
    """Sinh dữ liệu churn mô phỏng, có missing, outlier và mất cân bằng lớp (~15-20% churn)."""
    rng = np.random.default_rng(seed)

    signup = pd.Timestamp("2022-01-01") + pd.to_timedelta(rng.integers(0, 900, n), unit="D")
    tenure_months = rng.integers(1, 72, n)
    monthly_charges = rng.normal(70, 25, n).clip(10, 200)
    contract = rng.choice(["month-to-month", "one-year", "two-year"], n, p=[0.55, 0.25, 0.20])
    payment = rng.choice(["e-wallet", "bank-transfer", "credit-card", "cash"], n)
    region = rng.choice([f"R{i:02d}" for i in range(30)], n) # high-cardinality
    support_calls = rng.poisson(1.5, n)
    data_usage_gb = rng.lognormal(mean=2.5, sigma=0.8, size=n) # lệch pha'i
    age = rng.normal(38, 12, n).clip(18, 85)

    # Xác suất churn phụ thuộc phi tuyến vào các biến (ground truth logic)
    logit = (
        -2.2
        + 1.3 * (contract == "month-to-month")
        - 0.03 * tenure_months
        + 0.012 * (monthly_charges - 70)
        + 0.35 * support_calls
        + 0.4 * (payment == "cash")
        - 0.01 * (age - 38)
    )
    churn = rng.random(n) < 1 / (1 + np.exp(-logit))

    df = pd.DataFrame(
        {
            "customer_id": [f"C{i:07d}" for i in range(n)],
            "signup_date": signup,
            "age": age.round(0),
            "tenure_months": tenure_months,
            "monthly_charges": monthly_charges.round(2),

```

```

        "total_charges": (monthly_charges * tenure_months).round(2),
        "contract": contract,
        "payment_method": payment,
        "region": region,
        "support_calls": support_calls,
        "data_usage_gb": data_usage_gb.round(2),
        "churn": churn.astype(int),
    }
)

# Bơm "vấn đề" thực tế"
for col, rate in {"age": 0.05, "data_usage_gb": 0.08, "payment_method": 0.03}.items():
    df.loc[rng.random(n) < rate, col] = np.nan
outlier_idx = rng.choice(n, size=30, replace=False)
df.loc[outlier_idx, "monthly_charges"] *= 8 # ngoại lai do lỗi nhập
liệu
df.loc[rng.choice(n, 50, replace=False), "contract"] = "Month-to-Month" # lỗi chuán hóa
chuỗi
df = pd.concat([df, df.sample(100, random_state=seed)], ignore_index=True) # bỏ ghi trùng
return df

if __name__ == "__main__":
    data = make_churn_data()
    print(data.shape, data["churn"].mean().round(3))
    data.to_parquet("data/raw/churn.parquet", index=False)

```

0.5. Thiết lập môi trường

```

# Cách 1: uv (nhẹ nhớt, khuyến nghị 2025+)
pip install uv
uv venv .venv && source .venv/bin/activate
uv pip install -e ".[dev,viz]"

# Cách 2: conda / mamba (khi cần thư viện C/CUDA phức tạp)
mamba create -n ds python=3.11 -y && mamba activate ds
pip install -e ".[dev,viz]"

# Khóa phiên bản để tái lập
uv pip compile pyproject.toml -o requirements.lock

```

.pre-commit-config.yaml

```

repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.5.0
  hooks:
    - id: ruff
      args: [--fix]
    - id: ruff-format
- repo: https://github.com/kynan/nbstripout # xoá output notebook trước khi commit
  rev: 0.7.1
  hooks:
    - id: nbstripout
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:

```

```
- id: check-added-large-files
  args: ["--maxkb=5000"]
- id: detect-private-key
```

Tái lập kết quả (Reproducibility)

```
import os
import random

import numpy as np

def seed_everything(seed: int = 42) -> None:
    random.seed(seed)
    np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    try:
        import torch

        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False
    except ImportError:
        pass
```

Checklist Chương 0

- ☐ Repo có cấu trúc rõ ràng, tách `src/`, `configs/`, `tests/`, `notebooks/`.
- ☐ Dependencies được khóa phiên bản (lock file).
- ☐ Secret nằm trong `.env`, `.env` có trong `.gitignore`.
- ☐ `pre-commit` đã cài: lint, format, strip notebook output.
- ☐ Có hàm `seed_everything` và seed được ghi vào config.

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN & THIẾT KẾ GIẢI PHÁP

"Một mô hình hoàn hảo cho một bài toán sai vẫn là một thất bại."

1.1. Từ câu hỏi kinh doanh đến bài toán ML

Bước	Câu hỏi cần trả lời	Ví dụ (Churn)
1. Mục tiêu kinh doanh	Doanh nghiệp muốn cải thiện chỉ số nào?	Giảm tỷ lệ rời bỏ 2 điểm % trong 2 quý
2. Hành động (Action)	Dự đoán xong thì ai làm gì?	CSKH gọi điện + tặng voucher cho top-K khách nguy cơ cao
3. Đơn vị dự đoán	Dự đoán cho đối tượng nào, lúc nào?	Mỗi khách hàng, vào ngày 1 hằng tháng
4. Nhãn (Label)	Định nghĩa chính xác "positive"?	Hủy dịch vụ trong 30 ngày tới kể từ ngày dự đoán
5. Loại bài toán	Phân loại/hồi quy/xếp hạng/phân cụm/chuỗi thời gian?	Phân loại nhị phân + xếp hạng top-K
6. Ràng buộc	Độ trễ, chi phí, giải thích, pháp lý?	Batch hằng tháng; phải giải thích lý do cho CSKH
7. Metric offline	Metric nào phản ánh mục tiêu?	PR-AUC, Recall@Top-10%, Lift
8. Metric online	Đo hiệu quả thực tế ra sao?	Tỷ lệ giữ chân nhóm treatment vs control (A/B test)
9. Baseline	Hiện tại làm thế nào?	Quy tắc: hợp đồng tháng + >3 cuộc gọi khiếu nại

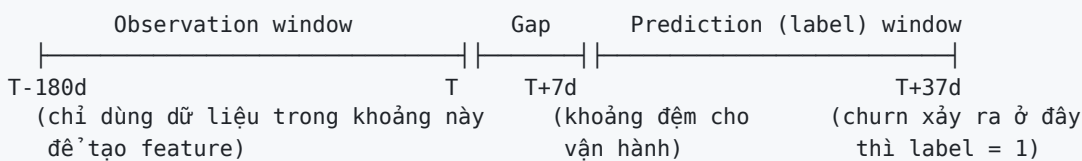
Các dạng bài toán và thuật toán điển hình

Dạng bài toán	Đầu ra	Ví dụ	Thuật toán khởi đầu
Phân loại nhị phân	0/1 + xác suất	Churn, gian lận, spam	Logistic Regression → LightGBM
Phân loại đa lớp	1 trong K lớp	Phân loại ticket, ảnh	Multinomial LR → GBM / CNN / Transformer
Đa nhãn (multi-label)	Tập con các nhãn	Gắn tag văn bản	One-vs-Rest, BCE loss
Hồi quy	Số thực	Giá nhà, doanh thu	Ridge → GBM
Xếp hạng (ranking)	Thứ tự	Tìm kiếm, gợi ý	LambdaMART (LightGBM ranker)

Dạng bài toán	Đầu ra	Ví dụ	Thuật toán khởi đầu
Chuỗi thời gian	Giá trị tương lai	Dự báo nhu cầu	Seasonal naive → ETS/ARIMA → GBM với lag features
Phân cụm	Nhóm	Phân khúc khách hàng	K-Means, HDBSCAN, GMM
Phát hiện bất thường	Điểm bất thường	Giám sát hệ thống	Isolation Forest, Autoencoder
Suy luận nhân quả	Hiệu ứng can thiệp (uplift)	Ai nên nhận voucher?	T-learner, X-learner, Causal Forest

Lưu ý của chuyên gia: Bài toán churn ở trên thực chất là bài toán **uplift** — ta không muốn tìm người *sẽ rời bỏ*, mà tìm người *sẽ ở lại nếu được can thiệp*. Hãy bắt đầu bằng mô hình churn, nhưng đề xuất thiết kế A/B test để tiến tới uplift modeling.

1.2. Định nghĩa nhãn và cửa sổ thời gian (tránh leakage ngay từ thiết kế)



- **Feature** chỉ được tính từ dữ liệu có sẵn **trước thời điểm T** (point-in-time correctness).
- **Gap** mô phỏng độ trễ vận hành (dữ liệu đến chậm, thời gian CSKH hành động).
- **Label** tính trong cửa sổ tương lai.

```
import pandas as pd

def build_label(events: pd.DataFrame, snapshot_date: str, horizon_days: int = 30,
               gap_days: int = 7) -> pd.DataFrame:
    """Gán nhãn churn cho mô~i khách hàng tại snapshot_date (point-in-time)."""
    t = pd.Timestamp(snapshot_date)
    start, end = t + pd.Timedelta(days=gap_days), t + pd.Timedelta(days=gap_days + horizon_days)
    active = events.loc[events["event_date"] < t, "customer_id"].unique()
    churned = events.loc[
        (events["event_type"] == "cancel") & events["event_date"].between(start, end),
        "customer_id",
    ].unique()
    labels = pd.DataFrame({"customer_id": active, "snapshot_date": t})
    labels["churn"] = labels["customer_id"].isin(churned).astype(int)
    return labels
```

1.3. Từ chi phí kinh doanh đến metric

Xây dựng **ma trận chi phí/lợi ích** cùng stakeholder:

	Dự đoán: Churn	Dự đoán: Không churn
Thực tế: Churn	TP: giữ được khách (+ 500k – 50k voucher)	FN: mất khách (– 500k)
Thực tế: Không churn	FP: tốn voucher (– 50k)	TN: 0

Từ đây suy ra: FN đắt gấp 10 lần FP → ưu tiên **Recall**, chọn ngưỡng theo **lợi nhuận kỳ vọng** (chương 9), không phải mặc định 0.5.

1.4. Tài liệu thiết kế (Design Doc) một trang

```
# [Tên dự án] – ML Design Doc
## 1. Bối cảnh & mục tiêu kinh doanh (KPI, giá trị kỳ vọng)
## 2. Định nghĩa bài toán (đơn vị, nhãn, cửa sổ thời gian, loại bài toán)
## 3. Dữ liệu (nguồn, khối lượng, độ trễ, chủ sở hữu, PII)
## 4. Baseline hiện tại & tiêu chí thành công (offline + online)
## 5. Phương pháp tiếp cận (feature, mô hình, validation)
## 6. Triển khai (batch/online, SLA độ trễ, tần suất retrain)
## 7. Giám sát & rủi ro (drift, fairness, fallback)
## 8. Kế hoạch & mốc thời gian
```

Checklist Chương 1

- ☐ Có mục tiêu kinh doanh đo được và hành động cụ thể sau dự đoán.
- ☐ Nhãn định nghĩa rõ ràng, có cửa sổ thời gian và gap.
- ☐ Có baseline (quy tắc / heuristic) để so sánh.
- ☐ Metric offline gắn với chi phí kinh doanh; có kế hoạch đo online.
- ☐ Ràng buộc (độ trễ, giải thích, pháp lý, PII) được ghi nhận.

CHƯƠNG 2. THU THẬP DỮ LIỆU (DATA COLLECTION & INGESTION)

2.1. Bản đồ nguồn dữ liệu

Nguồn	Ví dụ	Công cụ Python	Lưu ý production
File tĩnh	CSV, Excel, JSON, Parquet	<code>pandas</code> , <code>polars</code> , <code>pyarrow</code>	Ưu tiên Parquet (có schema, nén, đọc cột)
Cơ sở dữ liệu quan hệ	PostgreSQL, MySQL, SQL Server	<code>SQLAlchemy</code> , <code>psycopg2</code> , <code>connectorx</code>	Đọc theo chunk, đẩy tính toán xuống SQL
Data Warehouse	BigQuery, Snowflake, Redshift	<code>google-cloud-bigquery</code> , <code>snowflake-connector</code>	Chi phí theo lượng quét — chọn cột, lọc partition
Data Lake	S3, GCS, ADLS	<code>s3fs</code> , <code>gcsfs</code> , <code>pyarrow.dataset</code> , <code>duckdb</code>	Partition theo ngày, định dạng Parquet/Delta/Iceberg
REST/GraphQL API	CRM, thanh toán	<code>requests</code> , <code>httpx</code>	Retry, rate-limit, phân trang, timeout
Web scraping	Trang công khai	<code>httpx</code> , <code>BeautifulSoup</code> , <code>Playwright</code> , <code>Scrapy</code>	Tuân thủ robots.txt & điều khoản sử dụng
Streaming	Clickstream, IoT	<code>kafka-python</code> , <code>confluent-kafka</code>	Idempotency, xử lý dữ liệu đến trễ
Dữ liệu phi cấu trúc	Văn bản, ảnh, PDF	<code>pypdf</code> , <code>docling</code> , <code>Pillow</code> , <code>OpenCV</code>	Lưu metadata + đường dẫn, không nhét blob vào DB
Dữ liệu nhãn thủ công	Annotation	Label Studio, Argilla	Đo độ đồng thuận (Cohen's κ) giữa người gán nhãn

2.2. Đọc file — cơ bản đến tối ưu

```
import pandas as pd
import polars as pl

# CSV: luôn chỉ định dtype + parse_dates để tránh suy luận sai và tốn RAM
dtypes = {"customer_id": "string", "contract": "category", "region": "category"}
df = pd.read_csv("data/raw/churn.csv", dtype=dtypes, parse_dates=["signup_date"],
                 na_values=["", "NA", "null", "-"], encoding="utf-8")

# File lớn hơn RAM: đọc theo chunk
chunks = pd.read_csv("big.csv", chunksize=500_000, dtype=dtypes)
agg = pd.concat(c.groupby("region", observed=True)["monthly_charges"].sum() for c in chunks)
agg = agg.groupby(level=0).sum()

# Parquet: đọc chỉ các cột cần thiết + lọc (predicate pushdown)
df = pd.read_parquet("data/raw/churn.parquet", columns=["customer_id", "tenure_months", "churn"],
                    filters=[("tenure_months", ">", 6)])

# Polars lazy: tối ưu truy vấn tự động, đa luồng, nhanh hơn pandas 5-20 lần trên dữ liệu lớn
```

```
result = (
    pl.scan_parquet("data/raw/*.parquet")
    .filter(pl.col("tenure_months") > 6)
    .group_by("contract")
    .agg(pl.col("churn").mean().alias("churn_rate"), pl.len().alias("n"))
    .collect()
)
```

Giảm bộ nhớ DataFrame

```
import numpy as np
import pandas as pd

def optimize_dtypes(df: pd.DataFrame, cat_threshold: float = 0.5) -> pd.DataFrame:
    """Downcast số và chuyển chuỗi ít giá trị sang category. Thường giảm 50-80% RAM."""
    out = df.copy()
    for col in out.select_dtypes(include="integer").columns:
        out[col] = pd.to_numeric(out[col], downcast="integer")
    for col in out.select_dtypes(include="float").columns:
        out[col] = pd.to_numeric(out[col], downcast="float")
    for col in out.select_dtypes(include="object").columns:
        if out[col].nunique(dropna=True) / max(len(out), 1) < cat_threshold:
            out[col] = out[col].astype("category")
    before, after = df.memory_usage(deep=True).sum(), out.memory_usage(deep=True).sum()
    print(f"Memory: {before / 1e6:.1f} MB -> {after / 1e6:.1f} MB")
    return out
```

2.3. Đọc từ cơ sở dữ liệu (SQL)

```
from contextlib import contextmanager

import pandas as pd
from sqlalchemy import create_engine, text

from churn.config import Secrets

engine = create_engine(Secrets().db_url, pool_pre_ping=True, pool_size=5)

QUERY = text("""
    SELECT c.customer_id, c.signup_date, c.contract,
           COUNT(t.ticket_id)           AS support_calls_90d,
           AVG(b.amount)                AS avg_bill_6m
    FROM customers c
    LEFT JOIN tickets t ON t.customer_id = c.customer_id
                     AND t.created_at >= :snapshot - INTERVAL '90 days' AND t.created_at < :snapshot
    LEFT JOIN bills b   ON b.customer_id = c.customer_id
                     AND b.bill_date >= :snapshot - INTERVAL '6 months' AND b.bill_date < :snapshot
    WHERE c.signup_date < :snapshot
    GROUP BY c.customer_id, c.signup_date, c.contract
""")

def load_features(snapshot: str, chunksize: int = 100_000) -> pd.DataFrame:
    # Tham số hóa (bind params) => chống SQL injection + tái sử dụng execution plan
    with engine.connect() as conn:
```

```
parts = pd.read_sql(QUERY, conn, params={"snapshot": snapshot}, chunksize=chunksize)
return pd.concat(parts, ignore_index=True)
```

Nguyên tắc: đẩy JOIN/GROUP BY/WHERE xuống database (gần dữ liệu), chỉ kéo về kết quả đã tổng hợp. Mọi điều kiện thời gian phải < :snapshot để đảm bảo point-in-time.

DuckDB — SQL trên file Parquet/CSV không cần server

```
import duckdb

con = duckdb.connect()
df = con.execute("""
    SELECT contract, AVG(churn) AS churn_rate, COUNT(*) AS n
    FROM read_parquet('data/raw/*.parquet')
    GROUP BY contract ORDER BY churn_rate DESC
    """).df()
```

2.4. Thu thập qua API — retry, rate limit, phân trang

```
import logging
import time
from collections.abc import Iterator

import httpx
from tenacity import retry, retry_if_exception_type, stop_after_attempt, wait_exponential

log = logging.getLogger(__name__)

class APIClient:
    def __init__(self, base_url: str, token: str, rate_per_sec: float = 5.0, timeout: float = 30):
        self.client = httpx.Client(
            base_url=base_url,
            headers={"Authorization": f"Bearer {token}"},
            timeout=timeout,
        )
        self.min_interval = 1.0 / rate_per_sec
        self._last_call = 0.0

    def _throttle(self) -> None:
        wait = self.min_interval - (time.monotonic() - self._last_call)
        if wait > 0:
            time.sleep(wait)
        self._last_call = time.monotonic()

    @retry(
        retry=retry_if_exception_type((httpx.TransportError, httpx.HTTPStatusError)),
        wait=wait_exponential(multiplier=1, min=2, max=60),
        stop=stop_after_attempt(5),
        reraise=True,
    )
    def get(self, path: str, params: dict | None = None) -> dict:
        self._throttle()
        resp = self.client.get(path, params=params)
        if resp.status_code == 429: # Too Many Requests -> tôn trọng Retry-After
            time.sleep(float(resp.headers.get("Retry-After", 5)))
        resp.raise_for_status()
```

```

        return resp.json()

    def paginate(self, path: str, page_size: int = 100) -> Iterator[dict]:
        cursor = None
        while True:
            params = {"limit": page_size, **({"cursor": cursor} if cursor else {})}
            payload = self.get(path, params)
            yield from payload["data"]
            cursor = payload.get("next_cursor")
            if not cursor:
                break

```

Với hàng nghìn request, dùng `httpx.AsyncClient` + `asyncio.Semaphore` để giới hạn số kết nối đồng thời.

2.5. Web scraping có trách nhiệm

```

import urllib.robotparser

import httpx
from bs4 import BeautifulSoup

UA = "ResearchBot/1.0 (contact: data-team@example.com)"

def can_fetch(url: str) -> bool:
    rp = urllib.robotparser.RobotFileParser()
    base = "/".join(url.split("/")[:3])
    rp.set_url(f"{base}/robots.txt")
    rp.read()
    return rp.can_fetch(UA, url)

def scrape_table(url: str) -> list[dict]:
    if not can_fetch(url):
        raise PermissionError(f"robots.txt không cho phép: {url}")
    html = httpx.get(url, headers={"User-Agent": UA}, timeout=20).text
    soup = BeautifulSoup(html, "lxml")
    rows = soup.select("table.data tr")
    headers = [th.get_text(strip=True) for th in rows[0].select("th")]
    return [dict(zip(headers, (td.get_text(strip=True) for td in r.select("td")))) for r in
            rows[1:]]

```

Trang render bằng JavaScript → dùng `Playwright`. Luôn: giới hạn tốc độ, cache kết quả, không thu thập dữ liệu cá nhân trái phép (tuân thủ **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân tại Việt Nam, GDPR nếu có khách hàng EU).

2.6. Streaming (Kafka) — tiêu thụ sự kiện

```

import json

from confluent_kafka import Consumer

consumer = Consumer({

```

```

    "bootstrap.servers": "kafka:9092",
    "group.id": "churn-feature-builder",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,          # commit thu' công sau khi xử lý thành công
})
consumer.subscribe(["customer-events"])

try:
    while True:
        msg = consumer.poll(1.0)
        if msg is None or msg.error():
            continue
        event = json.loads(msg.value())
        # upsert_feature(event) pha'i idempotent (xử lý lại không gây sai)
        consumer.commit(msg, asynchronous=False)
finally:
    consumer.close()

```

2.7. Data Contract & kiểm tra chất lượng dữ liệu đầu vào

Dữ liệu sai là nguyên nhân số 1 gây sự cố mô hình production. **Validate tại biên (ingestion boundary).**

```

# src/churn/data/validate.py
import pandas as pd
import pandera.pandas as pa
from pandera import Check, Column

churn_schema = pa.DataFrameSchema(
    {
        "customer_id": Column(str, unique=True, nullable=False,
                               checks=Check.str_matches(r"^\d{7}$")),
        "signup_date": Column("datetime64[ns]", Check.le(pd.Timestamp.today())),
        "age": Column(float, Check.in_range(18, 100), nullable=True),
        "tenure_months": Column(int, Check.ge(0)),
        "monthly_charges": Column(float, Check.in_range(0, 2_000)),
        "contract": Column(str, Check.isin(["month-to-month", "one-year", "two-year"])),
        "churn": Column(int, Check.isin([0, 1])),
    },
    checks=[
        # Ràng buộc liên cột
        Check(lambda d: (d["total_charges"] >= d["monthly_charges"] * 0.9).mean() > 0.99,
                    error="total_charges không nhất quán với monthly_charges"),
    ],
    strict=False,
    coerce=True,
)

def validate(df):
    # lazy=True: gom TẤT CẢ lỗi thay vì dừng ở lỗi đầu tiên
    return churn_schema.validate(df, lazy=True)

```

Ghi chú phiên bản: với pandera < 0.20 dùng `import pandera as pa`. Trên dữ liệu thô của handbook, schema này sẽ **bắt được** lỗi `"Month-to-Month"` (sai chuẩn hóa chuỗi) — đúng mục đích: lỗi được phát hiện tại biên, rồi xử lý ở bước làm sạch (Chương 5).

Các chiều chất lượng dữ liệu cần kiểm tra:

Chiều	Câu hỏi	Ví dụ kiểm tra
Completeness	Thiếu bao nhiêu?	% null mỗi cột < ngưỡng
Uniqueness	Có trùng không?	customer_id duy nhất
Validity	Đúng định dạng/miền giá trị?	tuổi $\in [18, 100]$
Consistency	Logic giữa các cột/bảng?	end_date $\geq$ start_date
Timeliness	Dữ liệu có mới?	<code>max(event_time) > now - 1 day</code>
Volume	Số dòng có bất thường?	$\pm 30\%$ so với trung bình 7 ngày

Công cụ mạnh hơn cho hệ thống lớn: **Great Expectations**, **Soda Core**, **dbt tests**.

2.8. Versioning dữ liệu với DVC

```

pip install "dvc[s3]"
dvc init
dvc remote add -d storage s3://my-bucket/dvc-store
dvc add data/raw/churn.parquet          # tạo file churn.parquet.dvc (commit vào git)
git add data/raw/churn.parquet.dvc data/raw/.gitignore
git commit -m "data: snapshot churn 2026-10"
dvc push                                # đẩy dữ liệu thật lên S3
# Quay lại phiên ba'n cũ:
git checkout <commit> && dvc checkout

```

```

# dvc.yaml - pipeline có cache, chỉ chạy lại bước có thay đổi
stages:
  ingest:
    cmd: python -m churn.data.ingest --config configs/data.yaml
    deps: [src/churn/data/ingest.py, configs/data.yaml]
    outs: [data/raw/churn.parquet]
  featurize:
    cmd: python -m churn.features.build
    deps: [src/churn/features/build.py, data/raw/churn.parquet]
    outs: [data/processed/features.parquet]
  train:
    cmd: python -m churn.models.train --config configs/train.yaml
    deps: [src/churn/models/train.py, data/processed/features.parquet]
    params: [configs/train.yaml:]
    outs: [models/model.joblib]
    metrics: [reports/metrics.json: {cache: false}]

```

2.9. Pitfalls khi thu thập dữ liệu

1. **Survivorship bias:** chỉ lấy khách hàng *đang* hoạt động → thiếu hẳn nhóm đã churn.
2. **Dữ liệu bị ghi đè (overwrite):** bảng CRM chỉ lưu trạng thái hiện tại → feature tại quá khứ bị "nhìn trộm tương lai". Cần bảng lịch sử (SCD Type 2) hoặc snapshot.
3. **Lịch múi giờ:** chuẩn hóa mọi timestamp về UTC khi lưu, chuyển `Asia/Ho_Chi_Minh` khi hiển thị.

4. **Encoding tiếng Việt:** luôn `utf-8`; chuẩn hóa Unicode `unicodedata.normalize("NFC", s)` (dữ liệu tiếng Việt hay lẫn NFC/NFD).
5. **Sampling bias:** dữ liệu huấn luyện từ một kênh/khu vực không đại diện cho toàn bộ quần thể.

Checklist Chương 2

- ☐ Biết rõ nguồn, chủ sở hữu, tần suất cập nhật, độ trễ của mọi bảng dữ liệu.
- ☐ Truy vấn đảm bảo point-in-time (không dùng dữ liệu sau thời điểm dự đoán).
- ☐ Có data contract & validation tự động tại bước ingestion.
- ☐ Dữ liệu thô bất biến, được version (DVC / snapshot có ngày).
- ☐ Tuân thủ quy định bảo vệ dữ liệu cá nhân; PII được mã hóa/ẩn danh.

CHƯƠNG 3. PHÂN TÍCH KHÁM PHÁ DỮ LIỆU (EDA)

3.1. Mục tiêu của EDA

EDA không phải là "vẽ thật nhiều biểu đồ". EDA trả lời 5 nhóm câu hỏi:

1. **Cấu trúc:** bao nhiêu dòng/cột, kiểu dữ liệu, đơn vị quan sát là gì, có trùng lặp?
2. **Chất lượng:** thiếu ở đâu, vì sao thiếu, ngoại lai, giá trị vô lý, sai chuẩn hóa?
3. **Phân phối:** từng biến phân phối thế nào (lệch, đa đỉnh, đuôi dài)?
4. **Quan hệ:** biến nào liên quan đến target; các biến tương quan với nhau (đa cộng tuyến)?
5. **Rủi ro:** có dấu hiệu **leakage**, drift theo thời gian, bias theo nhóm?

Quy tắc quan trọng: Làm EDA chi tiết trên **tập train** sau khi đã tách test (hoặc ít nhất là không dùng thông tin từ test để ra quyết định tiền xử lý). Nhìn vào test quá nhiều = overfitting bằng mắt.

3.2. Tổng quan nhanh (First look)

```
import numpy as np
import pandas as pd

from churn.data.synthetic import make_churn_data

df = make_churn_data()

print(df.shape)
df.info(memory_usage="deep")
display(df.head())
display(df.describe(include="all").T)

def overview(df: pd.DataFrame) -> pd.DataFrame:
    """Bảng tổng quan từng cột – thứ đầu tiên tôi chạy với mọi dataset."""
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "n_missing": df.isna().sum(),
        "pct_missing": (df.isna().mean() * 100).round(2),
        "n_unique": df.nunique(dropna=True),
        "pct_unique": (df.nunique(dropna=True) / len(df) * 100).round(2),
        "sample_values": [df[c].dropna().unique()[:5].tolist() for c in df.columns],
    }).sort_values("pct_missing", ascending=False)

display(overview(df))

# Trùng lặp: toàn dòng và theo khóa
print("Duplicated rows:", df.duplicated().sum())
print("Duplicated customer_id:", df["customer_id"].duplicated().sum())
```

```
# Phân phối target
print(df["churn"].value_counts(normalize=True).round(3))
```

Báo cáo tự động

```
# ydata-profiling: báo cáo HTML đầy đủ (phân phối, tương quan, missing, cảnh báo)
from ydata_profiling import ProfileReport

ProfileReport(df, title="Churn EDA", minimal=len(df) > 100_000).to_file("reports/eda.html")

# Các lựa chọn khác: sweetviz (so sánh train/test), dtale (giao diện tương tác), skimpy (terminal)
```

Báo cáo tự động là **điểm khởi đầu**, không thay thế phân tích có giả thuyết.

3.3. Phân tích giá trị thiếu (Missing Values)

Ba cơ chế thiếu (Rubin, 1976) — quyết định cách xử lý:

Cơ chế	Ý nghĩa	Ví dụ	Xử lý
MCAR — Missing Completely At Random	Thiếu hoàn toàn ngẫu nhiên	Lỗi truyền tin ngẫu nhiên	Xóa hoặc impute đơn giản đều không gây bias
MAR — Missing At Random	Thiếu phụ thuộc biến <i>quan sát được</i>	Người trẻ hay bỏ trống thu nhập	Impute có điều kiện (KNN, Iterative)
MNAR — Missing Not At Random	Thiếu phụ thuộc chính giá trị bị thiếu	Người thu nhập cao không khai thu nhập	Thêm cờ <code>is_missing</code> , mô hình hóa cơ chế thiếu

```
import matplotlib.pyplot as plt
import missingno as msno

msno.matrix(df.sample(2000, random_state=0)); plt.show() # mô hình thiếu theo dòng
msno.heatmap(df); plt.show() # tương quan giữa các cột bị thiếu

# Thiếu có liên quan đến target không? (dấu hiệu MAR/MNAR và tín hiệu dự đoán)
for col in df.columns[df.isna().any()]:
    rate = df.groupby(df[col].isna())["churn"].mean()
    print(f"{col:15s} churn khi có={rate.get(False, np.nan):.3f} | khi thiếu={rate.get(True, np.nan):.3f}")
```

Nếu tỷ lệ churn khác biệt rõ giữa nhóm thiếu và không thiếu → **cờ missing là một feature có giá trị**.

3.4. Phân tích đơn biến (Univariate)

Biến số

```
import seaborn as sns
from scipy import stats
```

```

num_cols = ["age", "tenure_months", "monthly_charges", "total_charges",
            "support_calls", "data_usage_gb"]

def describe_numeric(s: pd.Series) -> pd.Series:
    s = s.dropna()
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return pd.Series({
        "mean": s.mean(), "median": s.median(), "std": s.std(),
        "skew": s.skew(), "kurtosis": s.kurt(),
        "p01": s.quantile(0.01), "p99": s.quantile(0.99),
        "n_outlier_iqr": ((s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)).sum(),
        "n_zero": (s == 0).sum(),
    })

display(df[num_cols].apply(describe_numeric).T.round(2))

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, col in zip(axes.ravel(), num_cols):
    sns.histplot(df[col], kde=True, ax=ax)
    ax.set_title(f"{col} | skew={df[col].skew():.2f}")
plt.tight_layout()

```

Diễn giải:

- `|skew| > 1` → lệch mạnh → cân nhắc log/Box-Cox/Yeo-Johnson (Chương 6).
- `kurtosis` cao → đuôi dày, nhiều ngoại lai.
- Nhiều giá trị 0 → có thể là "không dùng dịch vụ" (zero-inflated) hoặc giá trị mặc định thay cho missing.

Biến phân loại

```

cat_cols = ["contract", "payment_method", "region"]
for col in cat_cols:
    vc = df[col].value_counts(dropna=False)
    print(f"\n{col}: {df[col].nunique()} mức")
    print(pd.concat([vc, (vc / len(df) * 100).round(2)], axis=1, keys=["n", "%"]).head(10))

```

Kiểm tra: **sai chính tả/hoa thường** (Month-to-Month vs month-to-month), **mức hiếm** (< 1%), **cardinality cao** (region 30 mức → cần target/frequency encoding).

3.5. Phân tích hai biến với target (Bivariate)

```

# Số vs target nhị phân: so sánh phân phối
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, col in zip(axes.ravel(), num_cols):
    sns.boxplot(data=df, x="churn", y=col, ax=ax, showfliers=False)
plt.tight_layout()

# Phân loại vs target: tỷ lệ churn theo từng mức + khoa'ng tin cậy
def rate_by_category(df: pd.DataFrame, col: str, target: str = "churn") -> pd.DataFrame:
    g = df.groupby(col, observed=True)[target].agg(["mean", "count"])
    se = np.sqrt(g["mean"] * (1 - g["mean"]) / g["count"])

```

```
g["ci95_low"], g["ci95_high"] = g["mean"] - 1.96 * se, g["mean"] + 1.96 * se
g["lift"] = g["mean"] / df[target].mean()
return g.sort_values("mean", ascending=False)
```

```
display(rate_by_category(df, "contract"))
```

```
# Biê'n số' chia bin -> tỷ lệ churn theo bin (phát hiện quan hệ phi tuyến)
df["tenure_bin"] = pd.qcut(df["tenure_months"], q=10, duplicates="drop")
df.groupby("tenure_bin", observed=True)["churn"].mean().plot(marker="o", title="Churn rate theo tenure")
```

Weight of Evidence (WoE) & Information Value (IV)

Kỹ thuật kinh điển trong credit scoring, rất hữu ích để xếp hạng sức mạnh dự đoán của biến:

$$WoE_i = \ln\left(\frac{\%Good_i}{\%Bad_i}\right), \quad IV = \sum_i (\%Good_i - \%Bad_i) \times WoE_i$$

```
def information_value(df: pd.DataFrame, feature: str, target: str, bins: int = 10) -> float:
    x = df[feature]
    if pd.api.types.is_numeric_dtype(x) and x.nunique() > bins:
        x = pd.qcut(x, q=bins, duplicates="drop")
    x = x.astype("object").where(x.notna(), "MISSING").astype(str) # missing = 1 nhóm riêng
    tab = pd.crosstab(x, df[target])
    good = (tab[0] + 0.5) / (tab[0].sum() + 0.5) # +0.5: làm tròn tránh log(0)
    bad = (tab[1] + 0.5) / (tab[1].sum() + 0.5)
    woe = np.log(good / bad)
    return float(((good - bad) * woe).sum())
```

```
iv = {c: information_value(df, c, "churn") for c in num_cols + cat_cols}
print(pd.Series(iv).sort_values(ascending=False).round(3))
```

IV	Sức mạnh dự đoán
< 0.02	Không có
0.02 - 0.1	Yếu
0.1 - 0.3	Trung bình
0.3 - 0.5	Mạnh
> 0.5	Đáng ngờ — kiểm tra leakage!

3.6. Tương quan & đa cộng tuyến

Cặp biến	Thước đo	Ghi chú
Số - Số (tuyến tính)	Pearson r	Nhạy với ngoại lai
Số - Số (đơn điệu)	Spearman ρ , Kendall τ	Bền vững hơn, dùng thứ hạng
Phân loại - Phân loại	Cramér's V	Dựa trên χ^2
Số - Phân loại	Correlation ratio η , ANOVA F	
Mọi loại, phi tuyến	Mutual Information, ΦK (phik)	Bắt quan hệ phi tuyến

```

from scipy.stats import chi2_contingency
from sklearn.feature_selection import mutual_info_classif

# Spearman heatmap
corr = df[num_cols].corr(method="spearman")
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="RdBu_r", center=0, vmin=-1, vmax=1)

def cramers_v(x: pd.Series, y: pd.Series) -> float:
    """Cramér's V có hiệu chi'nh bias (Bergsma, 2013)."""
    tab = pd.crosstab(x, y)
    chi2 = chi2_contingency(tab, correction=False)[0]
    n = tab.to_numpy().sum()
    r, k = tab.shape
    phi2 = max(0, chi2 / n - (k - 1) * (r - 1) / (n - 1))
    r_c, k_c = r - (r - 1) ** 2 / (n - 1), k - (k - 1) ** 2 / (n - 1)
    return float(np.sqrt(phi2 / max(min(k_c - 1, r_c - 1), 1e-12)))

print("Cramér's V(contract, churn) =", round(cramers_v(df["contract"], df["churn"]), 3))

# Mutual information (bắt quan hệ phi tuyến)
X_mi = df[num_cols].fillna(df[num_cols].median())
mi = mutual_info_classif(X_mi, df["churn"], random_state=0)
print(pd.Series(mi, index=num_cols).sort_values(ascending=False).round(4))

```

Variance Inflation Factor (VIF) — đa cộng tuyến

```

from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

X_vif = add_constant(df[num_cols].dropna())
vif = pd.Series([variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1])],
                 index=X_vif.columns).drop("const")
print(vif.sort_values(ascending=False).round(2)) # VIF > 5-10: đa cộng tuyến đáng kể

```

$\text{total_charges} \approx \text{monthly_charges} \times \text{tenure_months}$ → VIF cao. Với mô hình tuyến tính cần loại bỏ/kết hợp; với mô hình cây ít ảnh hưởng đến dự đoán nhưng làm **feature importance bị chia nhỏ** giữa các biến tương quan.

3.7. Phát hiện ngoại lai (Outliers)

```

from sklearn.ensemble import IsolationForest

def outlier_report(s: pd.Series) -> dict:
    s = s.dropna()
    q1, q3 = s.quantile([0.25, 0.75]); iqr = q3 - q1
    z = (s - s.mean()) / s.std()
    med = s.median(); mad = (s - med).abs().median()
    robust_z = 0.6745 * (s - med) / (mad if mad else 1e-9)
    return {
        "iqr_1.5": int(((s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)).sum()),
        "zscore_3": int((z.abs() > 3).sum()),
        "robust_z_3.5": int((robust_z.abs() > 3.5).sum()), # Iglewicz & Hoaglin
    }

```

```

}

display(pd.DataFrame({c: outlier_report(df[c]) for c in num_cols}).T)

# Ngoại lai đa biến
iso = IsolationForest(contamination=0.01, random_state=0)
flags = iso.fit_predict(df[num_cols].fillna(df[num_cols].median()))
print("Isolation Forest outliers:", (flags == -1).sum())

```

Ngoại lai ≠ lỗi. Câu hỏi đúng: *đây là lỗi nhập liệu, hay là sự kiện thực hiếm (VIP, gian lận)?* Hỏi domain expert trước khi xóa.

3.8. Phân tích theo thời gian & phát hiện drift sớm

```

monthly = (df.set_index("signup_date")
            .resample("MS")
            .agg(n=("churn", "size"), churn_rate=("churn", "mean"),
                 avg_charge=("monthly_charges", "median")))
monthly.plot(subplots=True, figsize=(12, 7), marker="o")

```

Nếu phân phối feature hoặc tỷ lệ target thay đổi mạnh theo thời gian → bắt buộc dùng **time-based split** (Chương 7), và cần giám sát drift (Chương 12).

3.9. Phát hiện Leakage trong EDA

Dấu hiệu nghi ngờ:

- Một biến đơn lẻ cho AUC > 0.9 hoặc IV > 0.5.
- Biến được tạo **sau** sự kiện target (ví dụ `cancellation_reason`, `last_bill_status = "closed"`).
- ID/thời gian tuần tự tương quan với target (dữ liệu được sắp xếp theo nhãn).

```

from sklearn.metrics import roc_auc_score

def single_feature_auc(df: pd.DataFrame, target: str) -> pd.Series:
    out = {}
    for col in df.select_dtypes("number").columns.drop(target):
        x = df[col].fillna(df[col].median())
        auc = roc_auc_score(df[target], x)
        out[col] = max(auc, 1 - auc)  # hướng không quan trọng
    return pd.Series(out).sort_values(ascending=False)

print(single_feature_auc(df, "churn").round(3))

```

Adversarial validation — train và test có cùng phân phối?

```

from lightgbm import LGBMClassifier
from sklearn.model_selection import cross_val_score

```

```
def adversarial_auc(train: pd.DataFrame, test: pd.DataFrame, features: list[str]) -> float:
    """AUC ~ 0.5: cùng phân phối. AUC >> 0.5: có drift / split có vênh lệch."""
    X = pd.concat([train[features], test[features]], ignore_index=True)
    y = np.r_[np.zeros(len(train)), np.ones(len(test))]
    clf = LGBMClassifier(n_estimators=200, verbose=-1)
    return cross_val_score(clf, X, y, cv=5, scoring="roc_auc").mean()
```

3.10. Mẫu báo cáo EDA gửi stakeholder

```
## Tóm tắt EDA – Churn (snapshot 2026-10)
- Dữ liệu: 20,100 dòng, 12 cột; 100 dòng trùng (đã loại). Tỷ lệ churn: 16.5%.
- Chất lượng: data_usage_gb thiếu 8% (churn khi thiếu 17.5% so với 16.5% – chênh lệch nhỏ);
  ~30 bản ghi monthly_charges gấp 8 lần bình thường, nghi lỗi nhập liệu; 50 bản ghi contract sai
  chuẩn hóa.
- Tín hiệu mạnh: contract (IV 0.32), tenure_months (IV 0.32), support_calls (IV 0.14).
- Rủi ro: total_charges đa cộng tuyến với tenure × monthly_charges.
- Đề xuất: time-based split; cờ missing cho data_usage_gb; target-encode region.
```

Checklist Chương 3

- ☐ Đã hiểu đơn vị quan sát, kiểm tra trùng lặp theo khóa.
- ☐ Phân tích missing (tỷ lệ, cơ chế, quan hệ với target).
- ☐ Phân phối đơn biến, ngoại lai đã được xác minh với domain expert.
- ☐ Quan hệ với target (rate-by-bin, IV, MI) và đa cộng tuyến (VIF).
- ☐ Đã kiểm tra leakage (single-feature AUC, logic thời gian) và drift theo thời gian.
- ☐ Có báo cáo tóm tắt kèm đề xuất hành động.

CHƯƠNG 4. PHÂN TÍCH DỮ LIỆU & THỐNG KÊ SUY LUẬN

EDA giúp *nhìn thấy* mẫu hình; phân tích thống kê giúp *khẳng định* mẫu hình đó không phải do ngẫu nhiên, và *định lượng* mức độ chắc chắn. Đây là kỹ năng phân biệt Data Scientist với người chỉ "chạy mô hình".

4.1. Bốn cấp độ phân tích

Cấp độ	Câu hỏi	Kỹ thuật
Mô tả (Descriptive)	Chuyện gì đã xảy ra?	Thống kê mô tả, KPI, dashboard, cohort
Chẩn đoán (Diagnostic)	Tại sao xảy ra?	Kiểm định giả thuyết, phân tích phân khúc, drill-down, hồi quy
Dự đoán (Predictive)	Chuyện gì sẽ xảy ra?	Machine Learning, dự báo chuỗi thời gian
Đề xuất (Prescriptive)	Nên làm gì?	Tối ưu hóa, uplift modeling, A/B test, mô phỏng

4.2. Thống kê mô tả theo nhóm & phân tích cohort

```
import numpy as np
import pandas as pd

from churn.data.synthetic import make_churn_data

df = make_churn_data().drop_duplicates()
df["contract"] = df["contract"].str.lower()

# Pivot nhiều chiều
pivot = pd.pivot_table(df, index="contract", columns="payment_method", values="churn",
                        aggfunc="mean", margins=True).round(3)

# Cohort theo tháng đăng ký: tỷ lệ churn & doanh thu trung bình
df["cohort"] = df["signup_date"].dt.to_period("Q")
cohort = df.groupby("cohort").agg(
    customers=("customer_id", "nunique"),
    churn_rate=("churn", "mean"),
    arpu=("monthly_charges", "median"),
)
```

Với dữ liệu sự kiện (event log), bảng cohort retention kinh điển:

```
def retention_matrix(events: pd.DataFrame) -> pd.DataFrame:
    """events: customer_id, event_date. Tra về % khách còn hoạt động sau k tháng."""
    e = events.copy()
    e["month"] = e["event_date"].dt.to_period("M")
    e["cohort"] = e.groupby("customer_id")["month"].transform("min")
```

```
e["age"] = (e["month"] - e["cohort"]).apply(lambda p: p.n)
counts = e.groupby(["cohort", "age"])["customer_id"].nunique().unstack(fill_value=0)
return counts.div(counts[0], axis=0).round(3)
```

4.3. Kiểm định giả thuyết — khung tư duy

1. Phát biểu **H₀** (không có khác biệt) và **H₁**.
2. Chọn mức ý nghĩa α (thường 0.05) **trước khi** nhìn dữ liệu.
3. Chọn kiểm định phù hợp (bảng dưới), kiểm tra giả định.
4. Tính p-value **và effect size + khoảng tin cậy** (p-value nhỏ $\neq$ khác biệt có ý nghĩa thực tế).
5. Hiệu chỉnh khi kiểm định nhiều lần (Bonferroni, Benjamini-Hochberg).

Bảng chọn kiểm định

Mục đích	Dữ liệu	Tham số (giả định chuẩn)	Phi tham số
So sánh 2 nhóm độc lập	Số	Welch's t-test	Mann-Whitney U
So sánh 2 nhóm ghép cặp	Số	Paired t-test	Wilcoxon signed-rank
So sánh ≥ 3 nhóm	Số	One-way ANOVA (Welch ANOVA)	Kruskal-Wallis
Độc lập giữa 2 biến phân loại	Tần số	χ^2 test of independence	Fisher's exact (mẫu nhỏ)
So sánh 2 tỷ lệ	Nhị phân	z-test cho tỷ lệ	Fisher's exact
Kiểm tra phân phối chuẩn	Số	Shapiro-Wilk (n<5000), D'Agostino	Q-Q plot
So sánh 2 phân phối	Số	—	Kolmogorov-Smirnov
Tương quan	Số	Pearson	Spearman, Kendall

```
from scipy import stats
from statsmodels.stats.proportion import proportions_ztest, proportion_confint

churn_yes = df.loc[df["churn"] == 1, "monthly_charges"].dropna()
churn_no = df.loc[df["churn"] == 0, "monthly_charges"].dropna()

# 1) Welch t-test (không giả định phương sai bằng nhau – nên dùng mặc định)
t, p = stats.ttest_ind(churn_yes, churn_no, equal_var=False)

# 2) Mann-Whitney U (phi tham số, bền vững với ngoại lai)
u, p_mw = stats.mannwhitneyu(churn_yes, churn_no, alternative="two-sided")

# 3) Effect size: Cohen's d và rank-biserial correlation
def cohens_d(a: np.ndarray, b: np.ndarray) -> float:
    na, nb = len(a), len(b)
    pooled = np.sqrt(((na - 1) * a.var(ddof=1) + (nb - 1) * b.var(ddof=1)) / (na + nb - 2))
    return (a.mean() - b.mean()) / pooled

d = cohens_d(churn_yes.to_numpy(), churn_no.to_numpy())
rank_biserial = 1 - 2 * u / (len(churn_yes) * len(churn_no))
print(f"Welch p={p:.2e}, MWU p={p_mw:.2e}, Cohen's d={d:.3f}, r_rb={rank_biserial:.3f}")
# |d|: 0.2 nhỏ, 0.5 trung bình, 0.8 lớn

# 4) Chi-square: contract có liên quan đến churn?
```

```

table = pd.crosstab(df["contract"], df["churn"])
chi2, p_chi, dof, expected = stats.chi2_contingency(table)
assert (expected >= 5).all(), "Kỳ vọng < 5 -> dùng Fisher's exact hoặc gộp nhóm"

# 5) Kruskal-Wallis: data_usage khác nhau giữa các loại hợp đồng?
groups = [g["data_usage_gb"].dropna() for _, g in df.groupby("contract")]
h, p_kw = stats.kruskal(*groups)

# 6) So sánh 2 tỷ lệ + khoa'ng tin cậy Wilson
cash = df["payment_method"] == "cash"
count = np.array([df.loc[cash, "churn"].sum(), df.loc[~cash, "churn"].sum()])
nobs = np.array([cash.sum(), (~cash).sum()])
z, p_prop = proportions_ztest(count, nobs)
ci_cash = proportion_confint(count[0], nobs[0], method="wilson")

```

Hiệu chỉnh đa kiểm định

Kiểm định 20 giả thuyết với $\alpha = 0.05 \rightarrow$ kỳ vọng ~ 1 kết quả "có ý nghĩa" giả.

```

from statsmodels.stats.multitest import multipletests

p_values = [0.001, 0.01, 0.02, 0.04, 0.2, 0.5]
reject, p_adj, _, _ = multipletests(p_values, alpha=0.05, method="fdr_bh") # Benjamini-Hochberg

```

4.4. Bootstrap — khoảng tin cậy cho mọi thống kê

Khi không có công thức giải tích (median, AUC, chênh lệch tỷ lệ phức tạp), bootstrap là công cụ vạn năng.

```

def bootstrap_ci(data, stat_fn, n_boot: int = 5000, alpha: float = 0.05, seed: int = 0):
    rng = np.random.default_rng(seed)
    data = np.asarray(data)
    stats_ = np.array([stat_fn(rng.choice(data, size=len(data), replace=True))
                       for _ in range(n_boot)])
    return np.quantile(stats_, [alpha / 2, 1 - alpha / 2])

print("95% CI median charges:", bootstrap_ci(churn_yes, np.median))

# scipy >= 1.7 có sẵn (phương pháp BCa chính xác hơn percentile)
res = stats.bootstrap((churn_yes.to_numpy(),), np.median, n_resamples=5000,
                      method="BCa", random_state=0)
print(res.confidence_interval)

```

4.5. A/B Testing chuẩn production

Bước 1 — Tính cỡ mẫu (power analysis) TRƯỚC khi chạy

```

from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

baseline, mde = 0.18, 0.02 # churn 18%, muốn phát hiện giảm 2 điểm %
effect = proportion_effectsize(baseline, baseline - mde)

```

```
n_per_group = NormalIndPower().solve_power(effect_size=effect, alpha=0.05, power=0.8,
                                             ratio=1.0, alternative="two-sided")
print(f"Cần ~{int(np.ceil(n_per_group))}, khách mỗi nhóm")
```

Bước 2 — Kiểm tra Sample Ratio Mismatch (SRM)

```
def srm_check(n_control: int, n_treatment: int, expected_ratio: float = 0.5) -> float:
    total = n_control + n_treatment
    expected = [total * (1 - expected_ratio), total * expected_ratio]
    return stats.chisquare([n_control, n_treatment], f_exp=expected).pvalue
```

$p < 0.001$ -> phân bố ngẫu nhiên có lỗi -> KHÔNG tin kết quả thí nghiệm

Bước 3 — Phân tích kết quả

```
def analyze_ab(conv_c: int, n_c: int, conv_t: int, n_t: int, alpha: float = 0.05) -> dict:
    p_c, p_t = conv_c / n_c, conv_t / n_t
    se = np.sqrt(p_c * (1 - p_c) / n_c + p_t * (1 - p_t) / n_t)
    z_crit = stats.norm.ppf(1 - alpha / 2)
    diff = p_t - p_c
    _, p_value = proportions_ztest([conv_t, conv_c], [n_t, n_c])
    return {
        "control": p_c, "treatment": p_t,
        "abs_diff": diff, "rel_lift": diff / p_c,
        "ci95": (diff - z_crit * se, diff + z_crit * se),
        "p_value": p_value,
    }
```

Kỹ thuật nâng cao:

- **CUPED** (Controlled-experiment Using Pre-Experiment Data): giảm phương sai 20-50% bằng covariate trước thí nghiệm.
- **Sequential testing / mSPRT**: cho phép "nhìn trộm" kết quả mà không lạm phát sai lầm loại I.
- **Bayesian A/B**: trả lời trực tiếp "xác suất B tốt hơn A là bao nhiêu".

CUPED: $Y_{adj} = Y - \theta (X - \text{mean}(X))$, $\theta = \text{cov}(X, Y) / \text{var}(X)$

```
def cuped(y: np.ndarray, x_pre: np.ndarray) -> np.ndarray:
    theta = np.cov(x_pre, y)[0, 1] / np.var(x_pre, ddof=1)
    return y - theta * (x_pre - x_pre.mean())
```

Bayesian A/B với Beta-Binomial

```
def prob_b_better(conv_a, n_a, conv_b, n_b, draws=200_000, seed=0):
    rng = np.random.default_rng(seed)
    a = rng.beta(1 + conv_a, 1 + n_a - conv_a, draws)
    b = rng.beta(1 + conv_b, 1 + n_b - conv_b, draws)
    return (b > a).mean(), np.mean(b / a - 1) # P(B>A), expected relative lift
```

4.6. Hồi quy thống kê để giải thích (statsmodels)

Mô hình ML tối ưu dự đoán; hồi quy thống kê tối ưu **diễn giải** (hệ số, p-value, CI).

```
import statsmodels.formula.api as smf

model = smf.logit(
    "churn ~ C(contract, Treatment('two-year')) + tenure_months + monthly_charges"
    " + support_calls + C(payment_method)",
    data=df.dropna(subset=["payment_method"]),
).fit(displ=False)
print(model.summary())

odds_ratios = pd.DataFrame({
    "OR": np.exp(model.params),
    "CI_low": np.exp(model.conf_int()[0]),
    "CI_high": np.exp(model.conf_int()[1]),
    "p": model.pvalues,
}).round(3)
# OR(support_calls)=1.42 => mô~i cuộc gọi hô~ trợ thêm, odds churn tăng 42% (giữ các biê~n khác cô~
định)
```

Tương quan $\neq$ nhân quả. Hệ số hồi quy chỉ là nhân quả khi không có biến gây nhiễu (confounder) bị bỏ sót. Muốn kết luận nhân quả: A/B test, hoặc các phương pháp quasi-experiment (Difference-in-Differences, Propensity Score Matching, Regression Discontinuity, Instrumental Variables) — thư viện [DoWhy](#), [EconML](#), [CausalML](#).

4.7. Phân tích chuỗi thời gian cơ bản

```
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import adfuller, kpss

daily = df.set_index("signup_date").resample("D")["customer_id"].count()

# Phân rã xu hướng - mùa vụ - phần dư
stl = STL(daily, period=7, robust=True).fit()
stl.plot()

# Kiểm định tính dừng: ADF (H0: không dừng), KPSS (H0: dừng) — dùng cả hai
print("ADF p =", adfuller(daily.dropna())[1])
print("KPSS p =", kpss(daily.dropna(), regression="c", nlags="auto")[1])
```

4.8. Nghịch lý Simpson — bài học kinh điển

Tỷ lệ churn của chương trình khuyến mãi A có thể *thấp hơn* B ở **mọi** phân khúc, nhưng *cao hơn* khi gộp chung — vì A được áp dụng nhiều hơn cho phân khúc rủi ro cao. Luôn phân tích theo phân khúc (stratify) với các biến gây nhiễu quan trọng.

```
def simpson_check(df: pd.DataFrame, treatment: str, outcome: str, stratum: str) -> pd.DataFrame:
    overall = df.groupby(treatment)[outcome].mean().rename("overall")
    by_stratum = df.groupby([stratum, treatment])[outcome].mean().unstack()
    return by_stratum, overall
```

Checklist Chương 4

- ☐ Kết luận dựa trên kiểm định phù hợp, đã kiểm tra giả định.
- ☐ Báo cáo effect size + khoảng tin cậy, không chỉ p-value.
- ☐ Hiệu chỉnh đa kiểm định khi kiểm định nhiều giả thuyết.
- ☐ A/B test: tính cỡ mẫu trước, kiểm tra SRM, không dừng sớm tùy tiện.
- ☐ Phân biệt rõ tương quan và nhân quả; kiểm tra Simpson's paradox.

CHƯƠNG 5. TIỀN XỬ LÝ DỮ LIỆU & FEATURE ENGINEERING

5.1. Nguyên tắc số 1: Fit trên train, Transform trên mọi tập

Mọi phép biến đổi **học tham số từ dữ liệu** (mean để impute, min/max để scale, mapping của target encoding, vocabulary của TF-IDF...) phải được **fit chỉ trên tập train**, rồi **transform** cho validation/test/production. Cách an toàn nhất: gói tất cả vào `sklearn.pipeline.Pipeline`.

Làm sạch (stateless) → Split → Pipeline[Impute → Encode → Scale → Model]
(an toàn trước split) (fit chỉ trên train, nằm trong CV)

Loại bước	Ví dụ	Làm trước split được không?
Stateless (không học từ dữ liệu)	Sửa chính tả, chuẩn hóa chuỗi, parse ngày, tạo <code>tenure_years = tenure/12</code> , xóa trùng lặp chính xác	✅ Được
Stateful (học từ dữ liệu)	Impute mean/median, scaling, encoding, feature selection, SMOTE, PCA	❌ Phải nằm trong Pipeline

5.2. Làm sạch dữ liệu (Data Cleaning)

```
# src/churn/features/clean.py
import unicodedata

import numpy as np
import pandas as pd

def normalize_text(s: pd.Series) -> pd.Series:
    """Chuẩn hóa chuỗi: Unicode NFC, bỏ khoảng trắng thừa, chữ thường."""
    return (s.astype("string")
            .map(lambda x: unicodedata.normalize("NFC", x) if isinstance(x, str) else x)
            .str.strip()
            .str.replace(r"\s+", " ", regex=True)
            .str.lower()
            .astype(object)
            .where(lambda x: x.notna(), np.nan)) # pd.NA -> np.nan cho sklearn

def clean_churn(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    # 1. Trùng lặp: toàn dòng, sau đó theo khóa nghiệp vụ (giữ ba'n ghi mới nhất)
    out = out.drop_duplicates()
    out = out.sort_values("signup_date").drop_duplicates("customer_id", keep="last")

    # 2. Chuẩn hóa phân loại + gom các biến thể
    for col in ["contract", "payment_method", "region"]:
        out[col] = normalize_text(out[col])
    out["contract"] = out["contract"].replace({"monthly": "month-to-month", "1-year": "one-year"})
```

```
# 3. Giá trị vô lý theo nghiệp vụ -> NaN (đề' imputer xử' lý), KHÔNG tự ý xóa dòng
out.loc[~out["age"].between(18, 100), "age"] = np.nan
out.loc[out["monthly_charges"] <= 0, "monthly_charges"] = np.nan

# 4. Kiê' u dữ liệu
out["signup_date"] = pd.to_datetime(out["signup_date"], errors="coerce", utc=False)
return out.reset_index(drop=True)
```

5.3. Xử lý giá trị thiếu (Imputation)

Phương pháp	Khi nào dùng	sklearn
Xóa dòng	Thiếu ít (<1-2%), MCAR, dữ liệu nhiều	dropna
Xóa cột	Thiếu > 60-70% và không mang tín hiệu	—
Mean / Median	Baseline, số; median bền với ngoại lai	SimpleImputer(strategy="median")
Mode / hằng số "missing"	Phân loại	SimpleImputer(strategy="constant", fill_value="missing")
Cờ thiếu (indicator)	Thiếu mang thông tin (MNAR)	SimpleImputer(add_indicator=True), MissingIndicator
KNN	Quan hệ cục bộ, dữ liệu vừa	KNNImputer
Iterative (MICE)	MAR, quan hệ đa biến	IterativeImputer (experimental)
Forward/backward fill, nội suy	Chuỗi thời gian	ffill, interpolate(method="time")
Đề mô hình tự xử lý	LightGBM/XGBoost/CatBoost/HistGradientBoosting xử lý NaN gốc	—

```
from sklearn.experimental import enable_iterative_imputer # noqa: F401
from sklearn.impute import IterativeImputer, KNNImputer, SimpleImputer
from sklearn.ensemble import ExtraTreesRegressor

median_imp = SimpleImputer(strategy="median", add_indicator=True)
knn_imp = KNNImputer(n_neighbors=5, weights="distance") # nên scale trước khi dùng KNN
mice_imp = IterativeImputer(
    estimator=ExtraTreesRegressor(n_estimators=50, random_state=0),
    max_iter=10, random_state=0,
)
```

Mẹo thực tế: So sánh các chiến lược impute **bằng cross-validation trên metric cuối cùng**, không phải bằng việc impute "giống thật" nhất.

5.4. Xử lý ngoại lai

Kỹ thuật	Mô tả	Ghi chú
Xóa	Loại bản ghi lỗi chắc chắn	Chỉ khi xác nhận là lỗi; không xóa trên tập test
Winsorize / Clipping	Cắt về percentile [p1, p99]	Học ngưỡng trên train

Kỹ thuật	Mô tả	Ghi chú
Biến đổi	log, Box-Cox, Yeo-Johnson	Giảm ảnh hưởng đuôi dài
Robust scaler	Dùng median & IQR	Chương 6
Mô hình bền vững	Mô hình cây, Huber loss	Ít nhạy với ngoại lai ở X

```

from sklearn.base import BaseEstimator, TransformerMixin, OneToOneFeatureMixin

class Winsorizer(OneToOneFeatureMixin, TransformerMixin, BaseEstimator):
    """Cắt giá trị về [q_low, q_high] học từ tập train. Tương thích Pipeline & set_output."""

    def __init__(self, q_low: float = 0.01, q_high: float = 0.99):
        self.q_low = q_low
        self.q_high = q_high

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.lower_ = np.nanquantile(X, self.q_low, axis=0)
        self.upper_ = np.nanquantile(X, self.q_high, axis=0)
        self.n_features_in_ = X.shape[1]
        return self

    def transform(self, X):
        return np.clip(np.asarray(X, dtype=float), self.lower_, self.upper_)

```

5.5. Mã hóa biến phân loại (Categorical Encoding)

Encoder	Cardinality	Mô hình phù hợp	Ghi chú
One-Hot	Thấp (< 15-20)	Tuyến tính, NN	<code>handle_unknown="infrequent_if_exist"</code> , <code>min_frequency</code> để gom mức hiếm
Ordinal	Có thứ tự tự nhiên	Mọi mô hình	<code>basic < standard < premium</code>
Ordinal (tùy ý)	Bất kỳ	Mô hình cây	Cây tự tách được
Frequency / Count	Cao	Cây	Đơn giản, không leakage target
Target / Mean encoding	Cao	Mọi mô hình	Phải dùng cross-fitting để tránh leakage
WoE	Trung bình	Logistic (scorecard)	Diễn giải tốt
Hashing	Rất cao (ID, URL)	Tuyến tính	Có xung đột hash
Embedding	Rất cao	Neural Network	Học được biểu diễn
Native categorical	Bất kỳ	LightGBM, CatBoost, HistGB	CatBoost dùng ordered target statistics

```

from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, TargetEncoder

ohe = OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=0.01,
                    sparse_output=False)

contract_order = [["month-to-month", "one-year", "two-year"]]
ordinal = OrdinalEncoder(categories=contract_order, handle_unknown="use_encoded_value",
                        unknown_value=-1)

# sklearn >= 1.3: TargetEncoder tự động cross-fitting trong fit_transform (chống leakage)
target_enc = TargetEncoder(target_type="binary", smooth="auto", cv=5, random_state=0)

```

Frequency encoder tự viết

```

class FrequencyEncoder(OneToOneFeatureMixin, TransformerMixin, BaseEstimator):
    def __init__(self, normalize: bool = True):
        self.normalize = normalize

    def fit(self, X, y=None):
        X = pd.DataFrame(X)
        self.maps_ = [X[c].value_counts(normalize=self.normalize).to_dict() for c in X.columns]
        self.n_features_in_ = X.shape[1]
        return self

    def transform(self, X):
        X = pd.DataFrame(X)
        return np.column_stack([X.iloc[:, i].map(m).fillna(0).to_numpy(dtype=float)
                                for i, m in enumerate(self.maps_)])

```

5.6. Feature Engineering — nơi tạo ra khác biệt lớn nhất

a) Feature nghiệp vụ (domain features)

```

class ChurnFeatures(TransformerMixin, BaseEstimator):
    """Feature stateless dựa trên tri thức nghiệp vụ. Đặt ở đầu Pipeline."""

    def __init__(self, reference_date: str | None = None):
        self.reference_date = reference_date

    def fit(self, X: pd.DataFrame, y=None):
        # Ngày tham chiếu học 1 lần lúc fit (hoặc lấy từ config) -> train/test/production nhất quán
        self.reference_date_ = (pd.Timestamp(self.reference_date) if self.reference_date
                                else X["signup_date"].max())
        self.feature_names_out_ = np.asarray(self.transform(X).columns, dtype=object)
        return self

    def transform(self, X: pd.DataFrame) -> pd.DataFrame:
        X = X.copy()
        X["days_since_signup"] = (self.reference_date_ - X["signup_date"]).dt.days
        X["signup_month"] = X["signup_date"].dt.month
        X["signup_dow"] = X["signup_date"].dt.dayofweek
        X["avg_charge_per_month"] = X["total_charges"] / X["tenure_months"].clip(lower=1)
        X["charge_vs_expected"] = X["total_charges"] / (X["monthly_charges"] *
                                                         X["tenure_months"].clip(lower=1))
        X["calls_per_year"] = X["support_calls"] / (X["tenure_months"] / 12).clip(lower=1 / 12)
        X["is_new_customer"] = (X["tenure_months"] <= 3).astype(int)

```

```
X["high_value"] = (X["monthly_charges"] > 100).astype(int)
return X.drop(columns=["signup_date", "customer_id"], errors="ignore")

def get_feature_names_out(self, input_features=None):
    return self.feature_names_out_
```

b) Feature thời gian — mã hóa chu kỳ

```
def cyclical_encode(df: pd.DataFrame, col: str, period: int) -> pd.DataFrame:
    """Tháng 12 và tháng 1 gần nhau: dùng sin/cos thay vì số nguyên."""
    df[f"{col}_sin"] = np.sin(2 * np.pi * df[col] / period)
    df[f"{col}_cos"] = np.cos(2 * np.pi * df[col] / period)
    return df
```

c) Feature tổng hợp từ dữ liệu giao dịch (aggregations theo cửa sổ thời gian)

```
def window_aggregates(tx: pd.DataFrame, snapshot: str, windows=(7, 30, 90)) -> pd.DataFrame:
    """tx: customer_id, ts, amount. Chỉ dùng giao dịch TRƯỚC snapshot (point-in-time)."""
    t = pd.Timestamp(snapshot)
    tx = tx[tx["ts"] < t]
    feats = []
    for w in windows:
        sub = tx[tx["ts"] >= t - pd.Timedelta(days=w)]
        agg = sub.groupby("customer_id")["amount"].agg(["count", "sum", "mean", "std", "max"])
        agg.columns = [f"amt_{c}_{w}d" for c in agg.columns]
        feats.append(agg)
    out = pd.concat(feats, axis=1).fillna(0)
    # Feature xu hướng: hoạt động gần đây so với dài hạn
    out["trend_7_vs_90"] = out["amt_sum_7d"] / (out["amt_sum_90d"] / (90 / 7) + 1e-9)
    last = tx.groupby("customer_id")["ts"].max()
    out["recency_days"] = (t - last).dt.days
    return out
```

d) Feature văn bản

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=5, max_features=20_000, sublinear_tf=True)
# Tiếng Việt: tách từ trước bằng underthesea / pyvi để có "hợp_đồng", "khấu_nội"
# Nâng cao: embeddings từ sentence-transformers (vd. "bkai-foundation-models/vietnamese-bi-encoder")
```

e) Feature tương tác & đa thức

```
from sklearn.preprocessing import PolynomialFeatures, SplineTransformer

poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
spline = SplineTransformer(n_knots=5, degree=3) # quan hệ phi tuyến mượt cho mô hình tuyến tính
```

f) Binning (rời rạc hóa)

```
from sklearn.preprocessing import KBinsDiscretizer

kbins = KBinsDiscretizer(n_bins=10, encode="ordinal", strategy="quantile")
# strategy="kmeans" hoặc binning tối ưu theo target: thư viện `optbinning`
```

5.7. Pipeline tiền xử lý hoàn chỉnh (production-ready)

```
# src/churn/features/build.py
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import StandardScaler

NUM_COLS = ["age", "tenure_months", "monthly_charges", "total_charges", "support_calls",
            "data_usage_gb", "days_since_signup", "avg_charge_per_month",
            "charge_vs_expected", "calls_per_year"]
LOW_CARD_COLS = ["contract", "payment_method"]
HIGH_CARD_COLS = ["region"]
PASSTHROUGH = ["is_new_customer", "high_value", "signup_month", "signup_dow"]

def build_preprocessor(scale: bool = True) -> Pipeline:
    numeric = Pipeline([
        ("winsor", Winsorizer(0.01, 0.99)),          # clip trước (giữ NaN), tránh clip cột có 0/1
        ("impute", SimpleImputer(strategy="median", add_indicator=True)),
        ("scale", StandardScaler() if scale else "passthrough"),
    ])
    low_card = Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="missing")),
        ("ohe", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=0.01,
                               sparse_output=False)),
    ])
    high_card = Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="missing")),
        ("te", TargetEncoder(target_type="binary", random_state=0)),
    ])
    columns = ColumnTransformer(
        [
            ("num", numeric, NUM_COLS),
            ("low", low_card, LOW_CARD_COLS),
            ("high", high_card, HIGH_CARD_COLS),
            ("pass", "passthrough", PASSTHROUGH),
        ],
        remainder="drop",          # an toàn: cột lạ trong production bị bỏ qua
        verbose_feature_names_out=False,
    )
    return Pipeline([("domain", ChurnFeatures()), ("columns", columns)])

preprocessor = build_preprocessor()
# X_train_t = preprocessor.fit_transform(X_train, y_train)  # TargetEncoder cần y
# X_test_t = preprocessor.transform(X_test)
# preprocessor.get_feature_names_out()
```

Lưu ý: trong production nên truyền `reference_date` = ngày snapshot từ config. Nếu để trống, ngày tham chiếu được học một lần lúc `fit` (ngày đăng ký lớn nhất của tập train) và dùng lại cho mọi lần `transform`.

5.8. Lựa chọn đặc trưng (Feature Selection)

Nhóm	Phương pháp	Ưu	Nhược
Filter	Variance threshold, tương quan, MI, χ^2 , IV	Nhanh	Bỏ qua tương tác
Wrapper	RFE/RFE CV, Sequential Feature Selection	Tính đến mô hình	Chậm
Embedded	L1 (Lasso), tree importance	Cân bằng	Phụ thuộc mô hình
Model-agnostic	Permutation importance , SHAP, Boruta , null importance	Đáng tin cậy	Tốn tính toán

```

from sklearn.feature_selection import RFECV, SelectFromModel, VarianceThreshold
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

# Embedded: L1 logistic
l1_selector = SelectFromModel(
    LogisticRegression(penalty="l1", solver="liblinear", C=0.1, class_weight="balanced")
)

# Wrapper: RFE + CV tự chọn số lượng feature tối ưu
rfecv = RFECV(
    estimator=LogisticRegression(max_iter=2000),
    step=1, cv=StratifiedKFold(5, shuffle=True, random_state=0),
    scoring="average_precision", min_features_to_select=5, n_jobs=-1,
)

```

Null importance — phát hiện feature "ảo"

```

from lightgbm import LGBMClassifier

def null_importance(X: pd.DataFrame, y: pd.Series, n_runs: int = 30, seed: int = 0) ->
pd.DataFrame:
    """So sánh importance thật với importance khi target bị xáo trộn."""
    rng = np.random.default_rng(seed)
    params = dict(n_estimators=200, learning_rate=0.05, verbose=-1, importance_type="gain")
    actual = LGBMClassifier(**params).fit(X, y).feature_importances_
    null = np.array([
        LGBMClassifier(**params, random_state=i).fit(X,
        rng.permutation(y.to_numpy())).feature_importances_
        for i in range(n_runs)
    ])
    score = np.log(1e-10 + actual / (1 + np.percentile(null, 75, axis=0)))
    return pd.DataFrame({"actual": actual, "null_p75": np.percentile(null, 75, axis=0),
        "score": score}, index=X.columns).sort_values("score", ascending=False)

```

5.9. Xử lý mất cân bằng lớp (Imbalanced Data)

Kỹ thuật	Mô tả	Khi nào dùng
Không làm gì + chọn metric đúng + chỉnh ngưỡng	PR-AUC, threshold tuning	Mặc định nên thử đầu tiên
Class weights	Phạt lỗi lớp thiểu số nặng hơn	Hầu hết mô hình hỗ trợ class_weight / scale_pos_weight
Random undersampling	Bớt lớp đa số	Dữ liệu rất lớn
Random oversampling	Nhân bản lớp thiểu số	Dữ liệu nhỏ
SMOTE / ADASYN / Borderline-SMOTE	Sinh mẫu tổng hợp	Mô hình tuyến tính/KNN; ít giúp GBM
SMOTE-NC	SMOTE cho dữ liệu hỗn hợp số + phân loại	
Focal loss	Tập trung vào mẫu khó	Deep learning
Ensemble resampling	BalancedRandomForest, EasyEnsemble	

```

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline  # BẮT BUỘC dùng pipeline cu'a imblearn

pipe = ImbPipeline([
    ("prep", build_preprocessor()),
    ("smote", SMOTE(sampling_strategy=0.5, k_neighbors=5, random_state=0)), # chỉ chạy khi fit
    ("clf", LogisticRegression(max_iter=2000)),
])

```

Cảnh báo quan trọng:

1. **Không bao giờ** oversample trước khi split hoặc trước cross-validation → mẫu tổng hợp rò rỉ sang tập validation → metric ảo.
2. Resampling/class weight **làm sai lệch xác suất** dự đoán. Nếu cần xác suất chính xác (định giá rủi ro), phải **calibrate lại** (Chương 9) hoặc hiệu chỉnh prior.
3. Kinh nghiệm thực tế với GBM: `scale_pos_weight` + tối ưu ngưỡng thường tốt ngang hoặc hơn SMOTE.

Checklist Chương 5

- ☐ Bước stateful nằm trong Pipeline, fit chỉ trên train.
- ☐ Có chiến lược missing rõ ràng (kèm cờ missing khi missing mang tín hiệu).
- ☐ Encoding phù hợp cardinality; target encoding có cross-fitting.
- ☐ Feature engineering đảm bảo point-in-time, có tài liệu ý nghĩa nghiệp vụ.
- ☐ Feature selection được đánh giá trong CV, không trên toàn bộ dữ liệu.
- ☐ Xử lý mất cân bằng nằm trong pipeline; đã cân nhắc ảnh hưởng tới calibration.

CHƯƠNG 6. CHUẨN HÓA & BIẾN ĐỔI DỮ LIỆU (SCALING & TRANSFORMATION)

6.1. Khi nào cần chuẩn hóa?

Nhóm mô hình	Cần scale?	Lý do
Dựa trên khoảng cách: KNN, K-Means, SVM (RBF), DBSCAN	✅ Bắt buộc	Biến có thang đo lớn chi phối khoảng cách
Tối ưu bằng gradient: Linear/Logistic (có regularization), Neural Network	✅ Bắt buộc	Hội tụ nhanh hơn; penalty L1/L2 công bằng giữa các hệ số
PCA, LDA	✅ Bắt buộc	PCA tối đa phương sai → biến thang lớn chiếm ưu thế
Mô hình cây: Decision Tree, Random Forest, GBM	❌ Không cần	Chỉ dựa trên thứ tự giá trị (split thresholds)
Naive Bayes	Tùy loại	Gaussian NB không cần, nhưng cần phân phối gần chuẩn

6.2. Các phương pháp scaling

Phương pháp	Công thức	Miền giá trị	Bền với ngoại lai	Dùng khi
StandardScaler (Z-score)	$z = \frac{x - \mu}{\sigma}$	$\sim (-3, 3)$	❌	Mặc định cho tuyến tính, NN, SVM
MinMaxScaler	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	$[0, 1]$	❌	NN với sigmoid, ảnh (pixel), cần miền cố định
RobustScaler	$x' = \frac{x - \text{median}}{IQR}$	Không cố định	✅	Dữ liệu nhiều ngoại lai
MaxAbsScaler	$x' = \frac{x}{\max x }$	$[-1, 1]$	❌	Dữ liệu thưa (sparse) — giữ số 0
Normalizer (theo dòng)	$x' = \frac{x}{\ x\ _2}$	Vector đơn vị	—	Văn bản TF-IDF, cosine similarity

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import (MaxAbsScaler, MinMaxScaler, Normalizer, RobustScaler,
                                   StandardScaler)

from churn.data.synthetic import make_churn_data

df = make_churn_data()
x = df[["monthly_charges"]].dropna() # có ~30 ngoại lai x8
```

```

scalers = {
    "standard": StandardScaler(),
    "minmax": MinMaxScaler(),
    "robust": RobustScaler(quantile_range=(25, 75)),
    "maxabs": MaxAbsScaler(),
}
summary = {name: pd.Series(s.fit_transform(x).ravel()).describe()[["mean", "std", "min", "50%", "max"]]}
for name, s in scalers.items():
    print(pd.DataFrame(summary).round(3))
# MinMax: phân lớn dữ liệu bị ép vào [0, 0.15] vì 1 ngoại lai -> mất phân gia'i
# Robust: phân thân phân phối giữ nguyên tỷ lệ

```

6.3. Biến đổi phân phối (làm "chuẩn hóa" hình dạng)

Phương pháp	Điều kiện	Ghi chú
Log: $\log(x)$, $\log(1+x)$	$x > 0$ / $x \geq 0$	Đơn giản, dễ diễn giải; np.log1p
Căn bậc hai, căn bậc ba	$x \geq 0$ / mọi x	Lệch vừa phải; dữ liệu đếm
Box-Cox	$x > 0$	Tự tìm λ tối ưu (MLE)
Yeo-Johnson	Mọi x (kể cả âm, 0)	Mặc định của PowerTransformer
QuantileTransformer	Mọi x	Ép về Uniform/Normal; phi tuyến mạnh, bền với ngoại lai; cần nhiều dữ liệu
Rank / percentile	Mọi x	Rank Gauss trong các cuộc thi

$$\text{Box-Cox: } x^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln x & \lambda = 0 \end{cases}$$

```

from scipy import stats
from sklearn.preprocessing import FunctionTransformer, PowerTransformer, QuantileTransformer

usage = df[["data_usage_gb"]].dropna()

transforms = {
    "raw": FunctionTransformer(),
    "log1p": FunctionTransformer(np.log1p, inverse_func=np.expm1, feature_names_out="one-to-one"),
    "box-cox": PowerTransformer(method="box-cox"), # yêu cầu > 0
    "yeo-johnson": PowerTransformer(method="yeo-johnson"),
    "quantile-normal": QuantileTransformer(output_distribution="normal", n_quantiles=1000,
                                           random_state=0),
}
for name, t in transforms.items():
    z = t.fit_transform(usage).ravel()
    print(f"{name:16s} skew={stats.skew(z):6.3f} kurtosis={stats.kurtosis(z):6.3f}")

pt = PowerTransformer(method="box-cox").fit(usage)
print("λ tối ưu =", pt.lambdas_)

```

Biến đổi target cho hồi quy

Khi target lệch phải (doanh thu, giá nhà), huấn luyện trên $\log(y)$ thường tốt hơn;

`TransformedTargetRegressor` tự động biến đổi ngược khi dự đoán:

```
from lightgbm import LGBMRegressor
from sklearn.compose import TransformedTargetRegressor

reg = TransformedTargetRegressor(
    regressor=LGBMRegressor(n_estimators=500, learning_rate=0.05, verbose=-1),
    func=np.log1p, inverse_func=np.expm1,
)
# Lưu ý: expm1(E[log y]) ≈ median chứ không phải mean của y (Jensen) -> có thể cần hiệu chỉnh
# smearing
```

6.4. Giảm chiều (Dimensionality Reduction)

Phương pháp	Loại	Dùng cho	Ghi chú
PCA	Tuyến tính	Nén feature, khử tương quan	Cần scale trước
TruncatedSVD	Tuyến tính	Ma trận thưa (TF-IDF)	LSA
LDA (Linear Discriminant)	Có giám sát	Tối đa phân tách lớp	$\leq K-1$ chiều
t-SNE	Phi tuyến	Chỉ trực quan hóa	Không bảo toàn khoảng cách toàn cục
UMAP	Phi tuyến	Trực quan hóa + làm feature	Nhanh hơn t-SNE, <code>umap-learn</code>
Autoencoder	Phi tuyến	Dữ liệu lớn/phức tạp	Deep learning

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline

num = df[["age", "tenure_months", "monthly_charges", "total_charges",
         "support_calls", "data_usage_gb"]].dropna()

pca_pipe = make_pipeline(StandardScaler(), PCA(n_components=0.95, svd_solver="full"))
Z = pca_pipe.fit_transform(num)
pca = pca_pipe.named_steps["pca"]
print("Số thành phần giữ 95% phương sai:", pca.n_components_)
print("Explained variance ratio:", pca.explained_variance_ratio_.round(3))

loadings = pd.DataFrame(pca.components_.T, index=num.columns,
                        columns=[f"PC{i + 1}" for i in range(pca.n_components_)])
print(loadings.round(2))
```

6.5. Ghép nối: scaling khác nhau cho từng nhóm cột

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

skewed = ["data_usage_gb", "total_charges"]
heavy_outlier = ["monthly_charges"]
regular = ["age", "tenure_months", "support_calls"]

scaling = ColumnTransformer([
    ("skewed", Pipeline([("imp", SimpleImputer(strategy="median")),
                        ("pow", PowerTransformer(method="yeo-johnson"))])), skewed),
    ("robust", Pipeline([("imp", SimpleImputer(strategy="median")),
                        ("rob", RobustScaler())])), heavy_outlier),
    ("std", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("std", StandardScaler())])), regular),
]).set_output(transform="pandas") # giữ tên cột -> dễ debug, dễ giải thích
```

6.6. Pitfalls

1. **Data leakage do scale trên toàn bộ dữ liệu:** `StandardScaler().fit(X)` trước khi split → mean/std chứa thông tin tập test. Luôn đặt scaler trong Pipeline.
2. **Scale biến one-hot:** thường không cần; với mô hình có regularization, có thể giữ 0/1 để dễ diễn giải.
3. **Scale target phân loại:** không bao giờ.
4. **Box-Cox với giá trị ≤ 0 :** lỗi — dùng Yeo-Johnson.
5. **Quên lưu scaler:** production phải dùng *đúng* scaler đã fit lúc train → lưu cả pipeline (`joblib`), không lưu riêng model.
6. **Thay đổi phân phối trong production:** min/max của `MinMaxScaler` cố định từ train → giá trị mới có thể nằm ngoài $[0, 1]$. Cần `clip=True` hoặc giám sát drift.

Checklist Chương 6

- ☐ Chỉ scale khi mô hình cần; mô hình cây có thể bỏ qua.
- ☐ Chọn scaler theo phân phối & ngoại lai; biến lệch dùng log/Yeo-Johnson.
- ☐ Scaler nằm trong Pipeline, được serialize cùng mô hình.
- ☐ PCA/giảm chiều được fit chỉ trên train và đã kiểm tra explained variance.

CHƯƠNG 7. CHIA DỮ LIỆU & CHIẾN LƯỢC VALIDATION

"Chiến lược validation của bạn quan trọng hơn mô hình của bạn." — Mọi Kaggle Grandmaster.

Mục tiêu duy nhất của việc chia dữ liệu: **ước lượng trung thực hiệu năng của mô hình trên dữ liệu tương lai mà nó chưa từng thấy**. Vì vậy cách chia phải **mô phỏng đúng cách mô hình được dùng trong production**.

7.1. Ba tập dữ liệu và vai trò

Tập	Vai trò	Được dùng để	Tỷ lệ thường gặp
Train	Học tham số mô hình	<code>fit</code>	60-80%
Validation	Chọn mô hình, siêu tham số, ngưỡng, early stopping	Ra quyết định	10-20% (hoặc dùng CV)
Test (hold-out)	Ước lượng hiệu năng cuối cùng	Chỉ đánh giá 1 lần	10-20%

Quy tắc vàng: Tập test bị "đốt" ngay khi bạn dùng nó để ra bất kỳ quyết định nào. Nếu bạn lặp lại "train → xem test → chỉnh → xem test", test đã trở thành validation.

7.2. Bảng chọn chiến lược chia

Tình huống	Chiến lược	sklearn
Dữ liệu i.i.d., cân bằng	Random split / KFold	<code>train_test_split</code> , <code>KFold</code>
Phân loại mất cân bằng	Stratified	<code>StratifiedKFold</code> , <code>stratify=y</code>
Nhiều bản ghi / một thực thể (khách hàng, bệnh nhân)	Group — một nhóm chỉ ở 1 tập	<code>GroupKFold</code> , <code>StratifiedGroupKFold</code> , <code>GroupShuffleSplit</code>
Dữ liệu có thời gian, dự đoán tương lai	Time-based (out-of-time)	<code>TimeSeriesSplit</code> , split theo ngày
Chuỗi thời gian có độ trễ nhãn	Time split + gap / purging / embargo	<code>TimeSeriesSplit(gap=...)</code>
Dữ liệu ít (< vài nghìn mẫu)	Repeated (Stratified) KFold	<code>RepeatedStratifiedKFold</code>
Vừa tune vừa ước lượng không bias	Nested CV	<code>GridSearchCV</code> lồng trong <code>cross_val_score</code>
Dữ liệu không gian (địa lý)	Spatial / block CV	<code>GroupKFold</code> theo ô lưới

7.3. Hold-out split cơ bản (stratified)

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

from churn.data.synthetic import make_churn_data
from churn.features.clean import clean_churn

df = clean_churn(make_churn_data())
X, y = df.drop(columns=["churn"], df["churn"])

# Chia 2 bước: 70% train / 15% valid / 15% test
X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42)
X_valid, X_test, y_valid, y_test = train_test_split(
    X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, random_state=42)

for name, t in [("train", y_train), ("valid", y_valid), ("test", y_test)]:
    print(f"{name:5s} n={len(t):6d} churn_rate={t.mean():.4f}")
```

7.4. Out-of-time split — chuẩn cho hầu hết bài toán kinh doanh

```
def time_split(df: pd.DataFrame, date_col: str, train_end: str, valid_end: str,
              gap_days: int = 0):
    """train: < train_end | (gap) | valid: [train_end+gap, valid_end) | test: >= valid_end + gap"""
    d = df[date_col]
    gap = pd.Timedelta(days=gap_days)
    train = df[d < pd.Timestamp(train_end)]
    valid = df[(d >= pd.Timestamp(train_end) + gap) & (d < pd.Timestamp(valid_end))]
    test = df[d >= pd.Timestamp(valid_end) + gap]
    return train, valid, test

train_df, valid_df, test_df = time_split(df, "signup_date", "2023-12-01", "2024-03-01",
                                         gap_days=30)
```

Trong production, khuyến nghị đánh giá thêm **"out-of-time" + "out-of-sample"**: test là khách hàng mới **và** ở giai đoạn sau.

7.5. Cross-Validation

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import (GroupKFold, RepeatedStratifiedKFold, StratifiedGroupKFold,
                                     StratifiedKFold, TimeSeriesSplit, cross_validate)
from sklearn.pipeline import make_pipeline

from churn.features.build import build_preprocessor

pipe = make_pipeline(build_preprocessor(), LogisticRegression(max_iter=2000))

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
scores = cross_validate(
    pipe, X_train, y_train, cv=cv,
    scoring={"roc_auc": "roc_auc", "pr_auc": "average_precision", "f1": "f1"},
    return_train_score=True, n_jobs=-1,
)
res = pd.DataFrame(scores)
print(res.agg(["mean", "std"]).T.round(4))
# train_score >> test_score => overfitting ; std lớn => mô hình không ổn định / dữ liệu ít
```

Group K-Fold — tránh rò rỉ theo thực thể

Nếu một khách hàng có nhiều snapshot (mỗi tháng 1 dòng), random split để cùng khách hàng xuất hiện ở cả train và valid → mô hình "nhớ" khách hàng → metric ảo.

```
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
groups = X_train["customer_id"]
for fold, (tr, va) in enumerate(sgkf.split(X_train, y_train, groups=groups)):
    assert set(groups.iloc[tr]).isdisjoint(groups.iloc[va])
```

Time Series Split (expanding / sliding window)

```
X_sorted = X_train.sort_values("signup_date")
y_sorted = y_train.loc[X_sorted.index]

# Lưu ý: gap tính theo SƠ MÃ~U (dòng), không pha'ỉ sô' ngày -> dữ liệu pha'ỉ được sắp xếp theo thời gian
tscv = TimeSeriesSplit(n_splits=5, gap=500, test_size=None, max_train_size=None)
# max_train_size=N -> sliding window (khi dữ liệu cũ không còn đại diện)
for fold, (tr, va) in enumerate(tscv.split(X_sorted)):
    print(fold, X_sorted.iloc[tr]["signup_date"].max().date(), "->",
          X_sorted.iloc[va]["signup_date"].min().date())
```

Expanding window:	Sliding window:
Fold 1: [TRAIN][gap][VAL]	Fold 1: [TRAIN][gap][VAL]
Fold 2: [TRAIN TRAIN][gap][VAL]	Fold 2: [TRAIN][gap][VAL]
Fold 3: [TRAIN TRAIN TRAIN][gap][VAL]	Fold 3: [TRAIN][gap][VAL]

Purged K-Fold với Embargo (tài chính, nhãn chống lẩn)

Khi nhãn tại thời điểm t phụ thuộc dữ liệu trong $[t, t+h]$ (ví dụ lợi nhuận 5 ngày), mẫu train gần ranh giới validation có nhãn "chống lẩn" với validation → cần loại bỏ (purge) và thêm vùng cấm (embargo) — xem López de Prado, *Advances in Financial ML, 2018*.

```
from collections.abc import Iterator

def purged_time_splits(dates: pd.Series, n_splits: int = 5, horizon: pd.Timedelta =
    pd.Timedelta(days=30),
                      embargo: pd.Timedelta = pd.Timedelta(days=7)) -> Iterator[tuple[np.ndarray,
    np.ndarray]]:
    dates = dates.reset_index(drop=True)
    order = np.argsort(dates.to_numpy())
    folds = np.array_split(order, n_splits)
    for va_idx in folds:
        va_start, va_end = dates.iloc[va_idx].min(), dates.iloc[va_idx].max()
```

```
mask = ((dates + horizon) < va_start) | (dates > va_end + embargo) # purge + embargo
tr_idx = np.where(mask.to_numpy())[0]
yield tr_idx, va_idx
```

7.6. Nested Cross-Validation — ước lượng không bias khi có tuning

```
from sklearn.model_selection import GridSearchCV, cross_val_score

inner = StratifiedKFold(5, shuffle=True, random_state=1)
outer = StratifiedKFold(5, shuffle=True, random_state=2)

search = GridSearchCV(
    pipe, param_grid={"logisticregression__C": [0.01, 0.1, 1, 10]},
    cv=inner, scoring="average_precision", n_jobs=-1,
)
nested_scores = cross_val_score(search, X_train, y_train, cv=outer, scoring="average_precision")
print(f"Nested CV PR-AUC: {nested_scores.mean():.4f} ± {nested_scores.std():.4f}")
```

Chọn tham số trên chính tập dùng để báo cáo điểm → điểm bị **lạc quan (optimistic bias)**.
Nested CV tách hai vai trò này.

7.7. Out-of-Fold (OOF) predictions — nền tảng cho stacking & threshold tuning

```
from sklearn.model_selection import cross_val_predict

oof_proba = cross_val_predict(pipe, X_train, y_train, cv=cv, method="predict_proba", n_jobs=-1)[: ,
1]
# oof_proba: mô~i mẫ~u train đư~c dự đoán bở~i mô hình KHÔNG thâ~y nó -> dùng đê~
# (1) chọn ngưỡng, (2) calibrate, (3) huấ~n luyện meta-model khi stacking
```

7.8. Danh mục Data Leakage — kẻ thù số 1

Loại leakage	Ví dụ	Cách phòng
Target leakage	Feature <code>refund_amount</code> chỉ có sau khi khách hủy	Kiểm tra thời điểm sinh của từng feature
Train-test contamination	Scale/impute/SMOTE/feature selection trên toàn bộ dữ liệu	Pipeline + CV
Temporal leakage	Random split cho dữ liệu thời gian; feature dùng dữ liệu tương lai	Time split, point-in-time join
Group leakage	Cùng khách hàng/bệnh nhân ở train và test	Group split
Duplicate leakage	Bản ghi gần trùng ở train và test	Dedup trước split (cả near-duplicate)

Loại leakage	Ví dụ	Cách phòng
Preprocessing leakage trong target encoding	Mean encoding tính cả target của chính dòng đó	Cross-fitting, TargetEncoder
Hyperparameter leakage	Tune trên test	Validation riêng / nested CV

Test tự động chống leakage

```
def assert_no_leakage(train: pd.DataFrame, test: pd.DataFrame, id_col: str, date_col: str | None = None):
    overlap = set(train[id_col]) & set(test[id_col])
    assert not overlap, f"{len(overlap)} ID xuất hiện ở cả train và test"
    if date_col:
        assert train[date_col].max() < test[date_col].min(), "Train chứa dữ liệu sau thời điểm test"
```

Checklist Chương 7

- ☐ Cách chia mô phỏng đúng kịch bản production (thời gian, thực thể mới).
- ☐ Stratify cho phân loại mất cân bằng; group split khi có nhiều bản ghi/thực thể.
- ☐ Tập test chỉ dùng 1 lần cuối cùng; mọi quyết định dựa trên validation/CV.
- ☐ Có test tự động kiểm tra leakage (ID, thời gian, trùng lặp).
- ☐ Báo cáo mean $\pm$ std qua các fold, không chỉ 1 con số.

CHƯƠNG 8. HUẤN LUYỆN MÔ HÌNH (MODEL TRAINING)

8.1. Lộ trình huấn luyện chuẩn

1. Baseline ngây thơ → 2. Baseline nghiệp vụ → 3. Mô hình tuyến tính → 4. GBM mặc định
→ 5. Tuning siêu tham số → 6. Ensemble (nếu đáng) → 7. Phân tích lỗi → lặp lại

Mỗi bước phải **vượt bước trước một cách có ý nghĩa thống kê** mới đáng giữ độ phức tạp tăng thêm.

8.2. Chọn thuật toán

Dữ liệu	Lựa chọn đầu tiên	Thay thế
Bảng (tabular), < 1M dòng	LightGBM / XGBoost / CatBoost	Random Forest, Logistic Regression
Bảng, cần giải thích tuyệt đối (ngân hàng, bảo hiểm)	Logistic Regression + WoE (scorecard), EBM (interpret)	GAM, cây quyết định nông
Bảng, nhiều biến phân loại	CatBoost	LightGBM native categorical
Văn bản	Fine-tune Transformer (PhoBERT cho tiếng Việt)	TF-IDF + Logistic Regression (baseline mạnh)
Ảnh	CNN / ViT pretrained (transfer learning)	—
Chuỗi thời gian	GBM + lag/rolling features, ETS/ARIMA, Prophet	N-BEATS, TFT, foundation models (Chronos, TimesFM)
Rất ít dữ liệu (< 1000)	Mô hình tuyến tính có regularization, RF	Bayesian models

Bias - Variance trade-off

Triệu chứng	Chẩn đoán	Hành động
Train thấp, Valid thấp	Underfitting (high bias)	Mô hình phức tạp hơn, thêm feature, giảm regularization
Train cao, Valid thấp hơn nhiều	Overfitting (high variance)	Thêm dữ liệu, regularization, giảm độ sâu, early stopping, bớt feature
Train cao, Valid cao, Test thấp	Distribution shift / leakage trong valid	Xem lại cách split, adversarial validation

8.3. Baseline

```
import numpy as np
import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.metrics import average_precision_score, roc_auc_score

# (Tiếp tục từ chương 7: X_train, X_valid, y_train, y_valid đã có)

# Baseline 1: ngây thơ – dự đoán xác suất = tỷ lệ lớp
dummy = DummyClassifier(strategy="prior").fit(X_train, y_train)
p = dummy.predict_proba(X_valid)[:, 1]
print("Dummy PR-AUC:", round(average_precision_score(y_valid, p), 4)) # = tỷ lệ positive

# Baseline 2: quy tắc nghiệp vụ hiện tại
def rule_based_score(X: pd.DataFrame) -> np.ndarray:
    return ((X["contract"] == "month-to-month").astype(int) * 2
            + (X["support_calls"] >= 3).astype(int)
            + (X["tenure_months"] < 6).astype(int)).to_numpy()

s = rule_based_score(X_valid)
print("Rule ROC-AUC:", round(roc_auc_score(y_valid, s), 4),
      "PR-AUC:", round(average_precision_score(y_valid, s), 4))
```

8.4. So sánh nhiều mô hình bằng cùng một CV

```
from lightgbm import LGBMClassifier
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline

from churn.features.build import build_preprocessor

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
pos_weight = (y_train == 0).sum() / (y_train == 1).sum()

candidates = {
    "logreg": Pipeline([("prep", build_preprocessor(scale=True)),
                        ("clf", LogisticRegression(C=1.0, class_weight="balanced",
max_iter=3000))]),
    "random_forest": Pipeline([("prep", build_preprocessor(scale=False)),
                              ("clf", RandomForestClassifier(n_estimators=500, min_samples_leaf=5,
class_weight="balanced_subsample",
n_jobs=-1, random_state=42))]),
    "hist_gb": Pipeline([("prep", build_preprocessor(scale=False)),
                        ("clf", HistGradientBoostingClassifier(max_iter=500, learning_rate=0.05,
early_stopping=True,
random_state=42))]),
    "lightgbm": Pipeline([("prep", build_preprocessor(scale=False)),
                          ("clf", LGBMClassifier(n_estimators=600, learning_rate=0.03,
num_leaves=31,
subsample=0.8, subsample_freq=1,
colsample_bytree=0.8,
scale_pos_weight=pos_weight, verbose=-1,
random_state=42))])
```

```

}

rows = []
for name, model in candidates.items():
    r = cross_validate(model, X_train, y_train, cv=cv, n_jobs=-1,
                      scoring={"roc_auc": "roc_auc", "pr_auc": "average_precision",
                              "neg_log_loss": "neg_log_loss"})
    rows.append({"model": name,
                **{k.replace("test_", ""): f"{v.mean():.4f} ± {v.std():.4f}"
                  for k, v in r.items() if k.startswith("test_")},
                "fit_time_s": round(r["fit_time"].mean(), 2)})
print(pd.DataFrame(rows).set_index("model"))

```

8.5. Gradient Boosting chuyên sâu

Siêu tham số quan trọng của LightGBM

Nhóm	Tham số	Ý nghĩa	Khoảng tìm kiếm điển hình
Tốc độ học	learning_rate	Bước học; nhỏ hơn + nhiều cây hơn = tốt hơn nhưng chậm	0.01 - 0.1
	n_estimators	Số cây — dùng early stopping , đừng tune trực tiếp	100 - 10000
Độ phức tạp	num_leaves	Số lá tối đa (tham số chính)	15 - 255
	max_depth	Giới hạn độ sâu	-1, 4 - 12
	min_child_samples	Số mẫu tối thiểu/lá — chống overfit	10 - 200
Ngẫu nhiên	subsample (+ subsample_freq=1)	Lấy mẫu dòng	0.5 - 1.0
	colsample_bytree	Lấy mẫu cột	0.4 - 1.0
Regularization	reg_alpha (L1), reg_lambda (L2)		1e-8 - 10 (log)
Mất cân bằng	scale_pos_weight / is_unbalance		neg/pos

Early stopping đúng cách

```

import lightgbm as lgb

prep = build_preprocessor(scale=False)
Xtr = prep.fit_transform(X_train, y_train)  # fit preprocessor CHI' trên train
Xva = prep.transform(X_valid)

model = lgb.LGBMClassifier(
    n_estimators=10_000, learning_rate=0.02, num_leaves=31, min_child_samples=50,
    subsample=0.8, subsample_freq=1, colsample_bytree=0.8, reg_lambda=1.0,
    verbose=-1, random_state=42,
)
model.fit(
    Xtr, y_train,

```

```

eval_set=[(Xva, y_valid)],
eval_metric="average_precision",
callbacks=[lgb.early_stopping(stopping_rounds=200, first_metric_only=True),
           lgb.log_evaluation(period=500)],
)
print("Best iteration:", model.best_iteration_)

```

Lưu ý: Tập dùng cho early stopping đã "bị nhìn" → không dùng nó để báo cáo điểm cuối cùng. Khi train lại trên train+valid, đặt $n_estimators = best_iteration \times (1 + \text{tỷ lệ dữ liệu tăng thêm})$ (ví dụ $\times 1.1-1.2$).

XGBoost & CatBoost

```

from catboost import CatBoostClassifier
from xgboost import XGBClassifier

xgb = XGBClassifier(
    n_estimators=10_000, learning_rate=0.02, max_depth=6, min_child_weight=5,
    subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0, tree_method="hist",
    eval_metric="aucpr", early_stopping_rounds=200, scale_pos_weight=pos_weight,
    random_state=42, n_jobs=-1,
)
xgb.fit(Xtr, y_train, eval_set=[(Xva, y_valid)], verbose=False)

# CatBoost: đưa thẳng cột phân loại dạng chuỗi, không cần encode
cat_cols = ["contract", "payment_method", "region"]
feature_cols = ["age", "tenure_months", "monthly_charges", "total_charges",
                "support_calls", "data_usage_gb", *cat_cols]

def to_catboost(X: pd.DataFrame) -> pd.DataFrame:
    X = X[feature_cols].copy()
    X[cat_cols] = X[cat_cols].fillna("missing").astype(str)
    return X

cb = CatBoostClassifier(
    iterations=10_000, learning_rate=0.03, depth=6, l2_leaf_reg=3,
    eval_metric="PRAUC", auto_class_weights="Balanced",
    early_stopping_rounds=200, random_seed=42, verbose=0,
)
cb.fit(to_catboost(X_train), y_train, cat_features=cat_cols,
      eval_set=(to_catboost(X_valid), y_valid), use_best_model=True)

```

8.6. Tối ưu siêu tham số (Hyperparameter Optimization)

Phương pháp	Ưu	Nhược	Công cụ
Grid Search	Đơn giản, toàn diện	Bùng nổ tổ hợp	GridSearchCV
Random Search	Hiệu quả hơn grid với nhiều tham số (Bergstra & Bengio, 2012)	Không học từ lần thử trước	RandomizedSearchCV
Successive Halving / Hyperband	Loại sớm cấu hình kém		HalvingRandomSearchCV

Phương pháp	Ưu	Nhược	Công cụ
Bayesian (TPE, GP)	Học từ lịch sử, ít lần thử	Tuần tự hơn	Optuna , Hyperopt, scikit-optimize
Population-based	Tốt cho deep learning	Tốn tài nguyên	Ray Tune

Random Search (baseline cho tuning)

```

from scipy.stats import loguniform, randint, uniform
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    candidates["lightgbm"],
    param_distributions={
        "clf__num_leaves": randint(15, 128),
        "clf__min_child_samples": randint(10, 200),
        "clf__learning_rate": loguniform(0.01, 0.1),
        "clf__colsample_bytree": uniform(0.5, 0.5),
        "clf__reg_lambda": loguniform(1e-3, 10),
    },
    n_iter=40, cv=cv, scoring="average_precision", n_jobs=-1, random_state=42, refit=True,
)
search.fit(X_train, y_train)
print(search.best_score_, search.best_params_)

```

Optuna — Bayesian Optimization với pruning (khuyến nghị)

```

import optuna
from sklearn.base import clone
from sklearn.metrics import average_precision_score

def objective(trial: optuna.Trial) -> float:
    params = {
        "n_estimators": 2000,
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),
        "num_leaves": trial.suggest_int("num_leaves", 15, 255, log=True),
        "max_depth": trial.suggest_int("max_depth", 3, 12),
        "min_child_samples": trial.suggest_int("min_child_samples", 10, 300, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "subsample_freq": 1,
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.4, 1.0),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True),
        "scale_pos_weight": trial.suggest_float("scale_pos_weight", 1.0, pos_weight),
        "verbose": -1, "random_state": 42,
    }
    scores = []
    for fold, (tr, va) in enumerate(cv.split(X_train, y_train)):
        prep = build_preprocessor(scale=False)
        Xtr_f = prep.fit_transform(X_train.iloc[tr], y_train.iloc[tr])
        Xva_f = prep.transform(X_train.iloc[va])
        m = lgb.LGBMClassifier(**params)
        m.fit(Xtr_f, y_train.iloc[tr], eval_set=[(Xva_f, y_train.iloc[va])],
              eval_metric="average_precision", callbacks=[lgb.early_stopping(100, verbose=False)])
        scores.append(average_precision_score(y_train.iloc[va], m.predict_proba(Xva_f)[: , 1]))
        trial.set_user_attr(f"best_iter_fold{fold}", m.best_iteration_)
    trial.report(float(np.mean(scores)), step=fold)

```

```

    if trial.should_prune():                # dừng sớm trial kém sau vài fold
        raise optuna.TrialPruned()
    return float(np.mean(scores))

study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=42, multivariate=True),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=10, n_warmup_steps=1),
    study_name="churn-lgbm",
    storage="sqlite:///optuna.db", load_if_exists=True,    # lưu lịch sử, tiếp tục được
)
study.optimize(objective, n_trials=100, timeout=3600, show_progress_bar=True)
print("Best PR-AUC:", study.best_value)
print("Best params:", study.best_params)

# Phân tích: tham số nào quan trọng?
optuna.visualization.plot_param_importances(study).show()
optuna.visualization.plot_optimization_history(study).show()

```

Kinh nghiệm: Tuning thường chỉ mang lại thêm 1-3% so với tham số mặc định hợp lý. Feature engineering và dữ liệu tốt mang lại nhiều hơn. Đừng dành 80% thời gian cho tuning.

8.7. Ensemble

Kỹ thuật	Mô tả	Khi nào hiệu quả
Bagging	Trung bình nhiều mô hình trên mẫu bootstrap (Random Forest)	Giảm variance
Boosting	Mô hình sau sửa lỗi mô hình trước (GBM)	Giảm bias
Averaging / Voting	Trung bình xác suất nhiều mô hình khác loại	Mô hình đa dạng, tương quan lỗi thấp
Rank averaging	Trung bình thứ hạng thay vì xác suất	Metric dựa trên thứ hạng (AUC)
Stacking	Meta-model học trên OOF prediction	Thi đấu; production cân nhắc độ phức tạp
Seed averaging	Cùng mô hình, nhiều seed	Ổn định dự đoán, rẻ

```

from sklearn.ensemble import StackingClassifier, VotingClassifier

voting = VotingClassifier(
    estimators=[("lr", candidates["logreg"]), ("lgbm", candidates["lightgbm"]),
                ("rf", candidates["random_forest"])],
    voting="soft", weights=[1, 2, 1], n_jobs=-1,
)

stacking = StackingClassifier(
    estimators=[("lr", candidates["logreg"]), ("lgbm", candidates["lightgbm"]),
                ("rf", candidates["random_forest"])],
    final_estimator=LogisticRegression(C=1.0, max_iter=2000),
    cv=StratifiedKFold(5, shuffle=True, random_state=0),    # meta-model học trên OOF
    stack_method="predict_proba", n_jobs=-1,
)

```

Production trade-off: Stacking 3 mô hình = 3× độ trễ, 3× bảo trì, khó giải thích. Chỉ dùng khi cải thiện có giá trị kinh doanh rõ ràng.

8.8. Chẩn đoán bằng Learning Curve & Validation Curve

```
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve, validation_curve

sizes, tr_scores, va_scores = learning_curve(
    candidates["lightgbm"], X_train, y_train, cv=cv, scoring="average_precision",
    train_sizes=np.linspace(0.1, 1.0, 8), n_jobs=-1, shuffle=True, random_state=0,
)
plt.plot(sizes, tr_scores.mean(1), "o-", label="train")
plt.plot(sizes, va_scores.mean(1), "o-", label="validation")
plt.fill_between(sizes, va_scores.mean(1) - va_scores.std(1),
                 va_scores.mean(1) + va_scores.std(1), alpha=0.2)
plt.xlabel("Số mẫu train"); plt.ylabel("PR-AUC"); plt.legend(); plt.title("Learning curve")
# Validation còn tăng khi thêm dữ liệu -> thu thập thêm dữ liệu sẽ có lợi
# Hai đường hội tụ ở mức thấp -> underfitting: cần feature/mô hình tốt hơn

param_range = [7, 15, 31, 63, 127, 255]
tr_s, va_s = validation_curve(candidates["lightgbm"], X_train, y_train,
                              param_name="clf__num_leaves", param_range=param_range,
                              cv=cv, scoring="average_precision", n_jobs=-1)
```

8.9. Deep Learning cho dữ liệu bảng (PyTorch)

Với dữ liệu bảng, GBM thường thắng (Grinsztajn et al., 2022). Deep learning đáng thử khi: dữ liệu rất lớn, kết hợp đa phương thức (bảng + văn bản + ảnh), cần embedding cho biến cardinality cực cao, hoặc học online.

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

class TabularMLP(nn.Module):
    def __init__(self, n_in: int, hidden=(256, 128), dropout: float = 0.2):
        super().__init__()
        layers, d = [], n_in
        for h in hidden:
            layers += [nn.Linear(d, h), nn.BatchNorm1d(h), nn.SiLU(), nn.Dropout(dropout)]
            d = h
        layers.append(nn.Linear(d, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x).squeeze(-1)  # logits

def train_mlp(Xtr, ytr, Xva, yva, epochs=100, patience=10, lr=1e-3, batch_size=512, seed=42):
    torch.manual_seed(seed)
    device = "cuda" if torch.cuda.is_available() else "cpu"
    to_t = lambda a: torch.as_tensor(np.asarray(a, dtype=np.float32))
```

```

train_dl = DataLoader(TensorDataset(to_t(Xtr), to_t(ytr)), batch_size=batch_size, shuffle=True)
Xva_t = to_t(Xva).to(device)

model = TabularMLP(Xtr.shape[1]).to(device)
pos_w = torch.tensor([(ytr == 0).sum() / max((ytr == 1).sum(), 1)], device=device)
loss_fn = nn.BCEWithLogitsLoss(pos_weight=pos_w)
opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, mode="max", factor=0.5, patience=3)

best, best_state, wait = -np.inf, None, 0
for epoch in range(epochs):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
    model.eval()
    with torch.no_grad():
        p = torch.sigmoid(model(Xva_t)).cpu().numpy()
        score = average_precision_score(yva, p)
        sched.step(score)
    if score > best + 1e-4:
        best, wait = score, 0
        best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
    else:
        wait += 1
        if wait >= patience:
            print(f"Early stop at epoch {epoch}, best PR-AUC={best:.4f}")
            break
    model.load_state_dict(best_state)
return model

# Đầ'u vào pha'i đư'c scale (build_preprocessor(scale=True))
prep_nn = build_preprocessor(scale=True)
Xtr_nn = prep_nn.fit_transform(X_train, y_train)
Xva_nn = prep_nn.transform(X_valid)
mlp = train_mlp(Xtr_nn, y_train.to_numpy(), Xva_nn, y_valid.to_numpy())

```

Các kiến trúc chuyên cho tabular: **TabNet**, **FT-Transformer**, **TabPFN** (rất mạnh với dữ liệu nhỏ < 10k dòng), **SAINT**.

8.10. Script huấn luyện production (kết nối MLflow)

```

# src/churn/models/train.py
import argparse
import json
import logging
from pathlib import Path

import joblib
import lightgbm as lgb
import mlflow
import mlflow.sklearn
import pandas as pd
from mlflow.models import infer_signature

```

```

from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.pipeline import Pipeline

from churn.config import Secrets, load_config
from churn.data.validate import validate
from churn.features.build import build_preprocessor
from churn.features.clean import clean_churn
from churn.data.split import time_split # hàm ở chương 7

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("train")

def main(config_path: str) -> None:
    cfg = load_config(config_path)
    mlflow.set_tracking_uri(Secrets().mlflow_tracking_uri)
    mlflow.set_experiment("churn-prediction")

    df = clean_churn(pd.read_parquet("data/raw/churn.parquet"))
    validate(df)
    train_df, valid_df, test_df = time_split(df, "signup_date", "2023-12-01", "2024-03-01")
    X_tr, y_tr = train_df.drop(columns=cfg.target), train_df[cfg.target]
    X_va, y_va = valid_df.drop(columns=cfg.target), valid_df[cfg.target]

    with mlflow.start_run(run_name="lgbm") as run:
        mlflow.log_params(cfg.model_params)
        mlflow.log_params({"n_train": len(X_tr), "n_valid": len(X_va),
                           "train_churn_rate": round(y_tr.mean(), 4)})

        pipe = Pipeline([
            ("prep", build_preprocessor(scale=False)),
            ("clf", lgb.LGBMClassifier(**cfg.model_params, random_state=cfg.random_state,
verbose=-1)),
        ])
        pipe.fit(X_tr, y_tr)

        proba = pipe.predict_proba(X_va)[ :, 1]
        metrics = {"valid_roc_auc": roc_auc_score(y_va, proba),
                   "valid_pr_auc": average_precision_score(y_va, proba)}
        mlflow.log_metrics(metrics)
        log.info("Metrics: %s", metrics)

        Path("models").mkdir(exist_ok=True)
        joblib.dump(pipe, "models/model.joblib")
        Path("reports").mkdir(exist_ok=True)
        Path("reports/metrics.json").write_text(json.dumps(metrics, indent=2))

        mlflow.sklearn.log_model(
            pipe, "model",
            signature=infer_signature(X_va.head(100), proba[:100]),
            input_example=X_va.head(5),
            registered_model_name="churn-classifier",
        )
        log.info("Run ID: %s", run.info.run_id)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--config", default="configs/train.yaml")
    main(parser.parse_args().config)

```

```

# configs/train.yaml
target: churn

```

```
random_state: 42
model_params:
  n_estimators: 800
  learning_rate: 0.03
  num_leaves: 31
  min_child_samples: 50
  subsample: 0.8
  subsample_freq: 1
  colsample_bytree: 0.8
  reg_lambda: 1.0
```

Checklist Chương 8

- ☐ Có baseline ngẫu nhiên + baseline nghiệp vụ.
- ☐ Mọi mô hình so sánh trên cùng CV, cùng metric, báo cáo mean $\pm$ std.
- ☐ Early stopping dùng tập validation riêng, không phải test.
- ☐ Tuning có giới hạn ngân sách, lưu lịch sử (Optuna storage), seed cố định.
- ☐ Đã xem learning curve để quyết định: thêm dữ liệu hay cải thiện mô hình.
- ☐ Toàn bộ pipeline (tiền xử lý + mô hình) được lưu thành 1 artifact và log vào MLflow.

CHƯƠNG 9. ĐÁNH GIÁ MÔ HÌNH: METRICS, CONFUSION MATRIX, THRESHOLD & CALIBRATION

9.1. Nguyên tắc chọn metric

- 1. **Metric phải phản ánh chi phí kinh doanh** (Chương 1.3), không phải metric "phổ biến".
- 2. Tách biệt: **metric tối ưu (optimizing)** — một con số để chọn mô hình; và **metric ràng buộc (satisficing)** — phải đạt ngưỡng (ví dụ: độ trễ < 50ms, recall ≥ 0.7, chênh lệch fairness < 5%).
- 3. Báo cáo **khoảng tin cậy**, không chỉ ước lượng điểm.
- 4. Đánh giá **theo phân khúc (slice)**: mô hình tốt trung bình có thể rất tệ với một nhóm khách hàng.

Bảng tra nhanh

Bài toán	Metric chính	Metric bổ sung
Phân loại cân bằng	Accuracy, ROC-AUC, F1	Log loss
Phân loại mất cân bằng	PR-AUC , Recall@Precision, F-β	MCC, Balanced accuracy
Cần xác suất chính xác (định giá rủi ro)	Log loss, Brier score	ECE (calibration)
Xếp hạng top-K (marketing, CSKH)	Precision@K, Recall@K, Lift@K	Gain chart
Tín dụng	Gini (= 2·AUC – 1), KS	PSI (ổn định)
Hồi quy	MAE, RMSE	R², MAPE/sMAPE, pinball loss
Truy hồi thông tin / gợi ý	NDCG@K, MAP@K, MRR, Recall@K	F2 (ưu tiên recall)
Phân cụm	Silhouette, Davies–Bouldin	ARI/NMI (khi có nhãn)

9.2. Confusion Matrix — nền tảng của mọi metric phân loại

		DỰ ĐOÁN		
		Positive (1)	Negative (0)	
THỰC TẾ	P(1)	TP (True Positive) Đúng: churn	FN (False Negative) Bỏ sót khách churn	← Sai lầm loại II (bỏ sót)
	N(0)	FP (False Positive) Báo nhầm (tồn tiền)	TN (True Negative) Đúng: không churn	← FP = Sai lầm loại I

Lưu ý quy ước: `sklearn.metrics.confusion_matrix` trả về ma trận với **hàng = thực tế, cột = dự đoán**, theo thứ tự nhãn tăng dần `[0, 1]`, tức là `[[TN, FP], [FN, TP]]` — ngược thứ tự so với hình trên.

Các chỉ số dẫn xuất

Chỉ số	Công thức	Ý nghĩa	Tên khác
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Tỷ lệ dự đoán đúng	— vô dụng khi mất cân bằng
Precision	$\frac{TP}{TP + FP}$	Trong số dự đoán positive, bao nhiêu đúng?	PPV
Recall	$\frac{TP}{TP + FN}$	Trong số positive thật, bắt được bao nhiêu?	Sensitivity, TPR, Hit rate
Specificity	$\frac{TN}{TN + FP}$	Trong số negative thật, nhận đúng bao nhiêu?	TNR, Selectivity
FPR	$\frac{FP}{FP + TN} = 1 - \text{Specificity}$	Tỷ lệ báo động nhầm	Fall-out
FNR	$\frac{FN}{FN + TP} = 1 - \text{Recall}$	Tỷ lệ bỏ sót	Miss rate
NPV	$\frac{TN}{TN + FN}$	Dự đoán negative đáng tin đến đâu?	
F1	$2 \cdot \frac{P \cdot R}{P + R}$	Trung bình điều hòa P và R	
F-β	$(1 + \beta^2) \frac{P \cdot R}{\beta^2 P + R}$	$\beta > 1$ ưu tiên Recall (F2), $\beta < 1$ ưu tiên Precision (F0.5)	
Balanced Accuracy	$\frac{TPR + TNR}{2}$	Accuracy công bằng giữa các lớp	
MCC	$\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$	Tương quan dự đoán-thực tế, $\in [-1, 1]$; bền nhất khi mất cân bằng	Phi coefficient
Cohen's κ	$\frac{p_o - p_e}{1 - p_e}$	Đồng thuận vượt mức ngẫu nhiên	
Youden's J	$TPR - FPR$	Dùng chọn ngưỡng trên ROC	Informedness

Ví dụ tính tay

Tập kiểm tra 1000 khách, 200 churn. Mô hình dự đoán 250 churn, trong đó đúng 150.

	Dự đoán 1	Dự đoán 0	Tổng
Thực tế 1	TP = 150	FN = 50	200
Thực tế 0	FP = 100	TN = 700	800

- Accuracy = $(150 + 700)/1000 = \mathbf{0.85}$
- Precision = $150/250 = \mathbf{0.60}$; Recall = $150/200 = \mathbf{0.75}$; Specificity = $700/800 = \mathbf{0.875}$

- $F1 = 2 \cdot 0.6 \cdot 0.75 / (1.35) = \mathbf{0.667}$; $F2 = 5 \cdot 0.6 \cdot 0.75 / (4 \cdot 0.6 + 0.75) = \mathbf{0.714}$
- $MCC = (150 \cdot 700 - 100 \cdot 50) / \sqrt{(250 \cdot 200 \cdot 800 \cdot 750)} = 100000 / 173205 = \mathbf{0.577}$
- Mô hình "luôn dự đoán 0" có Accuracy = **0.80** nhưng Recall = 0, MCC = 0 → minh chứng Accuracy gây hiểu lầm.

Code: tính và trực quan hóa confusion matrix

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import (ConfusionMatrixDisplay, balanced_accuracy_score,
                             classification_report,
                             cohen_kappa_score, confusion_matrix, f1_score, fbeta_score,
                             matthews_corrcoef, precision_score, recall_score)

# y_valid, proba_valid: nhãn thật và xác suất dự đoán cu'a mô hình (chương 8)
proba_valid = pipe.predict_proba(X_valid)[: , 1]
threshold = 0.5
y_pred = (proba_valid >= threshold).astype(int)

tn, fp, fn, tp = confusion_matrix(y_valid, y_pred, labels=[0, 1]).ravel()

def binary_report(y_true, y_pred) -> pd.Series:
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    return pd.Series({
        "TP": tp, "FP": fp, "FN": fn, "TN": tn,
        "accuracy": (tp + tn) / (tp + tn + fp + fn),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred),
        "specificity": tn / (tn + fp) if (tn + fp) else 0.0,
        "npv": tn / (tn + fn) if (tn + fn) else 0.0,
        "f1": f1_score(y_true, y_pred),
        "f2": fbeta_score(y_true, y_pred, beta=2),
        "balanced_acc": balanced_accuracy_score(y_true, y_pred),
        "mcc": matthews_corrcoef(y_true, y_pred),
        "cohen_kappa": cohen_kappa_score(y_true, y_pred),
    })

print(binary_report(y_valid, y_pred).round(4))
print(classification_report(y_valid, y_pred, target_names=["stay", "churn"], digits=4))

# Vẽ 2 phiên bản: số tuyệt đối & chuẩn hóa theo hàng (= recall từng lớp)
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
ConfusionMatrixDisplay.from_predictions(y_valid, y_pred, display_labels=["stay", "churn"],
                                       cmap="Blues", values_format="d", ax=axes[0])
axes[0].set_title("Confusion matrix (counts)")
ConfusionMatrixDisplay.from_predictions(y_valid, y_pred, display_labels=["stay", "churn"],
                                       normalize="true", cmap="Blues", values_format=".2%",
                                       ax=axes[1])
axes[1].set_title("Normalized by true class (recall)")
plt.tight_layout()
```

`normalize` có 3 chế độ: "true" (chia theo hàng → recall từng lớp), "pred" (chia theo cột → precision từng lớp), "all" (chia tổng).

Confusion matrix đa lớp & cách lấy trung bình

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import multilabel_confusion_matrix
from sklearn.model_selection import train_test_split

Xi, yi = load_iris(return_X_y=True)
Xa, Xb, ya, yb = train_test_split(Xi, yi, test_size=0.4, stratify=yi, random_state=0)
yb_pred = LogisticRegression(max_iter=1000).fit(Xa, ya).predict(Xb)

print(confusion_matrix(yb, yb_pred))
print(multilabel_confusion_matrix(yb, yb_pred))  # 1 ma trận 2x2 (one-vs-rest) cho mỗi lớp

for avg in ["macro", "weighted", "micro"]:
    print(avg, round(f1_score(yb, yb_pred, average=avg), 4))
```

Kiểu trung bình	Cách tính	Khi nào dùng
macro	Trung bình đơn giản metric từng lớp	Mọi lớp quan trọng như nhau (kể cả lớp hiếm)
weighted	Trung bình có trọng số theo số mẫu (support)	Phản ánh phân phối dữ liệu
micro	Gộp TP/FP/FN toàn cục rồi tính	Đa nhãn; với đa lớp đơn nhãn, micro-F1 = accuracy

Đọc confusion matrix để phân tích lỗi

- **Hàng có nhiều giá trị ngoài đường chéo** → lớp đó khó nhận diện (recall thấp).
- **Cột có nhiều giá trị ngoài đường chéo** → mô hình hay "đổ" về lớp đó (precision thấp).
- **Cặp ô đối xứng lớn ($A \leftrightarrow B$)** → hai lớp bị nhầm lẫn lẫn nhau → cần feature phân biệt, hoặc gộp lớp nếu nghiệp vụ cho phép.

9.3. Metric không phụ thuộc ngưỡng

ROC curve & ROC-AUC

ROC vẽ TPR theo FPR khi quét mọi ngưỡng. **AUC = xác suất mô hình xếp một mẫu positive ngẫu nhiên cao hơn một mẫu negative ngẫu nhiên.**

Precision-Recall curve & PR-AUC (Average Precision)

Khi positive hiếm ($< 10\%$), ROC-AUC có thể cao "giả" vì FPR bị pha loãng bởi số lượng TN khổng lồ. **PR-AUC tập trung vào lớp positive** — baseline của PR-AUC là tỷ lệ positive (không phải 0.5).

```
from sklearn.metrics import (PrecisionRecallDisplay, RocCurveDisplay, average_precision_score,
                             brier_score_loss, log_loss, roc_auc_score)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
RocCurveDisplay.from_predictions(y_valid, proba_valid, name="LightGBM", ax=axes[0],
                                plot_chance_level=True)
PrecisionRecallDisplay.from_predictions(y_valid, proba_valid, name="LightGBM", ax=axes[1],
                                       plot_chance_level=True)

plt.tight_layout()

auc = roc_auc_score(y_valid, proba_valid)
print({
    "roc_auc": auc,
    "gini": 2 * auc - 1,
    "pr_auc": average_precision_score(y_valid, proba_valid),
    "log_loss": log_loss(y_valid, proba_valid),
    "brier": brier_score_loss(y_valid, proba_valid),
})
```

Metric	Đo cái gì	Ghi chú
ROC-AUC	Khả năng xếp hạng	Không nhạy với mất cân bằng — vừa là ưu vừa là nhược
PR-AUC	Xếp hạng tập trung vào positive	Ưu tiên khi lớp hiếm
Log loss	Chất lượng xác suất	Phạt rất nặng dự đoán sai mà tự tin
Brier score	MSE của xác suất	Dễ diễn giải hơn log loss, $\in [0, 1]$

KS statistic (tín dụng)

```
from scipy.stats import ks_2samp

ks = ks_2samp(proba_valid[y_valid == 1], proba_valid[y_valid == 0]).statistic
print("KS =", round(ks, 4)) # Khoa'ng cách lớn nhất giữa 2 CDF; > 0.4 thường được coi là tốt
```

9.4. Chọn ngưỡng quyết định (Threshold Tuning)

Ngưỡng 0.5 chỉ tối ưu khi chi phí FP = FN **và** xác suất được calibrate tốt. Trong thực tế gần như không bao giờ như vậy.

Quan trọng: chọn ngưỡng trên **validation hoặc OOF predictions**, rồi cố định và đánh giá trên test.

```
from sklearn.metrics import precision_recall_curve, roc_curve

def threshold_table(y_true, proba, thresholds=np.linspace(0.05, 0.95, 19)) -> pd.DataFrame:
    rows = [binary_report(y_true, (proba >= t).astype(int)).rename(round(t, 2)) for t in
thresholds]
    return pd.DataFrame(rows)[["precision", "recall", "f1", "f2", "mcc", "TP", "FP", "FN"]]

print(threshold_table(y_valid, proba_valid).round(3))

# (1) Ngưỡng tối đa F1 / F-beta
prec, rec, thr = precision_recall_curve(y_valid, proba_valid)
f1 = 2 * prec[:-1] * rec[:-1] / np.clip(prec[:-1] + rec[:-1], 1e-12, None) # len(thr) =
len(prec)-1
t_f1 = thr[np.argmax(f1)]
```

```
# (2) Youden's J trên ROC
fpr, tpr, thr_roc = roc_curve(y_valid, proba_valid)
t_youden = thr_roc[np.argmax(tpr - fpr)]

# (3) Ràng buộc nghiệp vụ: Precision tối thiểu 0.5, tối đa hóa Recall
ok = prec[:-1] >= 0.5
t_prec = thr[ok][np.argmax(rec[:-1][ok])] if ok.any() else None

# (4) Năng lực vận hành: CSKH chỉ gọi được top 10% khách
t_topk = np.quantile(proba_valid, 0.90)

# (5) Tối đa hóa lợi nhuận kỳ vọng (ma trận chi phí chương 1)
def expected_profit(y_true, proba, t, gain_tp=450_000, cost_fp=-50_000, cost_fn=-500_000):
    pred = proba >= t
    tp = np.sum(pred & (y_true == 1)); fp = np.sum(pred & (y_true == 0))
    fn = np.sum(~pred & (y_true == 1))
    return tp * gain_tp + fp * cost_fp + fn * cost_fn

grid = np.linspace(0.01, 0.99, 99)
profits = [expected_profit(y_valid.to_numpy(), proba_valid, t) for t in grid]
t_profit = grid[int(np.argmax(profits))]
print(dict(f1=t_f1, youden=t_youden, precision_constraint=t_prec, top10=t_topk, profit=t_profit))
```

Khi xác suất đã được calibrate, ngưỡng tối ưu lý thuyết theo chi phí là:

$$t^* = \frac{C_{FP}}{C_{FP} + C_{FN}} \quad (\text{chi phí tính theo giá trị dương, với } C_{TP} = C_{TN} = 0)$$

sklearn ≥ 1.5: TunedThresholdClassifierCV

```
from sklearn.metrics import make_scorer
from sklearn.model_selection import FixedThresholdClassifier, TunedThresholdClassifierCV

tuned = TunedThresholdClassifierCV(
    pipe, scoring=make_scorer(fbeta_score, beta=2), cv=5, thresholds=100,
    store_cv_results=True, random_state=0,
).fit(X_train, y_train)
print("Best threshold:", tuned.best_threshold_, "F2:", tuned.best_score_)

# Đóng gói ngưỡng cố định vào mô hình để deploy nhát quán
deployable = FixedThresholdClassifier(pipe, threshold=float(t_profit),
response_method="predict_proba")
```

9.5. Metric xếp hạng / kinh doanh: Lift & Gain

```
def lift_table(y_true, proba, n_bins: int = 10) -> pd.DataFrame:
    d = pd.DataFrame({"y": np.asarray(y_true), "p": proba})
    d["decile"] = pd.qcut(d["p"].rank(method="first", ascending=False), n_bins,
                          labels=range(1, n_bins + 1))
    t = d.groupby("decile", observed=True).agg(n=("y", "size"), positives=("y", "sum"),
                                              min_score=("p", "min"))

    t["rate"] = t["positives"] / t["n"]
    t["lift"] = t["rate"] / d["y"].mean()
    t["cum_capture"] = t["positives"].cumsum() / d["y"].sum() # Gain
    return t.round(4)
```

```
print(lift_table(y_valid, proba_valid))

def precision_at_k(y_true, proba, k: float = 0.1) -> float:
    n = int(len(proba) * k)
    idx = np.argsort(-proba)[:n]
    return float(np.asarray(y_true)[idx].mean())
```

Diễn giải cho stakeholder: "Nếu gọi 10% khách có điểm cao nhất, ta tiếp cận được 35% số khách sẽ rời bỏ, hiệu quả gấp 3.5 lần gọi ngẫu nhiên."

9.6. Calibration — xác suất có đáng tin không?

Mô hình **calibrated**: trong các khách được dự đoán 30% churn, thực tế ~30% churn. Quan trọng khi xác suất được dùng trực tiếp (tính lợi nhuận kỳ vọng, định giá, kết hợp nhiều mô hình).

Nguyên nhân mất calibration: class weight/SMOTE, mô hình boosting/SVM/Naive Bayes, overfitting.

```
from sklearn.calibration import CalibratedClassifierCV, CalibrationDisplay, calibration_curve

def expected_calibration_error(y_true, proba, n_bins: int = 10) -> float:
    bins = np.linspace(0, 1, n_bins + 1)
    idx = np.clip(np.digitize(proba, bins) - 1, 0, n_bins - 1)
    ece = 0.0
    for b in range(n_bins):
        m = idx == b
        if m.any():
            ece += m.mean() * abs(np.asarray(y_true)[m].mean() - proba[m].mean())
    return ece

# Calibrate: method="sigmoid" (Platt – ít dữ liệu) hoặc "isotonic" (nhiều dữ liệu, > ~1000 positive)
calibrated = CalibratedClassifierCV(pipe, method="isotonic", cv=5).fit(X_train, y_train)
proba_cal = calibrated.predict_proba(X_valid)[: , 1]

fig, ax = plt.subplots(figsize=(6, 6))
CalibrationDisplay.from_predictions(y_valid, proba_valid, n_bins=10, strategy="quantile",
                                   name="Uncalibrated", ax=ax)
CalibrationDisplay.from_predictions(y_valid, proba_cal, n_bins=10, strategy="quantile",
                                   name="Isotonic", ax=ax)
print("ECE before:", round(expected_calibration_error(y_valid, proba_valid), 4),
      "| after:", round(expected_calibration_error(y_valid, proba_cal), 4))
print("Brier before/after:", brier_score_loss(y_valid, proba_valid), brier_score_loss(y_valid,
proba_cal))
```

Calibration không thay đổi thứ hạng nhiều (AUC gần như giữ nguyên) nhưng cải thiện log loss/Brier và làm ngưỡng có ý nghĩa.

9.7. Metric hồi quy

Metric	Công thức	Đặc điểm
MAE	$\frac{1}{n} \sum y - \hat{y} $	Cùng đơn vị với y; bền với ngoại lai; tối ưu bởi median
MSE / RMSE	$\sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$	Phạt nặng sai số lớn; tối ưu bởi mean
R ²	$1 - \frac{SS_{res}}{SS_{tot}}$	Tỷ lệ phương sai được giải thích; có thể âm
Adjusted R ²	$1 - (1 - R^2) \frac{n-1}{n-p-1}$	Phạt số lượng feature
MAPE	$\frac{100}{n} \sum \left \frac{y - \hat{y}}{y} \right $	Dễ hiểu (%); vỡ khi y ≈ 0 , bất đối xứng
sMAPE	$\frac{100}{n} \sum \frac{ y - \hat{y} }{ y + \hat{y} }$	Đối xứng hơn MAPE
RMSLE	RMSE trên $\log(1 + y)$	Sai số tương đối; target lệch phải
MASE	MAE / MAE của naive forecast	Chuỗi thời gian; < 1 là tốt hơn naive
Pinball (quantile) loss	$\max(q(y - \hat{y}), (q - 1)(y - \hat{y}))$	Đánh giá dự báo phân vị / khoảng
WAPE	$\sum y - \hat{y} / \sum y $	Dự báo nhu cầu bán lẻ

```

from sklearn.metrics import (mean_absolute_error, mean_absolute_percentage_error,
                             mean_pinball_loss, mean_squared_log_error, r2_score,
                             root_mean_squared_error)

def regression_report(y_true, y_pred, n_features: int | None = None) -> dict:
    y_true, y_pred = np.asarray(y_true, float), np.asarray(y_pred, float)
    r2 = r2_score(y_true, y_pred)
    out = {
        "MAE": mean_absolute_error(y_true, y_pred),
        "RMSE": root_mean_squared_error(y_true, y_pred),      # sklearn >= 1.4
        "R2": r2,
        "MAPE_%": 100 * mean_absolute_percentage_error(y_true, y_pred),
        "sMAPE_%": 100 * np.mean(2 * np.abs(y_true - y_pred) /
                                   np.clip(np.abs(y_true) + np.abs(y_pred), 1e-12, None)),
        "WAPE_%": 100 * np.abs(y_true - y_pred).sum() / np.abs(y_true).sum(),
        "MedAE": float(np.median(np.abs(y_true - y_pred))),
    }
    if (y_true >= 0).all() and (y_pred >= 0).all():
        out["RMSLE"] = np.sqrt(mean_squared_log_error(y_true, y_pred))
    if n_features:
        n = len(y_true)
        out["adj_R2"] = 1 - (1 - r2) * (n - 1) / (n - n_features - 1)
    return out

def mase(y_true, y_pred, y_train, m: int = 1) -> float:
    """m = chu kỳ mùa vụ (1 = naive, 7 = seasonal naive theo tuầ'n)."""
    y_train = np.asarray(y_train)
    scale = np.mean(np.abs(y_train[m:] - y_train[:-m]))
    return float(np.mean(np.abs(np.asarray(y_true) - np.asarray(y_pred))) / scale)

```

Phân tích phần dư (residual analysis)

```
def residual_plots(y_true, y_pred):
    res = np.asarray(y_true) - np.asarray(y_pred)
    fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
    axes[0].scatter(y_pred, res, s=5, alpha=0.4); axes[0].axhline(0, color="r")
    axes[0].set(xlabel="Predicted", ylabel="Residual", title="Residual vs Predicted") # hình
    #phê~u => heteroscedasticity
    axes[1].scatter(y_true, y_pred, s=5, alpha=0.4)
    lim = [min(np.min(y_true), np.min(y_pred)), max(np.max(y_true), np.max(y_pred))]
    axes[1].plot(lim, lim, "r--"); axes[1].set(xlabel="Actual", ylabel="Predicted", title="Actual
    vs Predicted")
    from scipy import stats
    stats.probplot(res, dist="norm", plot=axes[2]); axes[2].set_title("Q-Q plot residual")
    plt.tight_layout()
```

9.8. Metric phân cụm

```
from sklearn.cluster import KMeans
from sklearn.metrics import (adjusted_rand_score, calinski_harabasz_score, davies_bouldin_score,
                             normalized_mutual_info_score, silhouette_score)
from sklearn.preprocessing import StandardScaler

Xc = StandardScaler().fit_transform(
    X_train[["tenure_months", "monthly_charges", "support_calls"]].fillna(0))
for k in range(2, 9):
    labels = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(Xc)
    print(k,
          "silhouette", round(silhouette_score(Xc, labels, sample_size=5000, random_state=0), 3),
          "DB", round(davies_bouldin_score(Xc, labels), 3),
          "CH", round(calinski_harabasz_score(Xc, labels), 1))
# Có nhãn thật: adjusted_rand_score(y, labels), normalized_mutual_info_score(y, labels)
```

9.9. Metric truy hồi & xếp hạng (Search, RAG, Recommender)

```
from sklearn.metrics import ndcg_score

def recall_at_k(relevant: set, ranked: list, k: int) -> float:
    return len(relevant & set(ranked[:k])) / len(relevant) if relevant else 0.0

def precision_at_k_list(relevant: set, ranked: list, k: int) -> float:
    return len(relevant & set(ranked[:k])) / k

def average_precision(relevant: set, ranked: list, k: int | None = None) -> float:
    ranked = ranked[:k] if k else ranked
    hits, score = 0, 0.0
    for i, doc in enumerate(ranked, start=1):
        if doc in relevant:
            hits += 1
            score += hits / i
    return score / min(len(relevant), len(ranked)) if relevant else 0.0
```

```
def reciprocal_rank(relevant: set, ranked: list) -> float:
    return next((1 / i for i, d in enumerate(ranked, 1) if d in relevant), 0.0)

def f_beta_sets(relevant: set, retrieved: set, beta: float = 2.0) -> float:
    """F2: ưu tiên recall gấp beta^2 = 4 lần precision – phù hợp truy hỏi văn bản pháp luật."""
    tp = len(relevant & retrieved)
    if tp == 0:
        return 0.0
    p, r = tp / len(retrieved), tp / len(relevant)
    return (1 + beta**2) * p * r / (beta**2 * p + r)

# NDCG với độ liên quan phân cấp (graded relevance)
true_rel = np.array([[3, 2, 0, 0, 1]]) # độ liên quan thật của 5 tài liệu
scores = np.array([[0.9, 0.8, 0.7, 0.1, 0.6]]) # điểm mô hình
print("NDCG@3 =", ndcg_score(true_rel, scores, k=3))
# MAP@K, MRR = trung bình average_precision / reciprocal_rank qua tất cả truy vấn
```

9.10. So sánh mô hình có ý nghĩa thống kê

Mô hình B có PR-AUC 0.612 so với A là 0.605 — B có thực sự tốt hơn?

```
from statsmodels.stats.contingency_tables import mcnemar

def paired_bootstrap(y_true, p_a, p_b, metric=average_precision_score, n_boot=2000, seed=0):
    """CI cho chênh lệch metric(B) - metric(A) trên CÙNG tập test (ghép cặp)."""
    rng = np.random.default_rng(seed)
    y_true = np.asarray(y_true)
    n = len(y_true)
    diffs = []
    for _ in range(n_boot):
        idx = rng.integers(0, n, n)
        if y_true[idx].min() == y_true[idx].max():
            continue
        diffs.append(metric(y_true[idx], p_b[idx]) - metric(y_true[idx], p_a[idx]))
    diffs = np.array(diffs)
    return {"mean_diff": diffs.mean(), "ci95": np.percentile(diffs, [2.5, 97.5]),
            "p_b_better": (diffs > 0).mean()}

def mcnemar_test(y_true, pred_a, pred_b):
    """So sánh 2 bộ phân loại (nhãn cứng) trên cùng tập test."""
    a_ok, b_ok = pred_a == y_true, pred_b == y_true
    table = [[np.sum(a_ok & b_ok), np.sum(a_ok & ~b_ok)],
              [np.sum(~a_ok & b_ok), np.sum(~a_ok & ~b_ok)]]
    return mcnemar(table, exact=False, correction=True).pvalue

def corrected_resampled_ttest(scores_a, scores_b, n_train: int, n_test: int):
    """Nadeau & Bengio (2003): t-test hiệu chỉnh cho điểm CV (các fold không độc lập)."""
    from scipy import stats
    d = np.asarray(scores_b) - np.asarray(scores_a)
    k = len(d)
    var = d.var(ddof=1) * (1 / k + n_test / n_train)
    t = d.mean() / np.sqrt(var)
    return t, 2 * stats.t.sf(abs(t), df=k - 1)
```

Nếu khoảng tin cậy của chênh lệch chứa 0 → **chưa đủ bằng chứng** B tốt hơn → giữ mô hình đơn giản hơn.

9.11. Phân tích lỗi & đánh giá theo phân khúc (Slice analysis)

```
def slice_report(X: pd.DataFrame, y_true, proba, slice_col: str, threshold: float) -> pd.DataFrame:
    d = X[[slice_col]].copy()
    d["y"], d["p"] = np.asarray(y_true), proba
    d["pred"] = (d["p"] >= threshold).astype(int)
    rows = []
    for key, g in d.groupby(slice_col, observed=True):
        if g["y"].nunique() < 2:
            continue
        rows.append({slice_col: key, "n": len(g), "pos_rate": g["y"].mean(),
                    "roc_auc": roc_auc_score(g["y"], g["p"]),
                    "pr_auc": average_precision_score(g["y"], g["p"]),
                    "recall": recall_score(g["y"], g["pred"]),
                    "precision": precision_score(g["y"], g["pred"], zero_division=0)})
    return pd.DataFrame(rows).sort_values("roc_auc")

print(slice_report(X_valid, y_valid, proba_valid, "contract", t_profit).round(3))

# Xem các lỗi tự tin nhất (FN có điểm thấp nhất, FP có điểm cao nhất) để tìm pattern
errors = X_valid.assign(y=y_valid.values, p=proba_valid)
worst_fn = errors[(errors.y == 1)].nsmallest(20, "p")
worst_fp = errors[(errors.y == 0)].nlargest(20, "p")
```

9.12. Báo cáo đánh giá cuối cùng trên tập test

```
def final_evaluation(model, X_test, y_test, threshold: float, n_boot: int = 1000) -> pd.DataFrame:
    proba = model.predict_proba(X_test)[ :, 1]
    pred = (proba >= threshold).astype(int)
    rng = np.random.default_rng(0)
    y = np.asarray(y_test)
    metrics = {"roc_auc": lambda yy, pp, dd: roc_auc_score(yy, pp),
              "pr_auc": lambda yy, pp, dd: average_precision_score(yy, pp),
              "recall": lambda yy, pp, dd: recall_score(yy, dd),
              "precision": lambda yy, pp, dd: precision_score(yy, dd, zero_division=0),
              "f2": lambda yy, pp, dd: fbeta_score(yy, dd, beta=2),
              "brier": lambda yy, pp, dd: brier_score_loss(yy, pp)}

    rows = []
    for name, fn in metrics.items():
        point = fn(y, proba, pred)
        boots = []
        for _ in range(n_boot):
            i = rng.integers(0, len(y), len(y))
            if y[i].min() != y[i].max():
                boots.append(fn(y[i], proba[i], pred[i]))
        lo, hi = np.percentile(boots, [2.5, 97.5])
        rows.append({"metric": name, "value": point, "ci95_low": lo, "ci95_high": hi})
    return pd.DataFrame(rows).set_index("metric").round(4)
```

Checklist Chương 9

- ☐ Metric chính gắn với mục tiêu kinh doanh; có metric ràng buộc.
- ☐ Không dùng Accuracy làm metric chính khi dữ liệu mất cân bằng.
- ☐ Confusion matrix được phân tích ở ngưỡng thực sự sẽ deploy.
- ☐ Ngưỡng chọn trên validation/OOF theo chi phí hoặc năng lực vận hành.
- ☐ Đã kiểm tra calibration nếu xác suất được dùng trực tiếp.
- ☐ Kết quả test có khoảng tin cậy; so sánh mô hình bằng kiểm định ghép cặp.
- ☐ Có slice analysis theo các phân khúc quan trọng.

CHƯƠNG 10. GIẢI THÍCH MÔ HÌNH (XAI) & FAIRNESS

10.1. Vì sao cần giải thích?

- **Tin tưởng & chấp nhận:** CSKH cần biết *vì sao* khách bị đánh dấu nguy cơ cao để tư vấn đúng.
- **Debug:** phát hiện leakage, feature vô lý (mô hình dựa vào `customer_id`?).
- **Tuân thủ pháp lý:** quyết định tín dụng, tuyển dụng, bảo hiểm đòi hỏi giải thích được (EU AI Act, quy định ngân hàng).
- **Insight kinh doanh:** yếu tố nào thúc đẩy churn → hành động phòng ngừa.

Phạm vi	Câu hỏi	Kỹ thuật
Toàn cục (global)	Feature nào quan trọng nhất với mô hình?	Permutation importance, mean SHAP , PDP
Cục bộ (local)	Vì sao khách hàng X bị dự đoán churn 82%?	SHAP waterfall, LIME
Hình dạng quan hệ	Churn thay đổi thế nào khi tenure tăng?	PDP, ICE, ALE, SHAP dependence
Phản thực tế (counterfactual)	Cần thay đổi gì để khách không churn?	DiCE

10.2. Feature importance — các loại và cạm bẫy

Loại	Ưu	Nhược
Impurity/split-based (<code>feature_importances_</code>)	Có sẵn, nhanh	Thiên vị biến liên tục và cardinality cao; tính trên train
Gain-based (LightGBM <code>importance_type="gain"</code>)	Tốt hơn split count	Vẫn tính trên train
Permutation importance	Model-agnostic, tính trên validation	Sai lệch khi các feature tương quan mạnh
SHAP	Có nền tảng lý thuyết (Shapley), cả global & local	Tốn tính toán (trừ TreeSHAP); giải thích <i>mô hình</i> , không phải <i>nhân quả</i>

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.inspection import permutation_importance

# pipe: Pipeline(preprocessing, LGBMClassifier) đã fit; X_valid, y_valid từ các chương trước
result = permutation_importance(
    pipe, X_valid, y_valid, scoring="average_precision",
    n_repeats=10, random_state=0, n_jobs=-1,
```

```
)
perm = (pd.DataFrame({"mean": result.importances_mean, "std": result.importances_std},
                    index=X_valid.columns)
        .sort_values("mean", ascending=False))
print(perm.round(4))
# Hoán vị trên CỘT GỐC (trước tiên xử lý) -> importance theo ngôn ngữ nghiệp vụ
```

10.3. SHAP (SHapley Additive exPlanations)

Với mỗi dự đoán: $f(x) = \phi_0 + \sum_{j=1}^M \phi_j$, trong đó ϕ_0 là giá trị kỳ vọng (base value) và ϕ_j là đóng góp của feature j . Với mô hình cây, giá trị SHAP mặc định nằm trên thang **log-odds**.

```
import shap

prep, clf = pipe.named_steps["prep"], pipe.named_steps["clf"]
X_valid_t = pd.DataFrame(prep.transform(X_valid), columns=prep.get_feature_names_out(),
                        index=X_valid.index)

explainer = shap.TreeExplainer(clf)
sample = X_valid_t.sample(2000, random_state=0)
sv = explainer(sample) # shap.Explanation

# 1) Global: beeswarm - tầm quan trọng + chiều tác động
shap.plots.beeswarm(sv, max_display=15)

# 2) Global: bar - mean |SHAP|
shap.plots.bar(sv, max_display=15)

# 3) Dependence: quan hệ phi tuyến + tương tác
shap.plots.scatter(sv[:, "tenure_months"], color=sv[:, "monthly_charges"])

# 4) Local: gia'i thích 1 khách hàng
i = int(np.argmax(clf.predict_proba(sample)[:, 1]))
shap.plots.waterfall(sv[i], max_display=10)
```

Nếu `sv.values` có 3 chiều (một số phiên bản trả về cho cả 2 lớp), dùng `sv[:, :, 1]` để lấy lớp positive.

Sinh "lý do" dễ hiểu cho người dùng cuối (reason codes)

```
def top_reasons(sv_row, k: int = 3) -> list[str]:
    contrib = pd.Series(sv_row.values, index=sv_row.feature_names)
    top = contrib[contrib > 0].nlargest(k)
    return [f"{feat} = {sv_row.data[list(sv_row.feature_names).index(feat)]:.2f} "
            f"({val:.2f} log-odds)" for feat, val in top.items()]

print(top_reasons(sv[i]))
# ['support_calls = 1.80 (+0.61 log-odds)', 'contract_month-to-month = 1.00 (+0.55 log-odds)', ...]
```

Trong production, ánh xạ tên feature kỹ thuật sang câu tiếng Việt: "support_calls" → "Gọi tổng đài hỗ trợ nhiều lần".

10.4. Partial Dependence (PDP), ICE và ALE

```
from sklearn.inspection import PartialDependenceDisplay

fig, ax = plt.subplots(figsize=(14, 4))
PartialDependenceDisplay.from_estimator(
    pipe, X_valid.sample(3000, random_state=0),
    features=["tenure_months", "monthly_charges", "support_calls"],
    kind="both",          # "average" = PDP, "individual" = ICE, "both" = cả hai
    subsample=200, centered=True, random_state=0, ax=ax,
)
```

- **PDP** giả định feature độc lập → sai lệch khi feature tương quan (tạo ra tổ hợp phi thực tế như tenure=1 nhưng total_charges rất lớn).
- **ALE** (Accumulated Local Effects — thư viện `alibi`, `PyALE`) khắc phục vấn đề này.
- **ICE** khác nhau nhiều giữa các dòng → có tương tác mạnh.

10.5. Mô hình "glass-box" — giải thích được từ bản chất

```
# Explainable Boosting Machine (Microsoft InterpretML): GAM + boosting, độ chính xác gần GBM
from interpret.glassbox import ExplainableBoostingClassifier

ebm = ExplainableBoostingClassifier(interactions=10, random_state=0)
# ebm.fit(X_train_clean, y_train); ebm.explain_global().visualize()
```

Logistic Regression + WoE (scorecard) vẫn là chuẩn mực trong ngân hàng nhờ tính minh bạch tuyệt đối.

10.6. Fairness — công bằng giữa các nhóm

Tiêu chí	Định nghĩa	Ý nghĩa
Demographic parity	$P(\hat{Y} = 1 \mid A = a)$ bằng nhau giữa các nhóm	Tỷ lệ được chọn như nhau
Equal opportunity	TPR bằng nhau giữa các nhóm	Người "xứng đáng" có cơ hội như nhau
Equalized odds	TPR và FPR bằng nhau	
Predictive parity	Precision bằng nhau	
Calibration within groups	Xác suất được calibrate trong từng nhóm	

Định lý bất khả thi (Kleinberg et al., 2016; Chouldechova, 2017): khi tỷ lệ cơ sở khác nhau giữa các nhóm, không thể đồng thời thỏa mãn calibration và equalized odds. **Chọn tiêu chí là quyết định nghiệp vụ/đạo đức**, cần stakeholder tham gia.

```

from fairlearn.metrics import (MetricFrame, demographic_parity_difference,
                              equalized_odds_difference, selection_rate)
from sklearn.metrics import precision_score, recall_score

threshold = 0.35
y_pred = (pipe.predict_proba(X_valid)[: , 1] >= threshold).astype(int)
age_group = pd.cut(X_valid["age"], bins=[0, 30, 45, 60, 120],
                  labels=["<30", "30-45", "45-60", "60+"]).astype(str)

mf = MetricFrame(
    metrics={"selection_rate": selection_rate, "recall": recall_score,
            "precision": precision_score},
    y_true=y_valid, y_pred=y_pred, sensitive_features=age_group,
)
print(mf.by_group.round(3))
print("Chênh lệch lớn nhất:\n", mf.difference().round(3))
print("Demographic parity diff:",
      round(demographic_parity_difference(y_valid, y_pred, sensitive_features=age_group), 3))
print("Equalized odds diff:",
      round(equalized_odds_difference(y_valid, y_pred, sensitive_features=age_group), 3))

```

Giảm thiểu bias: (1) **Pre-processing** — cân bằng lại dữ liệu, loại proxy feature; (2) **In-processing** — ràng buộc fairness khi huấn luyện (`fairlearn.reductions.ExponentiatedGradient`); (3) **Post-processing** — ngưỡng khác nhau theo nhóm (`fairlearn.postprocessing.ThresholdOptimizer`) — cần cân nhắc pháp lý.

10.7. Model Card — tài liệu hóa mô hình

```

# Model Card: churn-classifier v3
- **Mục đích sử dụng:** xếp hạng khách hàng có nguy cơ rời bỏ trong 30 ngày đề'CSKH liên hệ.
- **Không dùng cho:** quyết định từ chối dịch vụ, định giá cá nhân hóa.
- **Dữ liệu huấn luyện:** snapshot 01/2023–11/2023, 1.2M khách hàng; nhấn = hủy trong 30 ngày.
- **Hiệu năng (test out-of-time 12/2023–02/2024):** PR-AUC 0.61 [0.59–0.63], Recall@Top10% 0.38.
- **Ngưỡng:** 0.35 (tối đa lợi nhuận kỳ vọng).
- **Fairness:** chênh lệch recall giữa các nhóm tuổi ≤ 0.05.
- **Hạn chế:** kém chính xác với khách < 3 tháng (thiếu lịch sử); chưa kiểm định cho khách doanh nghiệp.
- **Giám sát:** PSI hằng tuần; retrain khi PR-AUC giảm > 10% hoặc PSI > 0.25.
- **Chủ sở hữu:** Data Science Team – liên hệ: ds-team@company.vn

```

Checklist Chương 10

- ☐ Importance tính trên validation (permutation/SHAP), không chỉ split-based trên train.
- ☐ Hướng tác động của feature hợp lý về nghiệp vụ (sanity check).
- ☐ Có reason codes cho dự đoán cá nhân nếu người dùng cuối cần.
- ☐ Đã đo fairness theo các nhóm nhạy cảm liên quan.
- ☐ Có Model Card ghi rõ mục đích, hạn chế, chủ sở hữu.

CHƯƠNG 11. TRỰC QUAN HÓA DỮ LIỆU & KẾT QUẢ

11.1. Nguyên tắc thiết kế biểu đồ

1. **Một biểu đồ — một thông điệp.** Tiêu đề nên là kết luận ("Khách hợp đồng tháng churn gấp 3 lần"), không phải mô tả ("Churn theo hợp đồng").
2. **Tối đa hóa tỷ lệ data-ink** (Tufte): bỏ viền, lưới nặng, hiệu ứng 3D, màu thừa.
3. **Trục bắt đầu từ 0 với biểu đồ cột.** Biểu đồ đường có thể không.
4. **Màu có chủ đích:** xám cho ngữ cảnh, một màu nhấn cho điểm cần chú ý; bảng màu thân thiện người mù màu (viridis, cividis, palette colorblind).
5. **Ghi chú trực tiếp** (direct labeling) thay vì bắt người đọc dò chú thích.
6. Luôn ghi **đơn vị, nguồn dữ liệu, thời gian, cỡ mẫu.**

11.2. Chọn biểu đồ theo mục đích

Mục đích	Biểu đồ phù hợp	Tránh
Phân phối 1 biến số	Histogram, KDE, box/violin, ECDF	Pie chart
So sánh phân phối giữa nhóm	Box, violin, ridgeline, ECDF chồng	Nhiều histogram chồng đặc
So sánh giá trị giữa các nhóm	Bar (ngang nếu nhãn dài), dot plot	Pie > 5 phần, 3D bar
Thành phần (tỷ lệ)	Stacked bar 100%, treemap	Pie nhiều lát
Xu hướng theo thời gian	Line, area	Bar cho chuỗi dài
Quan hệ 2 biến số	Scatter (+ hexbin khi nhiều điểm), regression line	
Tương quan nhiều biến	Heatmap, pairplot	
Dữ liệu địa lý	Choropleth, bubble map	
Đánh giá mô hình	ROC, PR, calibration, confusion matrix, lift, residual	

11.3. Thiết lập style chung cho toàn dự án

```
# src/churn/viz/style.py
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

PALETTE = {"primary": "#1f5fa8", "accent": "#d1495b", "neutral": "#9aa5b1", "good": "#2a9d8f"}
```

```
def set_style() -> None:
    sns.set_theme(style="whitegrid", context="notebook", palette="colorblind")
    mpl.rcParams.update({
        "figure.figsize": (10, 5.5), "figure.dpi": 110, "savefig.dpi": 200,
        "savefig.bbox": "tight", "axes.spines.top": False, "axes.spines.right": False,
        "axes.titleweight": "bold", "axes.titlesize": 13, "axes.titlelocation": "left",
        "font.family": "DejaVu Sans", # hỗ trợ tiếng Việt có dấu
    })

def save(fig, name: str) -> None:
    fig.savefig(f"reports/figures/{name}.png")
    fig.savefig(f"reports/figures/{name}.svg")
```

11.4. Biểu đồ EDA (matplotlib + seaborn)

```
import numpy as np
import pandas as pd

from churn.data.synthetic import make_churn_data
from churn.features.clean import clean_churn
from churn.viz.style import PALETTE, mpl, plt, save, set_style, sns

set_style()
df = clean_churn(make_churn_data())

# 1) Phân phối + so sánh theo target: histogram chồng mật độ
fig, ax = plt.subplots()
sns.histplot(data=df, x="monthly_charges", hue="churn", stat="density", common_norm=False,
             element="step", bins=50, ax=ax)
ax.set(title="Khách churn có cước tháng cao hơn", xlabel="Cước tháng (nghìn VNĐ)",
       ylabel="Mật độ")

# 2) ECDF – so sánh phân phối chính xác hơn histogram (không phụ thuộc số bin)
fig, ax = plt.subplots()
sns.ecdfplot(data=df, x="tenure_months", hue="churn", ax=ax)
ax.set(title="50% khách churn có thời gian gắn bó < 1 năm")

# 3) Tỷ lệ churn theo nhóm, có khoảng tin cậy (seaborn tự bootstrap CI)
fig, ax = plt.subplots()
order = df.groupby("contract")["churn"].mean().sort_values(ascending=False).index
sns.barplot(data=df, x="contract", y="churn", order=order, errorbar=("ci", 95),
            color=PALETTE["primary"], ax=ax)
ax.yaxis.set_major_formatter(mpl.ticker.PercentFormatter(1.0))
for c in ax.containers:
    ax.bar_label(c, fmt=lambda v: f"{v:.1%}", padding=3)
ax.set(title="Hợp đồng theo tháng có tỷ lệ churn cao nhất", xlabel="", ylabel="Tỷ lệ churn")

# 4) Violin theo nhóm
fig, ax = plt.subplots()
sns.violinplot(data=df, x="contract", y="support_calls", hue="churn", split=True,
               inner="quart", ax=ax)

# 5) Heatmap tương quan (tam giác dưới)
num = df.select_dtypes("number").drop(columns=["churn"])
corr = num.corr(method="spearman")
fig, ax = plt.subplots(figsize=(8, 6.5))
sns.heatmap(corr, mask=np.triu(np.ones_like(corr, bool)), annot=True, fmt=".2f",
            cmap="RdBu_r", center=0, vmin=-1, vmax=1, square=True, linewidths=0.5, ax=ax)
```

```

ax.set_title("Tương quan Spearman")

# 6) Pairplot (lấy mẫu để nhanh)
sns.pairplot(df.sample(1500, random_state=0),
              vars=["tenure_months", "monthly_charges", "support_calls"],
              hue="churn", corner=True, plot_kws={"s": 8, "alpha": 0.5})

# 7) Chuỗi thời gian với highlight
monthly = df.set_index("signup_date").resample("MS")["churn"].mean()
fig, ax = plt.subplots()
ax.plot(monthly.index, monthly.values, color=PALETTE["neutral"])
peak = monthly.idxmax()
ax.scatter([peak], [monthly.max()], color=PALETTE["accent"], zorder=3)
ax.annotate(f"Đỉnh {monthly.max():.1%}", (peak, monthly.max()), xytext=(10, 10),
            textcoords="offset points")
ax.set(title="Tỷ lệ churn theo cohort tháng đăng ký")

```

11.5. Dashboard biểu đồ đánh giá mô hình (một hình, 6 panel)

```

from sklearn.calibration import calibration_curve
from sklearn.metrics import (ConfusionMatrixDisplay, average_precision_score,
                             precision_recall_curve, roc_auc_score, roc_curve)

def model_dashboard(y_true, proba, threshold: float = 0.5, title: str = "Model evaluation"):
    y_true = np.asarray(y_true)
    fig, axes = plt.subplots(2, 3, figsize=(18, 10))
    fig.suptitle(title, fontsize=16, fontweight="bold")

    # (1) ROC
    fpr, tpr, _ = roc_curve(y_true, proba)
    ax = axes[0, 0]
    ax.plot(fpr, tpr, label=f"AUC = {roc_auc_score(y_true, proba):.3f}")
    ax.plot([0, 1], [0, 1], "--", color="grey")
    ax.set(title="ROC curve", xlabel="FPR", ylabel="TPR"); ax.legend(loc="lower right")

    # (2) Precision-Recall
    prec, rec, _ = precision_recall_curve(y_true, proba)
    ax = axes[0, 1]
    ax.plot(rec, prec, label=f"AP = {average_precision_score(y_true, proba):.3f}")
    ax.axhline(y_true.mean(), ls="--", color="grey", label=f"Baseline = {y_true.mean():.3f}")
    ax.set(title="Precision-Recall curve", xlabel="Recall", ylabel="Precision"); ax.legend()

    # (3) Confusion matrix tại ngưỡng
    ConfusionMatrixDisplay.from_predictions(y_true, (proba >= threshold).astype(int),
                                           display_labels=["stay", "churn"], cmap="Blues",
                                           colorbar=False, ax=axes[0, 2])
    axes[0, 2].set_title(f"Confusion matrix @ {threshold:.2f}")

    # (4) Phân phối điểm dự đoán theo lớp
    ax = axes[1, 0]
    ax.hist(proba[y_true == 0], bins=50, alpha=0.6, density=True, label="stay")
    ax.hist(proba[y_true == 1], bins=50, alpha=0.6, density=True, label="churn")
    ax.axvline(threshold, color="k", ls="--")
    ax.set(title="Phân phối điểm dự đoán", xlabel="P(churn)"); ax.legend()

    # (5) Calibration
    frac_pos, mean_pred = calibration_curve(y_true, proba, n_bins=10, strategy="quantile")
    ax = axes[1, 1]
    ax.plot(mean_pred, frac_pos, "o-"); ax.plot([0, 1], [0, 1], "--", color="grey")

```

```

ax.set(title="Calibration (reliability)", xlabel="Xác suất dự đoán", ylabel="Tỷ lệ thực tế")

# (6) Cumulative gain
order = np.argsort(-proba)
gains = np.cumsum(y_true[order]) / y_true.sum()
pct = np.arange(1, len(y_true) + 1) / len(y_true)
ax = axes[1, 2]
ax.plot(pct, gains, label="Model"); ax.plot([0, 1], [0, 1], "--", color="grey", label="Random")
ax.set(title="Cumulative gain", xlabel="% khách được liên hệ", ylabel="% churn bắt được")
ax.legend()

fig.tight_layout()
return fig

# fig = model_dashboard(y_valid, proba_valid, threshold=0.35); save(fig, "model_dashboard")

```

11.6. Biểu đồ tương tác với Plotly

```

import plotly.express as px
import plotly.graph_objects as go

s = df.sample(3000, random_state=0).assign(label=lambda d: d["churn"].map({0: "stay", 1: "churn"}))
fig = px.scatter(s, x="tenure_months", y="monthly_charges", color="label",
                hover_data=["customer_id", "contract"], opacity=0.6,
                title="Tenure vs Cước tháng", template="plotly_white")
fig.write_html("reports/figures/scatter.html", include_plotlyjs="cdn")

# Funnel / sunburst cho phân khúc
seg = df.groupby(["contract", "payment_method"], observed=True).agg(n=("churn", "size"),
                                                                    churn_rate=("churn",
                                                                    "mean")).reset_index()
fig = px.sunburst(seg, path=["contract", "payment_method"], values="n", color="churn_rate",
                  color_continuous_scale="RdYlGn_r")

```

11.7. Dashboard ứng dụng với Streamlit

```

# app/dashboard.py - chạy: streamlit run app/dashboard.py
import joblib
import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(page_title="Churn Monitor", layout="wide")

@st.cache_resource
def load_model():
    return joblib.load("models/model.joblib")

@st.cache_data(ttl=3600)
def load_data() -> pd.DataFrame:
    return pd.read_parquet("data/processed/scoring_latest.parquet")

model, data = load_model(), load_data()

```

```

data["score"] = model.predict_proba(data)[: , 1]

st.title("📊 Customer Churn Monitor")
threshold = st.sidebar.slider("Ngưỡng rủi ro", 0.05, 0.95, 0.35, 0.05)
contracts = st.sidebar.multiselect("Loại hợp đồng", sorted(data["contract"].dropna().unique()))
view = data[data["contract"].isin(contracts)] if contracts else data

c1, c2, c3 = st.columns(3)
c1.metric("Số khách", f"{len(view):,}")
c2.metric("Nguy cơ cao", f"{(view['score'] >= threshold).sum():,}")
c3.metric("Điểm trung bình", f"{view['score'].mean():.1%}")

st.plotly_chart(px.histogram(view, x="score", nbins=50, title="Phân phối điểm rủi ro"),
                use_container_width=True)
st.dataframe(view.nlargest(100, "score")[["customer_id", "contract", "tenure_months", "score"]])
st.download_button("Tải danh sách gọi điện (CSV)",
                  view[view["score"] >= threshold].to_csv(index=False).encode("utf-8"),
                  file_name="call_list.csv")

```

Công cụ BI cho người dùng nghiệp vụ: **Power BI, Tableau, Looker, Apache Superset, Metabase.**

11.8. Kể chuyện bằng dữ liệu (Data Storytelling)

Cấu trúc trình bày kết quả cho lãnh đạo:

1. **Bối cảnh** — vấn đề kinh doanh và vì sao quan trọng (con số tiền).
2. **Phát hiện chính** — 3 insight, mỗi insight 1 biểu đồ có tiêu đề là kết luận.
3. **Giải pháp** — mô hình làm gì, hiệu quả so với cách hiện tại (lift, lợi nhuận kỳ vọng).
4. **Hành động đề xuất** — ai làm gì, khi nào, đo lường thế nào.
5. **Rủi ro & bước tiếp theo.**

Checklist Chương 11

- ☐ Mỗi biểu đồ có tiêu đề là thông điệp, có đơn vị và nguồn.
- ☐ Loại biểu đồ phù hợp mục đích; tránh pie nhiều lát, 3D.
- ☐ Bảng màu thống nhất, thân thiện người mù màu; font hỗ trợ tiếng Việt.
- ☐ Có dashboard đánh giá mô hình chuẩn (ROC, PR, CM, calibration, gain).
- ☐ Biểu đồ được lưu tự động (PNG + SVG) từ code, không chụp màn hình thủ công.

CHƯƠNG 12. MLOps — ĐƯA MÔ HÌNH VÀO PRODUCTION VÀ GIỮ NÓ ĐÚNG

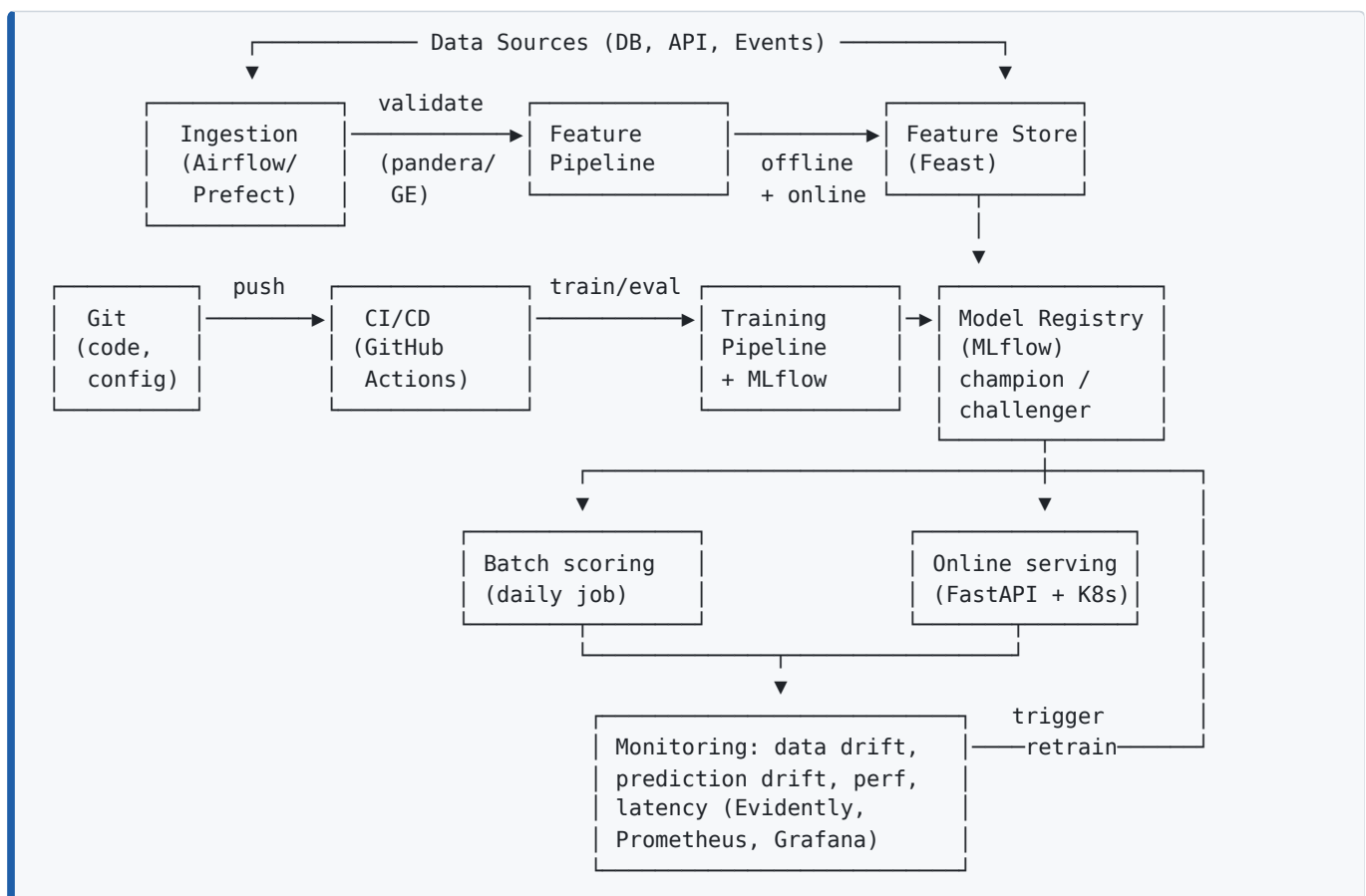
"Chỉ một phần nhỏ của hệ thống ML thực tế là code ML." — Sculley et al., Hidden Technical Debt in Machine Learning Systems (NeurIPS 2015).

12.1. Mức độ trưởng thành MLOps (theo Google Cloud)

Mức	Đặc điểm	Dấu hiệu
Level 0 — Thủ công	Notebook → file model → gửi cho kỹ sư deploy	Không tái lập được; retrain vài tháng/lần; không giám sát
Level 1 — Tự động hóa pipeline ML	Pipeline huấn luyện tự động, có feature store, model registry, giám sát, retrain theo trigger	Continuous Training (CT)
Level 2 — CI/CD cho pipeline	Thay đổi code pipeline được test, build, deploy tự động	CI/CD/CT đầy đủ

Mục tiêu thực tế cho phần lớn đội: **Level 1 vững chắc**.

12.2. Kiến trúc tham chiếu



12.3. Experiment Tracking & Model Registry với MLflow

```
# Server tracking (production: backend Postgres + artifact S3)
mlflow server --backend-store-uri postgresql://mlflow:***@db/mlflow \
  --artifacts-destination s3://ml-artifacts/mlflow --host 0.0.0.0 --port 5000
```

```
import mlflow
from mlflow import MlflowClient

mlflow.set_tracking_uri("http://mlflow:5000")
mlflow.set_experiment("churn-prediction")

# Autolog: tự động log params, metrics, model cho sklearn/lightgbm/xgboost
mlflow.autolog(log_input_examples=True, silent=True)

with mlflow.start_run(run_name="lgbm-optuna-best", tags={"team": "ds", "data_version": "2026-10"}):
    pipe.fit(X_train, y_train)
    mlflow.log_metric("valid_pr_auc", average_precision_score(y_valid, pipe.predict_proba(X_valid)[:, 1]))
    mlflow.log_artifact("reports/figures/model_dashboard.png")
    mlflow.log_dict(study.best_params, "best_params.json")
```

Quy trình promote mô hình: Champion / Challenger

(src/churn/models/registry.py)

```
client = MlflowClient()
MODEL = "churn-classifier"

def promote_if_better(candidate_version: str, metric: str = "test_pr_auc",
                     min_gain: float = 0.005) -> bool:
    """So sánh challenger với champion hiện tại; chỉ promote nếu tốt hơn rõ rệt."""
    cand = client.get_model_version(MODEL, candidate_version)
    cand_score = client.get_run(cand.run_id).data.metrics[metric]
    try:
        champ = client.get_model_version_by_alias(MODEL, "champion")
        champ_score = client.get_run(champ.run_id).data.metrics[metric]
    except Exception:
        # chưa có champion
        champ_score = float("-inf")

    if cand_score >= champ_score + min_gain:
        client.set_registered_model_alias(MODEL, "champion", candidate_version)
        client.set_model_version_tag(MODEL, candidate_version, "validated_by", "ci-gate")
        return True
    client.set_registered_model_alias(MODEL, "challenger", candidate_version)
    return False

# Nạp mô hình theo alias – code serving không cần biết số version
model = mlflow.sklearn.load_model(f"models:{MODEL}@champion")
```

12.4. Testing hệ thống ML

Tầng	Kiểm tra gì	Công cụ
Unit test	Hàm làm sạch, transformer, feature	pytest
Data test	Schema, miền giá trị, null, volume, freshness	pandera , Great Expectations
Model test — hành vi	Invariance, directional, minimum functionality (CheckList, Ribeiro 2020)	pytest
Model test — chất lượng	Metric $\geq$ ngưỡng; không tệ hơn champion; fairness	pytest + MLflow
Integration test	Pipeline end-to-end trên dữ liệu mẫu	pytest
API test	Schema request/response, lỗi đầu vào, độ trễ	httpx , locust

```
# tests/unit/test_features.py
import numpy as np
import pandas as pd
import pytest

from churn.features.clean import clean_churn
from churn.features.build import Winsorizer

def test_winsorizer_clips_to_train_quantiles():
```

```

X_train = np.arange(100, dtype=float).reshape(-1, 1)
w = Winsorizer(0.05, 0.95).fit(X_train)
out = w.transform(np.array([[ -1000.0], [ 50.0], [1000.0]]))
assert out[0, 0] == pytest.approx(np.quantile(X_train, 0.05))
assert out[1, 0] == 50.0
assert out[2, 0] == pytest.approx(np.quantile(X_train, 0.95))

```

```

def test_clean_normalizes_contract_and_dedups():
    raw = pd.DataFrame({
        "customer_id": ["C0000001", "C0000001"], "signup_date": ["2024-01-01", "2024-01-01"],
        "contract": [" Month-to-Month ", " Month-to-Month "], "payment_method": ["cash", "cash"],
        "region": ["R01", "R01"], "age": [30, 30], "monthly_charges": [50.0, 50.0],
    })
    out = clean_churn(raw)
    assert len(out) == 1
    assert out.loc[0, "contract"] == "month-to-month"

```

```
# tests/model/test_model_behavior.py
```

```

import joblib
import pandas as pd
import pytest

```

```

MODEL = joblib.load("models/model.joblib")
BASE = pd.DataFrame([
    "customer_id": "C0000001", "signup_date": pd.Timestamp("2024-01-01"), "age": 35.0,
    "tenure_months": 24, "monthly_charges": 70.0, "total_charges": 1680.0,
    "contract": "one-year", "payment_method": "credit-card", "region": "r01",
    "support_calls": 1, "data_usage_gb": 12.0,
])

```

```

def score(**overrides) -> float:
    return float(MODEL.predict_proba(BASE.assign(**overrides))[:, 1][0])

```

```

def test_invariance_to_customer_id():
    """Đô'i ID không được làm thay đô'i dự đoán."""
    assert score(customer_id="C9999999") == pytest.approx(score())

```

```

def test_directional_support_calls_increase_risk():
    """Nhiê'u cuộc gọi khiê'u nại hơn -> ru'i ro không được gia'm."""
    assert score(support_calls=8) >= score(support_calls=0)

```

```

def test_directional_contract():
    assert score(contract="month-to-month") > score(contract="two-year")

```

```

def test_handles_missing_and_unseen_values():
    p = score(age=None, payment_method="crypto", region="r99")
    assert 0.0 <= p <= 1.0

```

```
# tests/model/test_quality_gate.py – chạy trong CI sau khi train
```

```
import json
```

```
THRESHOLDS = {"valid_pr_auc": 0.45, "valid_roc_auc": 0.75}
```

```
def test_quality_gate():
```

```
metrics = json.load(open("reports/metrics.json"))
for name, minimum in THRESHOLDS.items():
    assert metrics[name] >= minimum, f"{name}={metrics[name]:.4f} < {minimum}"
```

12.5. Serving

Lựa chọn kiểu triển khai

Kiểu	Độ trễ	Khi nào dùng	Công nghệ
Batch	Giờ/ngày	Danh sách gọi điện hàng ngày, báo cáo	Airflow/Prefect + Spark/pandas, ghi ra DB
Online (request/response)	ms	Chấm điểm khi khách thao tác, chống gian lận	FastAPI, BentoML, KServe, Triton
Streaming	giây	Sự kiện liên tục (clickstream)	Kafka + Flink/Spark Streaming
Edge / on-device	ms, offline	Mobile, IoT	ONNX Runtime, TFLite, Core ML

Batch scoring

```
# src/churn/models/predict.py
import argparse
from datetime import date

import mlflow
import pandas as pd

from churn.data.validate import validate
from churn.features.clean import clean_churn

def batch_score(input_path: str, output_path: str, threshold: float,
                model_uri: str = "models:/churn-classifier@champion") -> None:
    model = mlflow.sklearn.load_model(model_uri)
    df = clean_churn(pd.read_parquet(input_path))
    validate(df.assign(churn=0)) # validate input trước khi chấ'm điề'm
    out = df[["customer_id"]].copy()
    out["score"] = model.predict_proba(df)[ :, 1]
    out["is_high_risk"] = out["score"] >= threshold
    out["score_date"] = date.today().isoformat()
    out["model_uri"] = model_uri
    out.to_parquet(output_path, index=False) # lưu đê' giám sát & audit

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--input"); p.add_argument("--output"); p.add_argument("--threshold",
    type=float)
    a = p.parse_args()
    batch_score(a.input, a.output, a.threshold)
```

Online serving với FastAPI

```
# src/churn/serving/api.py
import logging
import time
import uuid
from contextlib import asynccontextmanager
from datetime import datetime
from typing import Literal

import joblib
import pandas as pd
from fastapi import FastAPI, HTTPException
from prometheus_client import Counter, Histogram, make_asgi_app
from pydantic import BaseModel, Field

log = logging.getLogger("churn-api")
MODEL_PATH = "models/model.joblib"
MODEL_VERSION = "v3"
THRESHOLD = 0.35

PREDICTIONS = Counter("churn_predictions_total", "Số dự đoán", ["model_version", "label"])
LATENCY = Histogram("churn_predict_latency_seconds", "Độ trễ dự đoán",
                    buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1))

state: dict = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    state["model"] = joblib.load(MODEL_PATH) # nạp 1 lần khi khởi động
    log.info("Model loaded: %s", MODEL_VERSION)
    yield
    state.clear()

app = FastAPI(title="Churn Prediction API", version=MODEL_VERSION, lifespan=lifespan)
app.mount("/metrics", make_asgi_app()) # Prometheus scrape endpoint

class Customer(BaseModel):
    customer_id: str = Field(pattern=r"^C\d{7}$")
    signup_date: datetime
    age: float | None = Field(None, ge=18, le=100)
    tenure_months: int = Field(ge=0, le=600)
    monthly_charges: float = Field(gt=0, le=2000)
    total_charges: float = Field(ge=0)
    contract: Literal["month-to-month", "one-year", "two-year"]
    payment_method: str | None = None
    region: str
    support_calls: int = Field(ge=0)
    data_usage_gb: float | None = Field(None, ge=0)

class Prediction(BaseModel):
    customer_id: str
    churn_probability: float
    is_high_risk: bool
    model_version: str
    request_id: str

@app.get("/health")
```

```

def health() -> dict:
    return {"status": "ok"}

@app.get("/ready")
def ready() -> dict:
    if "model" not in state:
        raise HTTPException(503, "Model not loaded")
    return {"status": "ready", "model_version": MODEL_VERSION}

@app.post("/predict", response_model=list[Prediction])
def predict(customers: list[Customer]) -> list[Prediction]:
    if not customers or len(customers) > 1000:
        raise HTTPException(422, "Batch size phải trong [1, 1000]")
    start = time.perf_counter()
    df = pd.DataFrame([c.model_dump() for c in customers])
    df["signup_date"] = pd.to_datetime(df["signup_date"]).dt.tz_localize(None)
    try:
        proba = state["model"].predict_proba(df)[ :, 1]
    except Exception as exc:
        log.exception("Prediction failed")
        raise HTTPException(500, "Prediction error") from exc
    LATENCY.observe(time.perf_counter() - start)

    request_id = str(uuid.uuid4())
    results = []
    for cid, p in zip(df["customer_id"], proba):
        high = bool(p >= THRESHOLD)
        PREDICTIONS.labels(MODEL_VERSION, str(high)).inc()
        results.append(Prediction(customer_id=cid, churn_probability=round(float(p), 6),
                                   is_high_risk=high, model_version=MODEL_VERSION,
                                   request_id=request_id))
    # Log input + output (không log PII) để giám sát drift & làm dữ liệu huấn luyện sau này
    log.info("request_id=%s n=%d mean_p=%.4f", request_id, len(results), float(proba.mean()))
    return results

```

```

# tests/api/test_api.py
from fastapi.testclient import TestClient

from churn.serving.api import app

PAYLOAD = [{"customer_id": "C0000001", "signup_date": "2024-01-01T00:00:00", "age": 30,
            "tenure_months": 3, "monthly_charges": 90.0, "total_charges": 270.0,
            "contract": "month-to-month", "payment_method": "cash", "region": "r01",
            "support_calls": 4, "data_usage_gb": 5.0}]

def test_predict_ok():
    with TestClient(app) as client:
        # chạy lifespan -> nạp model
        r = client.post("/predict", json=PAYLOAD)
        assert r.status_code == 200
        assert 0 <= r.json()[0]["churn_probability"] <= 1

def test_predict_rejects_invalid_contract():
    with TestClient(app) as client:
        bad = [{"**PAYLOAD[0], "contract": "lifetime"}]
        assert client.post("/predict", json=bad).status_code == 422

```

12.6. Đóng gói với Docker

```
# Dockerfile – multi-stage, non-root, image nhỏ
FROM python:3.11-slim AS builder
WORKDIR /build
RUN pip install --no-cache-dir uv
COPY pyproject.toml requirements.lock ./
RUN uv pip install --system --no-cache -r requirements.lock
COPY src/ src/
RUN uv pip install --system --no-cache --no-deps .

FROM python:3.11-slim AS runtime
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/* \
    && useradd --create-home --uid 1000 app
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
COPY --from=builder /usr/local/bin /usr/local/bin
WORKDIR /app
COPY models/model.joblib models/model.joblib
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "churn.serving.api:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

`libgomp1` cần cho LightGBM. Trong hệ thống lớn, không "nướng" model vào image mà tải từ registry khi khởi động (tách vòng đời model và code).

```
# docker-compose.yml – môi trường local đầy đủ
services:
  api:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [mlflow]
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.14.1
    command: mlflow server --host 0.0.0.0 --port 5000 --backend-store-uri sqlite:///mlflow.db --
default-artifact-root /mlruns
    ports: ["5000:5000"]
    volumes: [".:/mlruns:/mlruns"]
  prometheus:
    image: prom/prometheus
    volumes: [".:/ops/prometheus.yml:/etc/prometheus/prometheus.yml"]
    ports: ["9090:9090"]
  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
```

12.7. CI/CD với GitHub Actions

```
# .github/workflows/ci.yml
name: ml-ci
on:
  pull_request:
```

```

push:
  branches: [main]

jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11", cache: pip }
      - run: pip install -e ".[dev]"
      - run: ruff check src tests && ruff format --check src tests
      - run: mypy src
      - run: pytest tests/unit tests/data -q

  train-and-gate:
    needs: lint-test
    runs-on: ubuntu-latest
    env:
      MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
      AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11", cache: pip }
      - run: pip install -e ".[dev]" "dvc[s3]"
      - run: dvc pull data/raw/churn.parquet
      - run: python -m churn.models.train --config configs/train.yaml
      - run: pytest tests/model -q # behavior tests + quality gate
      - uses: actions/upload-artifact@v4
        with: { name: model, path: models/ }

  build-push:
    if: github.ref == 'refs/heads/main'
    needs: train-and-gate
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with: { name: model, path: models/ }
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: "${ github.actor }", password: "${ secrets.GITHUB_TOKEN }" }
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${ github.repository }/churn-api:${ github.sha }

```

12.8. Chiến lược triển khai an toàn

Chiến lược	Cách làm	Ưu	Nhược
Shadow	Mô hình mới nhận traffic thật nhưng kết quả không được dùng; so sánh với mô hình cũ	Không rủi ro cho người dùng	Tốn tài nguyên; không đo được hiệu quả kinh doanh

Chiến lược	Cách làm	Ưu	Nhược
Canary	5% → 25% → 100% traffic, theo dõi metric từng bước	Giới hạn thiệt hại	Cần hạ tầng định tuyến
Blue/Green	Hai môi trường song song, chuyển traffic tức thời	Rollback nhanh	Gấp đôi tài nguyên
A/B test	Chia ngẫu nhiên người dùng, đo KPI kinh doanh	Đo được tác động nhân quả	Cần thời gian đủ cỡ mẫu (chương 4.5)
Multi-armed bandit	Tự động dồn traffic cho mô hình tốt hơn	Ít "chi phí cơ hội"	Phức tạp hơn để phân tích

Luôn có kế hoạch rollback và fallback (ví dụ: rule-based baseline khi mô hình lỗi hoặc timeout).

12.9. Giám sát (Monitoring)

Các loại suy giảm mô hình

Loại	Định nghĩa	Ví dụ	Phát hiện
Data drift (covariate shift)	$P(X)$ thay đổi	Khách hàng trẻ hơn sau chiến dịch marketing	PSI, KS, χ^2 , Jensen-Shannon
Prior/label shift	$P(Y)$ thay đổi	Tỷ lệ churn tăng do đối thủ giảm giá	Theo dõi tỷ lệ dự đoán dương / tỷ lệ nhãn
Concept drift	$P(Y X)$ thay đổi	Quan hệ giữa giá và churn đổi sau quy định mới	Metric hiệu năng khi có nhãn
Data quality issue	Pipeline lỗi	Cột bị null toàn bộ, đổi đơn vị	Data tests, tỷ lệ null, volume
Training-serving skew	Feature tính khác nhau giữa train và serve	Code feature viết 2 lần bằng 2 ngôn ngữ	So sánh phân phối feature log tại serving vs train

Population Stability Index (PSI)

$$PSI = \sum_{i=1}^B (A_i - E_i) \ln \frac{A_i}{E_i}$$

với E_i , A_i là tỷ lệ mẫu trong bin i của tập tham chiếu (train) và hiện tại.

PSI	Diễn giải
< 0.1	Ổn định
0.1 - 0.25	Thay đổi vừa — theo dõi
> 0.25	Thay đổi lớn — điều tra, cân nhắc retrain

```
# src/churn/monitoring/drift.py
import numpy as np
import pandas as pd
```

```

from scipy import stats
from scipy.spatial.distance import jensenshannon

def psi(expected: np.ndarray, actual: np.ndarray, bins: int = 10, eps: float = 1e-6) -> float:
    expected, actual = np.asarray(expected, float), np.asarray(actual, float)
    expected, actual = expected[~np.isnan(expected)], actual[~np.isnan(actual)]
    edges = np.unique(np.quantile(expected, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    e = np.histogram(expected, edges)[0] / len(expected) + eps
    a = np.histogram(actual, edges)[0] / len(actual) + eps
    return float(np.sum((a - e) * np.log(a / e)))

def categorical_psi(expected: pd.Series, actual: pd.Series, eps: float = 1e-6) -> float:
    cats = set(expected.dropna()) | set(actual.dropna())
    e = expected.value_counts(normalize=True).reindex(cats, fill_value=0) + eps
    a = actual.value_counts(normalize=True).reindex(cats, fill_value=0) + eps
    return float(np.sum((a - e) * np.log(a / e)))

def drift_report(reference: pd.DataFrame, current: pd.DataFrame,
                 num_cols: list[str], cat_cols: list[str]) -> pd.DataFrame:
    rows = []
    for c in num_cols:
        r, k = reference[c].dropna(), current[c].dropna()
        rows.append({"feature": c, "type": "num", "psi": psi(r, k),
                     "ks_pvalue": stats.ks_2samp(r, k).pvalue,
                     "null_rate_ref": reference[c].isna().mean(),
                     "null_rate_cur": current[c].isna().mean()})
    for c in cat_cols:
        cats = sorted(set(reference[c].dropna()) | set(current[c].dropna()))
        p = reference[c].value_counts(normalize=True).reindex(cats, fill_value=0)
        q = current[c].value_counts(normalize=True).reindex(cats, fill_value=0)
        rows.append({"feature": c, "type": "cat", "psi": categorical_psi(reference[c], current[c]),
                     "js_distance": float(jensenshannon(p, q, base=2)),
                     "null_rate_ref": reference[c].isna().mean(),
                     "null_rate_cur": current[c].isna().mean(),
                     "new_categories": sorted(set(current[c].dropna()) -
                                             set(reference[c].dropna()))})
    out = pd.DataFrame(rows)
    out["status"] = pd.cut(out["psi"], [-np.inf, 0.1, 0.25, np.inf], labels=["ok", "warn",
    "alert"])
    return out.sort_values("psi", ascending=False)

```

Với dữ liệu rất lớn, KS test gần như luôn cho p-value ≈ 0 dù khác biệt không đáng kể $\rightarrow$ dựa vào **độ lớn khác biệt** (PSI, khoảng cách Wasserstein, JS) hơn là p-value.

Evidently — báo cáo drift tự động

```

# evidently >= 0.7
from evidently import Report
from evidently.preset import DataDriftPreset, DataSummaryPreset

report = Report([DataDriftPreset(), DataSummaryPreset()])
snapshot = report.run(current_data=current_df, reference_data=reference_df)
snapshot.save_html("reports/drift_report.html")

# evidently < 0.7 (API cũ):

```

```
# from evidently.report import Report
# from evidently.metric_preset import DataDriftPreset
# r = Report(metrics=[DataDriftPreset()]); r.run(reference_data=ref, current_data=cur)
# r.save_html("drift.html")
```

Bảng giám sát production đề xuất

Nhóm	Metric	Ngưỡng cảnh báo ví dụ	Tần suất
Hệ thống	p95 latency, error rate, throughput, CPU/RAM	p95 > 200ms; lỗi > 1%	Real-time
Dữ liệu	Null rate, schema, volume, category mới	Null tăng > 5 điểm %; volume $\pm 30\%$	Mỗi batch
Drift input	PSI từng feature (top features theo importance)	PSI > 0.25	Hàng ngày/tuần
Drift output	Phân phối điểm, tỷ lệ dự đoán dương	PSI điểm > 0.1	Hàng ngày
Hiệu năng	PR-AUC, recall, calibration khi có nhãn (trễ 30 ngày)	Giảm > 10% so với baseline	Khi nhãn về
Kinh doanh	Tỷ lệ giữ chân nhóm được liên hệ vs control	—	Hàng tháng

12.10. Retraining & Orchestration

Chiến lược retrain	Mô tả	Phù hợp
Định kỳ (scheduled)	Hàng tuần/tháng	Đơn giản, dễ vận hành — mặc định tốt
Theo trigger	Khi drift/hiệu năng vượt ngưỡng	Môi trường thay đổi bất thường
Online / incremental	Cập nhật liên tục	Gợi ý, quảng cáo, dữ liệu streaming

Quy trình retrain **không bao giờ tự động deploy mù quáng**: train → validate dữ liệu → evaluate trên out-of-time test → so sánh champion (gate) → shadow/canary → promote.

```
# flows/retrain.py – Prefect 2/3
from prefect import flow, task, get_run_logger

@task(retries=3, retry_delay_seconds=60)
def ingest(snapshot: str) -> str:
    ... # tra' về` đường dẫn dữ liệu
    return f"data/raw/churn_{snapshot}.parquet"

@task
def validate_data(path: str) -> str:
    import pandas as pd
    from churn.data.validate import validate
    validate(pd.read_parquet(path))
    return path
```

```

@task
def train(path: str) -> str:
    ...                                     # gọi churn.models.train, tra' về` model version
    return "12"

@task
def gate_and_promote(version: str) -> bool:
    from churn.models.registry import promote_if_better
    return promote_if_better(version)

@flow(name="churn-retrain")
def retrain_flow(snapshot: str):
    log = get_run_logger()
    version = train(validate_data(ingest(snapshot)))
    promoted = gate_and_promote(version)
    log.info("Model v%s promoted=%s", version, promoted)

if __name__ == "__main__":
    retrain_flow.serve(name="monthly", cron="0 2 1 * *", parameters={"snapshot": "latest"})

```

```

# dags/churn_retrain.py - Airflow 2.x (TaskFlow API)
import pendulum
from airflow.decorators import dag, task

@dag(schedule="0 2 1 * *", start_date=pendulum.datetime(2026, 1, 1, tz="Asia/Ho_Chi_Minh"),
      catchup=False, tags=["ml", "churn"], default_args={"retries": 2})
def churn_retrain():
    @task
    def ingest(ds=None) -> str: ...

    @task
    def validate(path: str) -> str: ...

    @task
    def train(path: str) -> str: ...

    @task
    def promote(version: str) -> None: ...

    promote(train(validate(ingest())))

churn_retrain()

```

12.11. Feature Store — chống training-serving skew

Feature store (Feast, Tecton, Databricks/Vertex/SageMaker Feature Store) đảm bảo **cùng một định nghĩa feature** cho cả huấn luyện (offline, point-in-time join) và serving (online, độ trễ thấp).

```

# feature_repo/features.py - Feast
from datetime import timedelta

```

```

from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64

customer = Entity(name="customer", join_keys=["customer_id"])

usage_source = FileSource(path="data/processed/usage_features.parquet",
                           timestamp_field="event_timestamp")

usage_fv = FeatureView(
    name="customer_usage",
    entities=[customer],
    ttl=timedelta(days=90),
    schema=[Field(name="support_calls_90d", dtype=Int64),
            Field(name="avg_bill_6m", dtype=Float32)],
    source=usage_source,
)

# Training: store.get_historical_features(entity_df=labels_with_timestamps, features=[])
# Serving : store.get_online_features(features=[], entity_rows=[{"customer_id": "C0000001"}])

```

12.12. Tối ưu hiệu năng serving

- **ONNX / Treelite:** chuyển mô hình sang định dạng tối ưu (`skl2onnx` , `onnxmltools`), nhanh hơn 2-10 lần.
- **Batch inference** thay vì từng dòng; vectorize tiến xử lý.
- **Cache** kết quả cho input lặp lại; **pre-compute** điểm offline khi có thể.
- **Scale ngang** bằng Kubernetes HPA theo CPU/RPS; GPU chỉ khi cần (deep learning).

12.13. Bảo mật, quyền riêng tư & quản trị

- Secret qua biến môi trường / secret manager (Vault, AWS Secrets Manager) — **không bao giờ** commit vào git.
- Ẩn danh hóa/hash PII trước khi đưa vào pipeline huấn luyện; giới hạn quyền truy cập theo vai trò.
- Ghi lại **lineage**: mô hình version X được train từ dữ liệu version Y, code commit Z, config W.
- Audit log mọi dự đoán quan trọng (input hash, output, model version, thời gian).
- Đánh giá rủi ro AI theo khung pháp lý áp dụng (Nghị định 13/2023/NĐ-CP về dữ liệu cá nhân; EU AI Act nếu liên quan).
- Kiểm tra lỗi hỏng phụ thuộc (`pip-audit`), quét image (`trivy`).

Checklist Chương 12

- ☐ Mọi thí nghiệm được log (params, metrics, artifact, data version, git commit).
- ☐ Model registry với alias champion/challenger; promote qua quality gate tự động.
- ☐ Có unit test, data test, behavior test, quality gate trong CI.
- ☐ API có validate input (pydantic), health/ready, metrics, logging, xử lý lỗi.
- ☐ Docker image non-root, phiên bản cố định; CI/CD build & deploy tự động.
- ☐ Triển khai qua shadow/canary; có rollback & fallback.
- ☐ Giám sát hệ thống, dữ liệu, drift, hiệu năng, KPI kinh doanh với ngưỡng cảnh báo.
- ☐ Có chiến lược retrain và runbook xử lý sự cố.

CHƯƠNG 13. CHECKLIST PRODUCTION & ANTI-PATTERNS

13.1. Checklist tổng hợp trước khi Go-Live

A. Bài toán & dữ liệu

- ☐ Mục tiêu kinh doanh, hành động sau dự đoán, KPI online đã được stakeholder ký duyệt.
- ☐ Định nghĩa nhãn có cửa sổ thời gian rõ ràng; feature đảm bảo point-in-time.
- ☐ Data contract + validation tự động tại ingestion; dữ liệu thô được version.
- ☐ PII được ẩn danh; tuân thủ quy định bảo vệ dữ liệu.

B. Mô hình

- ☐ Vượt baseline nghiệp vụ một cách có ý nghĩa thống kê (CI của chênh lệch không chứa 0).
- ☐ Validation mô phỏng production (out-of-time, group); không có leakage (có test tự động).
- ☐ Ngưỡng được chọn theo chi phí/năng lực vận hành; xác suất đã calibrate nếu dùng trực tiếp.
- ☐ Slice analysis & fairness đã được review.
- ☐ Giải thích mô hình hợp lý về nghiệp vụ; có Model Card.

C. Kỹ thuật

- ☐ Toàn bộ tiền xử lý + mô hình là 1 artifact; tái lập được từ (code commit, data version, config, seed).
- ☐ Test: unit, data, behavior, quality gate, API — chạy trong CI.
- ☐ API: validate input, xử lý giá trị thiếu/lạ, timeout, health/ready, logging không chứa PII.
- ☐ Load test đạt SLA (p95 latency, throughput).
- ☐ Docker image an toàn, phụ thuộc được quét lỗ hổng.

D. Vận hành

- ☐ Dashboard giám sát: hệ thống, dữ liệu, drift, hiệu năng, KPI kinh doanh.
- ☐ Cảnh báo có người nhận (on-call) và runbook xử lý.
- ☐ Kế hoạch rollback + fallback đã được diễn tập.
- ☐ Chiến lược retrain và quy trình promote có gate.
- ☐ Thu thập nhãn/feedback để đánh giá và huấn luyện lại.

13.2. 20 Anti-patterns tôi gặp nhiều nhất

#	Anti-pattern	Hậu quả	Cách đúng
1	Fit scaler/imputer/encoder trên toàn bộ dữ liệu trước split	Metric ảo	Pipeline + CV

#	Anti-pattern	Hậu quả	Cách đúng
2	Random split cho dữ liệu có thời gian	Hiệu năng production thấp hơn nhiều	Out-of-time split
3	SMOTE trước cross-validation	Metric ảo nghiêm trọng	<code>imblearn.pipeline</code>
4	Dùng Accuracy cho dữ liệu mất cân bằng	Mô hình "luôn dự đoán 0" trông tốt	PR-AUC, recall, MCC
5	Ngưỡng mặc định 0.5	Sai lệch chi phí kinh doanh	Threshold tuning theo chi phí
6	Tune trên tập test	Ước lượng lạc quan	Validation riêng / nested CV
7	Feature có sau sự kiện target	Leakage — mô hình vô dụng khi deploy	Kiểm tra thời điểm sinh feature
8	Cùng thực thể ở train và test	Mô hình "nhớ" thực thể	Group split
9	Báo cáo 1 con số không có CI	Quyết định dựa trên nhiều	Bootstrap CI, mean $\pm$ std
10	Notebook là "production code"	Không tái lập, không test	Module + test + CI
11	Code feature viết lại khi serving	Training-serving skew	Dùng chung code/pipeline, feature store
12	Không lưu phiên bản dữ liệu	Không tái lập được kết quả	DVC / snapshot
13	Xóa ngoại lai trên tập test	Đánh giá không trung thực	Chỉ xử lý theo quy tắc học từ train
14	Dùng mô hình phức tạp khi chưa có baseline	Không biết giá trị thật của mô hình	Baseline trước
15	Tin feature importance mặc định của cây	Kết luận sai về yếu tố quan trọng	Permutation importance, SHAP trên validation
16	Diễn giải SHAP/hệ số như nhân quả	Hành động kinh doanh sai	A/B test, causal inference
17	Deploy xong là xong	Mô hình suy giảm âm thầm	Monitoring + retrain
18	Tự động deploy mô hình retrain không có gate	Mô hình tụt production	Quality gate + shadow/canary
19	Hard-code secret, đường dẫn, tham số	Rò rỉ bảo mật, khó thay đổi	<code>.env</code> , config YAML
20	Tối ưu metric offline mà quên KPI online	Mô hình "tốt" nhưng không tạo giá trị	Gắn metric với chi phí, đo bằng A/B test

13.3. Câu hỏi review mô hình (dành cho Tech Lead / Reviewer)

1. Nếu tôi xóa feature quan trọng nhất, hiệu năng giảm bao nhiêu? Feature đó có sẵn tại thời điểm dự đoán không?
2. Phân phối điểm của tập test có giống tập train không (adversarial AUC)?

3. Mô hình hoạt động thế nào với khách hàng mới (cold start), giá trị thiếu, category chưa từng thấy?
4. Cải thiện so với baseline có ý nghĩa thống kê và có giá trị kinh doanh không?
5. Điều gì xảy ra nếu nguồn dữ liệu X ngừng cập nhật 3 ngày?
6. Ai được cảnh báo khi mô hình suy giảm, và họ làm gì?
7. Làm sao rollback trong 5 phút?

PHỤ LỤC

Phụ lục A. Cheatsheet thư viện theo giai đoạn

Giai đoạn	Thư viện	Dùng để
Thao tác dữ liệu	pandas , polars , duckdb , pyarrow	DataFrame, SQL trên file, Parquet
Dữ liệu lớn	pyspark , dask , ray	Xử lý phân tán
Thu thập	SQLAlchemy , httpx , requests , tenacity , BeautifulSoup , playwright , scrapy , confluent-kafka	DB, API, scraping, streaming
Chất lượng dữ liệu	pandera , great_expectations , soda-core	Data contract, kiểm tra
Versioning	dvc , lakeFS , Delta Lake	Phiên bản dữ liệu
EDA	ydata-profiling , sweetviz , missingno , dtale	Báo cáo tự động
Thống kê	scipy.stats , statsmodels , pingouin	Kiểm định, hồi quy, chuỗi thời gian
Nhân quả	DoWhy , EconML , CausalML	Uplift, causal inference
Tiền xử lý	scikit-learn , category_encoders , feature-engine , imbalanced-learn , optbinning	Pipeline, encoding, resampling
Văn bản tiếng Việt	underthesea , pyvi , transformers (PhoBERT), sentence-transformers	Tách từ, embedding
Mô hình cổ điển	scikit-learn	Linear, tree, SVM, KNN, clustering
Gradient boosting	lightgbm , xgboost , catboost	Mô hình bảng mạnh nhất
Deep learning	pytorch , lightning , tensorflow/keras , transformers	NN, NLP, CV
Chuỗi thời gian	statsforecast , sktime , darts , prophet	Dự báo
Tuning	optuna , ray[tune] , hyperopt	Tối ưu siêu tham số
Giải thích	shap , lime , interpret , alibi , dice-ml	XAI
Fairness	fairlearn , aif360	Đo & giảm bias
Trực quan hóa	matplotlib , seaborn , plotly , altair	Biểu đồ
Dashboard	streamlit , dash , gradio , Superset	Ứng dụng dữ liệu
Tracking/Registry	mlflow , wandb , neptune	Thí nghiệm, mô hình
Serving	fastapi , bentoml , kserve , triton , onnxruntime	API, tối ưu suy luận
Orchestration	airflow , prefect , dagster , kubeflow	Pipeline
Feature store	feast	Offline/online feature
Monitoring	evidently , nannyml , prometheus , grafana	Drift, hiệu năng

Giai đoạn	Thư viện	Dùng để
Chất lượng code	ruff , mypy , pytest , pre-commit	Lint, type, test

Phụ lục B. Bảng chọn metric nhanh

Bài toán?	
Phân loại	
Cân xác suất chính xác?	Log loss, Brier, ECE
Lớp positive hiếm (<10%)?	PR-AUC, Recall@Precision≥x, F-β, MCC
Hành động trên top-K?	Precision@K, Lift@K, Gain
Lớp cân bằng, chi phí lỗi như nhau?	Accuracy, F1, ROC-AUC
Đa lớp	Macro-F1 (lớp hiếm quan trọng) / Weighted-F1
Hồi quy	
Ngoại lai nhiều / cân đề~hiệu?	MAE, MedAE
Phạt nặng lỗi lớn?	RMSE
Sai số tương đối, y > 0?	RMSLE, MAPE (y xa 0), sMAPE, WAPE
Dự báo khoảng / phân vị?	Pinball loss, coverage
Xếp hạng / truy hồi	NDCG@K, MAP@K, MRR, Recall@K, F2
Phân cụm	Silhouette, Davies–Bouldin (+ ARI/NMI nếu có nhãn)

Phụ lục C. Bảng chọn thuật toán

Tiêu chí	Logistic/Linear	Random Forest	GBM (LGBM/XGB/Cat)	Neural Net	KNN	SVM
Hiệu năng trên dữ liệu bảng	Trung bình	Khá	Cao nhất	Khá-Cao (dữ liệu lớn)	Thấp-TB	Khá
Cần scale	Có	Không	Không	Có	Có	Có
Xử lý NaN gốc	Không	Có (sklearn ≥ 1.4)	Có	Không	Không	Không
Giải thích	Rất tốt	TB	TB (SHAP)	Kém	TB	Kém
Tốc độ train	Rất nhanh	Nhanh	Nhanh	Chậm	Không cần train	Chậm (n lớn)
Tốc độ dự đoán	Rất nhanh	TB	Nhanh	Nhanh (GPU)	Chậm	TB
Dữ liệu nhỏ	Tốt	Tốt	Khá	Kém	Tốt	Tốt
Ngoại suy	Có	Không	Không	Có	Không	Hạn chế

Phụ lục D. Thuật ngữ Anh - Việt

English	Tiếng Việt
Exploratory Data Analysis (EDA)	Phân tích khám phá dữ liệu
Feature engineering	Kỹ thuật tạo đặc trưng

English	Tiếng Việt
Data leakage	Rò rỉ dữ liệu
Imputation	Điền giá trị thiếu
Encoding	Mã hóa
Scaling / Normalization / Standardization	Co giãn / Chuẩn hóa min-max / Chuẩn hóa z-score
Cross-validation	Kiểm định chéo
Hyperparameter tuning	Tối ưu siêu tham số
Overfitting / Underfitting	Quá khớp / Chưa khớp
Confusion matrix	Ma trận nhầm lẫn
Precision / Recall	Độ chính xác (dự đoán dương) / Độ bao phủ (độ nhạy)
Calibration	Hiệu chuẩn xác suất
Threshold	Ngưỡng quyết định
Drift	Trôi dạt (thay đổi phân phối)
Model registry	Kho quản lý mô hình
Serving / Inference	Phục vụ mô hình / Suy luận
Champion / Challenger	Mô hình đương nhiệm / Mô hình thách đấu
Point-in-time correctness	Đúng thời điểm (không nhìn trộm tương lai)
Training-serving skew	Lệch giữa huấn luyện và phục vụ

Phụ lục E. Tài liệu tham khảo

1. Géron, A. — *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed., O'Reilly, 2022.
2. Hastie, T., Tibshirani, R., Friedman, J. — *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
3. Kuhn, M., Johnson, K. — *Feature Engineering and Selection*, CRC Press, 2019.
4. Huyen, C. — *Designing Machine Learning Systems*, O'Reilly, 2022.
5. Molnar, C. — *Interpretable Machine Learning*, 2nd ed., 2022
(christophm.github.io/interpretable-ml-book).
6. Kohavi, R., Tang, D., Xu, Y. — *Trustworthy Online Controlled Experiments*, Cambridge, 2020.
7. López de Prado, M. — *Advances in Financial Machine Learning*, Wiley, 2018.
8. Sculley, D. et al. — *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015.
9. Breck, E. et al. — *The ML Test Score: A Rubric for ML Production Readiness*, IEEE Big Data 2017.
10. Grinsztajn, L., Oyallon, E., Varoquaux, G. — *Why do tree-based models still outperform deep learning on tabular data?*, NeurIPS 2022.
11. Ribeiro, M. T. et al. — *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*, ACL 2020.
12. Tài liệu chính thức: scikit-learn.org, lightgbm.readthedocs.io, optuna.org, mlflow.org, fastapi.tiangolo.com, docs.evidentlyai.com.

Hết handbook. Phiên bản 1.0 — 10/2026. Góp ý & cập nhật qua Pull Request.